Interim Report

Level 2



ChainOps

Smart Contract DevSecOps and Operations Platform

Team IntelliCoders

Index	Name
244063P	W.W.A.S.T Fernando
244176N	L.L.K.N Rathnasiri
244087R	J.M.D.S Jayasiri
244092D	J.B.A.I.N Jayawickrama
244059J	F.F Fazreen

Faculty of Information Technology

University of Moratuwa

2026

Interim Report

Level 2

ChainOps

Smart Contract DevSecOps and Operations Platform

Team IntelliCoders

Index	Name
244063P	W.W.A.S.T Fernando
244176N	L.L.K.N Rathnasiri
244087R	J.M.D.S Jayasiri
244092D	J.B.A.I.N Jayawickrama
244059J	F.F Fazreen

Supervised by: Ms MGNN PREAMADASA

Faculty of Information Technology

University of Moratuwa

2026

Abstract

In today's rapidly evolving software development environment, blockchain technology and smart contracts play an important role in developing decentralized applications. However, the smart contract development lifecycle involves multiple processes, including compilation, testing, security analysis, deployment, and monitoring. These processes are often managed through separate tools, increasing complexity and making workflow coordination difficult. This project introduces **ChainOps, a Smart Contract DevSecOps and Operations Platform**, designed to integrate existing development and security tools into a centralized environment.

The primary objective of ChainOps is to develop a secure, scalable, and user-friendly web application that automates the smart contract development lifecycle while improving efficiency, security, and traceability. The platform provides GitHub integration, automated compilation and testing, security analysis, policy-based validation, deployment approval management, blockchain deployment, and runtime monitoring. Role-based access control ensures secure operations, while an **AI-powered assistant** helps authorized users understand pipeline failures, security findings, deployment statuses, and other operational information.

The system processes inputs such as GitHub repository events, smart contract revisions, deployment requests, and blockchain events to generate outputs including test reports, security findings, deployment records, monitoring alerts, and automated notifications. By integrating established tools, ChainOps maintains traceability between source code revisions, validation results, deployment activities, and runtime operations, reducing manual effort and improving operational visibility.

The proposed system follows a modular architecture emphasizing secure access management, workflow automation, and efficient communication between components. The technology stack includes SvelteKit with TypeScript for frontend development, NestJS with TypeScript for backend services, and Neon PostgreSQL with Prisma ORM for data management. Redis and BullMQ support asynchronous processing, Docker provides isolated execution environments, Hardhat and Slither handle compilation, testing, and security analysis, while viem enables blockchain communication. An LLM-based API supports restricted AI assistance. Overall, ChainOps aims to provide an efficient, secure, and integrated platform for managing smart contract development and operations.

Table of Content

Chapter 1 - Introduction 4
1.1 Introduction 5
1.2 Background and Motivation 5
1.3 Problem in Brief 5
1.4 Aim and Objectives 6
1.5 Proposed Solution 6
1.5.1 Core Features 6
1. Pipeline and Execution Management 7
2. Security Analysis and Governance 7
3. Blockchain Deployment and Contract Registry 7
4. Blockchain Monitoring and Alerts 8
5. Platform Access, Notifications, and AI Assistance 8
1.5.2 Technology Integration 8
1.5.3 Expected Impact 9
1.6 Summary 9
Chapter 2 - Review of others' work 9
2.1. Introduction 9
2.2 Similar Products 9
2.2.1 Tenderly 10
2.2.2 thirdweb 10
2.2.3 OpenZeppelin 10
2.2.4 GitHub Actions 10
2.3 Comparison of Existing Solutions 11
2.4 Summary 11
Chapter 3 - Technologies Adapted 12
3.2 Frontend Technologies 12
3.2.1 SvelteKit with TypeScript 12
3.3 Backend Technologies 12
3.3.1 NestJS with TypeScript 13
3.4 Database and Data Management Technologies 13

3.4.1 Neon PostgreSQL 13
3.4.2 Prisma ORM 14
3.5 Artifact Storage Technologies 14
3.5.1 Neon Object Storage 14
3.5.2 Cloudflare R2 14
3.6 Background Job Processing 14
3.6.1 Redis and BullMQ 15
3.7 Execution Environment 15
3.7.1 Docker 15
3.8 Smart Contract Development and Security Technologies 15
3.8.1 Hardhat 15
3.8.2 Slither 16
3.9 Blockchain Integration 16
3.9.1 viem 16
3.10 Application Deployment Technologies 16
3.10.1 Vercel 17
3.10.2 Linux VPS and Docker Compose 17
3.11 Version Control and Collaborative Development 17
3.11.1 GitHub Monorepo 17
3.12 Summary 18
Chapter 4 - Our Approach 18
4.1 Introduction 18
4.2 User-Centric Design 19
4.2.1 Developers 19
4.2.2 Workspace Owners and Administrators 19
4.2.3 Authorized Reviewers 19
4.3 Functional Process Flows 20
4.3.1 Repository Integration and Pipeline Execution 20
4.3.2 Security Analysis and Policy Validation 20
4.3.3 Deployment Approval and Blockchain Deployment 20
4.3.4 Blockchain Monitoring and Alert Generation 21
4.3.5 Notification and Dashboard Management 21

4.3.6 AI-Assisted Operational Query Processing 21
4.4 Software Process Model 22
Chapter 5 - Analysis and Design 23
5.1 Introduction 23
5.2 Use case diagram 23
Figure 5.2.1 24
5.3 Activity diagram 24
5.4 Sequence diagram 25
5.5 Class diagram 36
5.6 ER diagram 37
5.7 Summary 37
5.8 UI Design 37
Chapter 6 - Implementation 37
6.1 Introduction 37
6.2 Current Implementation Status 37
6.3 User Interface and User Experience Design 37
6.3.1 Authentication and Workspace Management 37
6.3.2 Global Dashboard 37
6.3.3 Pipeline and Security Management Interfaces 37
6.3.4 Deployment and Monitoring Interfaces 37
6.3.5 Notification and AI Assistant Interfaces 37
6.4 Preparation for System Implementation 37
6.5 Planned Implementation and Integration 37
6.6 Current Limitations and Next Steps 37
Chapter 7 - Discussion 37
7.1 Introduction 37
7.2 Evaluation and Preliminary Results 37
7.3 Comparison with Similar Works 37
7.4 Further Work 37
7.5 Conclusion 37
Chapter 8 - References 37
Appendix A - Individual Contribution 37

Appendix B – Project Timeline 37
Appendix C – Software Requirements Specification 37
Chapter 1 - Introduction1.1 Introduction 37
1.2 Purpose 37
1.3 Document conventions 37
1.4 Intended audience and reading suggestions 37
1.5 Product scope 37
1.6 References and source precedence 37
1.7 Summary 37
Chapter 2 - Overall description 37
2.1 Introduction 37
2.2 Product perspective 37
2.3 Product functions 37
2.4 User classes and characteristics 37
2.5 Operating environment 37
2.6 Design and implementation constraints 37
2.7 User documentation 37
2.8 Assumptions and dependencies 37
2.9 Summary 37
Chapter 3 - External interface requirements 37
3.1 Introduction 37
3.2 User interfaces 37
3.3 Hardware interfaces 37
3.4 Software interfaces 37
3.5 Communications interfaces 37
3.6 User stories 37
3.7 Summary 37
Chapter 4 - System features 37
4.1 Introduction 37
4.2 Authentication and account management 37
4.2.1 Stimulus and response 37
4.2.2 Exceptions and controls 37

4.2.3 Functional requirements 37
4.2.4 AC-AUTH Acceptance scenario 37
4.3 Workspace and project management 37
4.3.1 Stimulus and response 37
4.3.2 Exceptions and controls 37
4.3.3 Functional requirements 37
4.3.4 AC-WS Acceptance scenario 37
4.4 GitHub integration and repository management 37
4.4.1 Stimulus and response 37
4.4.2 Exceptions and controls 37
4.4.3 Functional requirements 37
4.4.4 AC-GH Acceptance scenario 37
4.5 Pipeline configuration and execution 37
4.5.1 Stimulus and response 37
4.5.2 Exceptions and controls 37
4.5.3 Functional requirements 37
4.5.4 AC-PIPE Acceptance scenario 37
4.6 Compilation and automated testing 37
4.6.1 Stimulus and response 37
4.6.2 Exceptions and controls 37
4.6.3 Functional requirements 37
4.6.4 AC-BUILD Acceptance scenario 37
4.7 Security analysis and policy enforcement 37
4.7.1 Stimulus and response 37
4.7.2 Exceptions and controls 37
4.7.3 Functional requirements 37
4.7.4 AC-SEC Acceptance scenario 37
4.8 Deployment governance and approvals 37
4.8.1 Stimulus and response 37
4.8.2 Exceptions and controls 37
4.8.3 Functional requirements 37
4.8.4 AC-GOV Acceptance scenario 37

4.9 Blockchain network and deployment management 37
4.9.1 Stimulus and response 37
4.9.2 Exceptions and controls 37
4.9.3 Functional requirements 37
4.9.4 AC-DEP Acceptance scenario 37
4.10 Contract registry 37
4.10.1 Stimulus and response 37
4.10.2 Exceptions and controls 37
4.10.3 Functional requirements 37
4.10.4 AC-REG Acceptance scenario 37
4.11 Monitoring events and alerts 37
4.11.1 Stimulus and response 37
4.11.2 Exceptions and controls 37
4.11.3 Functional requirements 37
4.11.4 AC-MON Acceptance scenario 37
4.12 Notifications 37
4.12.1 Stimulus and response 37
4.12.2 Exceptions and controls 37
4.12.3 Functional requirements 37
4.12.4 AC-NOT Acceptance scenario 37
4.13 Audit logs and reporting 37
4.13.1 Stimulus and response 37
4.13.2 Exceptions and controls 37
4.13.3 Functional requirements 37
4.13.4 AC-AUD Acceptance scenario 37
4.14 Dashboard and reporting 37
4.14.1 Stimulus and response 37
4.14.2 Exceptions and controls 37
4.14.3 Functional requirements 37
4.14.4 AC-DASH Acceptance scenario 37
4.15 Optional AI assistant 37
4.15.1 Stimulus and response 37

4.15.2 Exceptions and controls 37
4.15.3 Functional requirements 37
4.15.4 AC-AI Acceptance scenario 37
4.16 Summary 37
Chapter 5 - Other nonfunctional requirements 37
5.1 Introduction 37
5.2 Performance requirements 37
5.3 Security requirements 37
5.4 Reliability and recoverability 37
5.5 Usability and accessibility 37
5.6 Maintainability and testability 37
5.7 Compatibility and portability 37
5.8 Auditability and observability 37
5.9 Safety requirements 37
5.10 Business rules 37
5.11 Quality verification profile 37
5.12 Summary 37
Chapter 6 - Other requirements 37
6.1 Introduction 37
6.2 Logical data requirements 37
6.3 Data lifecycle and retention 37
6.4 Privacy and regulatory scope 37
6.5 Localization and time representation 37
6.6 Backup and recovery 37
6.7 Configuration and release documentation 37
6.8 Acceptance and change control 37
6.9 Summary 37
References 37
Appendix A Glossary 37
Appendix B Analysis models and traceability 37
B1 Logical relationships 37
B2 State transition model 37

B3 Functional traceability matrix 37
B4 Quality and clarification coverage 37
B5 End to end acceptance sequence 37
Appendix C Decisions to be determined 37
C1 Baseline consistency decisions 37

Chapter 1 - Introduction

1.1 Introduction

Blockchain smart contracts are software programs deployed to blockchain networks and are commonly developed through a lifecycle that includes source-code management, compilation, automated testing, security analysis, deployment, and post-deployment monitoring. In practice, these activities may be performed using separate tools, command-line utilities, scripts, blockchain RPC services, and monitoring interfaces. This fragmentation makes it difficult for a development team to maintain one clear and traceable view of how a particular source-code revision was validated, approved, deployed, and monitored. ChainOps is proposed as a developer-focused Smart Contract DevSecOps and operations platform for EVM-compatible smart-contract projects. It connects GitHub-based development with an automated and visible workflow for compilation, testing, security validation, policy evaluation, controlled deployment, contract registration, runtime monitoring, alerts, notifications, and audit history. ChainOps does not create a new blockchain, Solidity compiler, security analyser, or source-code hosting service. Instead, it integrates established tools and services and provides the orchestration, governance, traceability, and operational-control layer around them. GitHub remains the source of truth for project source code, while ChainOps manages lifecycle information associated with each relevant source-code revision.

1.2 Background and Motivation

A smart-contract development team may use GitHub for source control, Hardhat for compilation and testing, Slither or another established analyser for security analysis, deployment scripts and EVM RPC services for blockchain interaction, and separate tools for observing deployed contracts. Although each tool solves an important part of the process, the overall lifecycle can remain fragmented. A team may need to determine which exact source-code commit was compiled and tested, what security findings existed for that revision, whether it satisfied release policies, who approved a deployment, which blockchain transaction performed the deployment, what contract address was produced, and what important runtime events occurred afterward. When these records are distributed across repositories, command-line outputs, scripts, wallets, blockchain explorers, and monitoring services, maintaining this traceability becomes difficult. The motivation for ChainOps is therefore not to replace mature blockchain-development tools, but to connect them through a controlled lifecycle in which a smart-contract revision can be followed from a GitHub commit through validation and governance to deployment and runtime monitoring.

1.3 Problem in Brief

Smart contract development involves multiple processes, including compilation, testing, security analysis, deployment, and monitoring, which are often managed through separate tools and platforms. This fragmented approach increases manual effort, complicates workflow management, and makes it difficult to track source code revisions, security results, deployment approvals, and blockchain activities.

The lack of a centralized system reduces operational visibility, efficiency, and traceability throughout the development lifecycle. Therefore, **ChainOps** aims to address these challenges by integrating existing tools into a unified platform that automates workflows, improves security management, and provides centralized monitoring and lifecycle tracking.

1.4 Aim and Objectives

Aim

The aim of this project is to design and develop a Smart Contract DevSecOps and operations platform that connects GitHub-based smart-contract development with automated validation, security policy enforcement, controlled deployment, contract traceability, and post-deployment monitoring for EVM compatible projects.

Objectives

- Study the smart-contract development, validation, deployment, and monitoring workflow and relevant technologies.
- Design and implement authentication, organization/project management, environments, and role based access.
- Integrate GitHub repositories and repository events with a visible smart-contract pipeline workflow.
- Automate compilation, testing, and established security-analysis tools and collect their results.
- Apply policy-based release gates to validation and security results.
- Provide approval and governance workflows for sensitive deployment operations.
- Deploy validated smart-contract revisions to configured EVM development/test networks and track transaction status and confirmations.
- Maintain a versioned Contract Registry linking source commits, artifacts, networks, addresses, transactions, and deployment history.
- Monitor selected deployed-contract events and relevant transaction activity and generate configurable alerts and notifications.
- Maintain audit records and dashboards for lifecycle and operational visibility.
- Provide a restricted, read-oriented ChainOps AI Assistant for authorized operational queries.
- Evaluate the completed solution and prepare final project documentation.

1.5 Proposed Solution

The proposed solution introduces **ChainOps, a Smart Contract DevSecOps and Operations Platform** designed to simplify and automate the smart contract development lifecycle. The system integrates existing development, security, and blockchain tools into a centralized environment, enabling developers to manage processes from source code changes to deployment and post-deployment monitoring. By combining automated workflows, security validation, deployment governance, and AI-assisted guidance, ChainOps aims to improve development efficiency, security, and lifecycle traceability.

1.5.1 Core Features

1. Pipeline and Execution Management

ChainOps integrates with GitHub to automatically initiate development pipelines when relevant source code changes are detected. The system integrates Hardhat for automated smart contract compilation and testing.

Key functionalities include:

- GitHub repository and webhook integration.
- Automated pipeline execution and background processing.
- Smart contract compilation and testing.
- Pipeline status tracking, logs, and execution history.

This reduces manual effort and provides centralized visibility into development activities.

2. Security Analysis and Governance

The platform integrates security analysis tools such as Slither to identify potential vulnerabilities in smart contracts. Security findings and testing results are evaluated against predefined release policies before deployment.

Key functionalities include:

- Automated security vulnerability analysis.
- Policy-based validation and deployment restrictions.
- Deployment approval and rejection workflows.
- Security reports and governance audit records.

These features strengthen security and ensure that deployment operations follow established authorization requirements.

3. Blockchain Deployment and Contract Registry

ChainOps supports controlled deployment of validated smart contracts to EVM-compatible blockchain networks. It maintains deployment records and tracks relevant blockchain transactions.

Key functionalities include:

- Smart contract deployment to configured networks.
- Transaction status and confirmation tracking.
- Contract address and version management.
- Centralized Contract Registry and deployment history.

This ensures traceability between source code revisions, validation results, and deployed contracts.

4. Blockchain Monitoring and Alerts

The monitoring component observes selected blockchain events and transactions associated with deployed smart contracts. It processes relevant event information and generates alerts based on configured conditions.

Key functionalities include:

- Smart contract event and transaction monitoring.
- Event decoding using contract ABI information.
- Configurable alert rules.
- Monitoring dashboards and operational history.

These features improve visibility into smart contract activities after deployment.

5. Platform Access, Notifications, and AI Assistance

ChainOps provides centralized user management, role-based access control, and operational communication across all modules. An AI-powered assistant helps authorized users understand technical information through natural language queries.

Key functionalities include:

- Authentication, workspace, project, and environment management.
- Role-based permissions and secure access control.
- Global dashboard and automated in-app/email notifications.
- AI-assisted explanations of pipeline failures, security findings, deployment statuses, and monitoring alerts.

The AI Assistant operates with restricted read-only access and cannot independently perform privileged operations.

1.5.2 Technology Integration

The proposed system uses SvelteKit with TypeScript for frontend development, NestJS with TypeScript for backend services, and Neon PostgreSQL with Prisma ORM for data management. Redis and BullMQ support asynchronous pipeline execution, while Docker provides isolated environments for processing smart contract projects. Hardhat is integrated for compilation and testing, Slither for security analysis, and viem for blockchain interaction. Object storage services manage generated artifacts, while Vercel and Linux VPS infrastructure support application deployment. An LLM-based API is planned to support AI-

assisted operational explanations.. An LLM-based API supports AI-assisted explanations, while background processing technologies enable automated pipeline execution.

1.5.3 Expected Impact

ChainOps aims to address fragmented smart contract development workflows by providing a unified platform for automation, security validation, deployment management, and monitoring. By integrating existing tools and maintaining centralized operational records, the proposed system is expected to reduce repetitive tasks, improve collaboration, strengthen security governance, and provide end-to-end traceability throughout the smart contract development lifecycle.

1.6 Summary

This chapter introduced the background and motivation for developing ChainOps, a Smart Contract DevSecOps and Operations Platform. The project aim, objectives, problem overview, and proposed solution were discussed, highlighting the need for a centralized platform to automate smart contract development processes, improve security, and ensure lifecycle traceability. The structure of the report was also outlined to provide an overview of the subsequent chapters.

Chapter 2 - Review of others' work

2.1. Introduction

Smart contract development requires various tools for source code management, compilation, testing, security analysis, deployment, and monitoring. Several existing platforms provide solutions for automating and managing different stages of this lifecycle. Platforms such as Tenderly, thirdweb, OpenZeppelin, and GitHub Actions offer functionalities related to blockchain development, deployment automation, security, and operational monitoring. Analyzing these solutions helps identify their strengths, limitations, and approaches to smart contract lifecycle management. This chapter examines existing platforms and highlights opportunities for ChainOps to provide an integrated development environment with centralized workflow management, security governance, lifecycle traceability, and AI-assisted operational guidance.

2.2 Similar Products

Existing blockchain development platforms offer valuable functionalities that influence the design of ChainOps. Tenderly provides transaction simulation, debugging, and monitoring capabilities, while thirdweb simplifies smart contract deployment and management. OpenZeppelin offers tools for secure blockchain transaction execution and monitoring, whereas GitHub Actions supports automated development workflows. ChainOps draws inspiration from these solutions while focusing on connecting source code changes,

automated validation, security policies, deployment approvals, contract records, and runtime monitoring within a unified platform.

2.2.1 Tenderly

Tenderly is a blockchain development and operations platform that provides tools for smart contract simulation, transaction debugging, monitoring, and alerting. It enables developers to analyze blockchain transactions, investigate execution failures, simulate contract interactions, and monitor on-chain activities. Its integrated development and observability tools improve visibility into smart contract behavior and help identify operational issues [1].

Tenderly serves as an important reference for ChainOps, particularly in designing blockchain monitoring dashboards, transaction tracking, and alert management. While Tenderly provides extensive blockchain development and operational capabilities, ChainOps places additional emphasis on connecting GitHub-based pipeline execution, static security analysis, release policy evaluation, and deployment approval records within a traceable workflow.

2.2.2 thirdweb

thirdweb is a Web3 development platform that provides tools for building, deploying, and managing smart contracts across EVM-compatible blockchain networks. Its deployment tools support popular frameworks such as Hardhat and provide contract management dashboards, contract verification, and deployment versioning capabilities [2].

For ChainOps, thirdweb serves as a reference for simplifying smart contract deployment and organizing deployed contract information. ChainOps extends this approach by emphasizing the relationship between source code revisions, automated testing, security findings, deployment approvals, and runtime activities through a centralized Contract Registry and lifecycle management process.

2.2.3 OpenZeppelin

OpenZeppelin provides widely used tools for secure smart contract development and blockchain operations. Its open-source Relayer supports transaction submission, signing, and status tracking, while OpenZeppelin Monitor observes blockchain events and transactions and generates notifications based on configurable conditions [3], [4].

These tools provide useful references for ChainOps, particularly in transaction management, monitoring, and security-related operational workflows. While OpenZeppelin Relayer and Monitor provide specialized blockchain infrastructure services, ChainOps aims to integrate similar capabilities with pipeline automation, role-based access management, policy-based validation, and centralized operational reporting.

2.2.4 GitHub Actions

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that automates software development workflows based on repository events. Developers can configure workflows to compile, test, analyze, and deploy applications automatically. For smart contract projects, GitHub Actions can integrate tools such as Hardhat and Slither to perform automated testing and security analysis [5], [6].

GitHub Actions provides an important reference for ChainOps pipeline automation and repository integration. However, implementing a complete smart contract lifecycle using GitHub Actions requires configuring and connecting additional blockchain-specific tools and services. ChainOps aims to simplify this process through a dedicated interface that combines pipeline results, security validation, deployment governance, contract tracking, and monitoring information.

2.3 Comparison of Existing Solutions

The reviewed solutions demonstrate different approaches to blockchain development and operations. Table 2.1 summarizes their primary functionalities and their relevance to the proposed ChainOps platform.

Table 2.1: Comparison of Existing Solutions

Existing solution	Primary functionalities	Relevance to ChainOps
Tenderly	Transaction simulation, debugging, monitoring, and alerts	Monitoring dashboards and transaction visibility
Thirdweb	Smart contract deployment, verification, and management	Deployment management and Contract Registry
OpenZeppelin	Transaction relaying, monitoring, and notifications	Secure blockchain operations and alert management
GitHub Actions	Automated CI/CD workflows, testing, and deployment	GitHub integration and pipeline automation
ChainOps (Proposed)	Pipeline automation, security validation, governance, deployment, monitoring, and AI assistance	Unified and traceable smart contract lifecycle management

2.4 Summary

This chapter reviewed existing solutions, including Tenderly, thirdweb, OpenZeppelin, and GitHub Actions, highlighting their contributions to blockchain development, automation, deployment, and monitoring. These platforms provide valuable capabilities for different stages of the smart contract lifecycle. Based on the reviewed approaches, ChainOps proposes an integrated platform that combines automated development workflows, security validation, deployment governance, contract tracking, monitoring, and AI-assisted operational guidance.

The following chapter discusses the proposed system's approach, architecture, and technologies.

Chapter 3 - Technologies Adapted

3.1 Introduction

This chapter presents the technologies selected for the design and development of ChainOps, a Smart Contract DevSecOps and Operations Platform. Since ChainOps integrates multiple development processes, including compilation, testing, security analysis, deployment, and monitoring, a suitable technology stack is essential to ensure efficient workflow automation, security, scalability, and communication between system components.

The proposed system follows a modular architecture consisting of a web-based frontend, backend API services, a relational database, background job processing, isolated execution environments, and blockchain integration tools. The selected technologies are categorized into frontend technologies, backend technologies, data storage, asynchronous processing, smart contract development and security tools, blockchain interaction, deployment infrastructure, and version control.

The following sections describe each selected technology and explain its role in implementing the ChainOps platform.

3.2 Frontend Technologies

The frontend serves as the primary interface through which users interact with ChainOps. It provides access to workspace and project management, pipeline execution results, security findings, deployment information, blockchain monitoring, notifications, and AI-assisted operational guidance.

3.2.1 SvelteKit with TypeScript

SvelteKit is a modern web application framework built on Svelte that supports the development of interactive and responsive user interfaces. It provides features such as file-based routing, server-side rendering, and efficient application navigation.In ChainOps, SvelteKit is selected to develop the web interface, including the global dashboard, project management pages, pipeline monitoring interfaces, deployment management, and notification center. Its component-based approach supports the creation of reusable interface elements and helps maintain a consistent user experience across different modules.

TypeScript is used alongside SvelteKit to provide static type checking, improve code maintainability, and reduce development errors. This combination supports the development of a structured and responsive frontend that communicates efficiently with backend API services.

3.3 Backend Technologies

The backend is responsible for processing user requests, managing business logic, coordinating system modules, and communicating with external development tools and blockchain networks.

3.3.1 NestJS with TypeScript

NestJS is a Node.js-based framework designed for developing scalable and maintainable server-side applications. It provides a modular architecture, dependency injection, and built-in support for developing RESTful APIs.

In ChainOps, NestJS is selected as the main backend framework for implementing authentication, workspace and project management, repository integration, pipeline coordination, security validation, deployment management, and notification services.

Its modular structure allows the five major functional components of ChainOps to be developed independently while maintaining communication through shared services and APIs. TypeScript further improves code consistency and supports reliable integration between different backend components.

3.4 Database and Data Management Technologies

ChainOps requires structured and reliable data storage to maintain user information, project configurations, pipeline execution records, security findings, deployment histories, monitoring data, and audit records.

3.4.1 Neon PostgreSQL

Neon is a serverless PostgreSQL database platform that provides managed relational database services. It supports standard PostgreSQL functionality while providing cloud-based database infrastructure.

In ChainOps, Neon PostgreSQL is selected as the primary database for storing structured platform data, including:

- User accounts, workspaces, and access permissions.
- Project and environment configurations.
- Pipeline execution records and results.
- Security findings and release policies.
- Deployment requests and approval records.
- Contract Registry and monitoring information.
- Notifications and audit histories.

PostgreSQL provides relational data management, transactional consistency, and support for complex queries. These capabilities are important for maintaining accurate relationships between source code revisions, validation results, deployment activities, and operational records.

3.4.2 Prisma ORM

Prisma is an Object-Relational Mapping (ORM) tool that simplifies database interaction in TypeScript applications. It enables developers to define database models, manage schema migrations, and perform database operations using a type-safe client.

ChainOps uses Prisma to establish communication between the NestJS backend and Neon PostgreSQL. It supports structured data management and simplifies operations involving users, projects, pipeline records, deployments, and other related entities.

Prisma helps reduce repetitive database code, improve maintainability, and ensure consistency between application data models and the database structure.

3.5 Artifact Storage Technologies

Smart contract development generates various artifacts, including compiled contract metadata, Application Binary Interfaces (ABIs), bytecode, and execution outputs. These files require a storage mechanism separate from the relational database.

3.5.1 Neon Object Storage

Neon Object Storage is selected as the primary storage solution for relevant smart contract build artifacts and associated files.

In ChainOps, object storage is intended to maintain selected generated artifacts that need to remain accessible after pipeline execution. The PostgreSQL database maintains the corresponding metadata and references to these stored files.

This approach separates large file storage from structured platform data and supports efficient artifact retrieval and management.

3.5.2 Cloudflare R2

Cloudflare R2 is considered as an alternative object storage solution. It provides object storage capabilities that can be used to maintain build artifacts and other generated files.

If required, Cloudflare R2 can replace the primary artifact storage service without significantly changing the overall responsibilities of the storage component.

3.6 Background Job Processing

Smart contract compilation, testing, and security analysis may require considerable execution time. Therefore, ChainOps adopts asynchronous background processing to prevent long-running tasks from blocking normal API operations.

3.6.1 Redis and BullMQ

Redis is an in-memory data store that supports fast data access and is commonly used for caching and message-based processing. BullMQ is a Node.js job queue library built on Redis that supports background job management.

In ChainOps, Redis and BullMQ are selected to manage asynchronous pipeline execution. When a relevant GitHub event is received, the backend can create a pipeline job and place it in a queue. Background workers then process the job and coordinate the required compilation, testing, and security analysis activities.

This approach supports job scheduling, execution status tracking, and configurable retry handling. It also separates time-consuming pipeline operations from user-facing API requests, improving system responsiveness and workflow management.

3.7 Execution Environment

3.7.1 Docker

Docker is a containerization platform that enables applications and their dependencies to run in isolated environments. It provides a consistent execution environment across different systems and simplifies dependency management.

ChainOps uses temporary Docker containers to execute smart contract compilation, testing, and security analysis tasks. When a pipeline is initiated, the required source code revision is retrieved from GitHub and processed within an isolated execution environment.

This approach reduces dependency conflicts between projects and allows execution environments to be created and removed as required. Appropriate resource limits and security restrictions can further reduce the risks associated with executing repository code.

3.8 Smart Contract Development and Security Technologies

ChainOps integrates established smart contract development and security tools rather than developing new compilers, testing frameworks, or vulnerability analyzers.

3.8.1 Hardhat

Hardhat is a development environment for Ethereum-compatible smart contracts. It provides tools for compiling Solidity contracts, executing automated tests, and managing smart contract development workflows.

In ChainOps, Hardhat is selected as the primary tool for automated smart contract compilation and testing. During pipeline execution, Hardhat processes the selected source code revision and generates compilation outputs, testing results, and relevant artifacts.

These results are collected by ChainOps and made available for pipeline monitoring and subsequent security validation.

3.8.2 Slither

Slither is a static analysis framework designed to identify potential security vulnerabilities and code quality issues in Solidity smart contracts.

ChainOps integrates Slither into its security validation process to analyze smart contract source code and generate security findings. The analysis results are collected and evaluated against configured release policies to determine whether a contract revision is eligible for deployment or requires further review.

This integration enables security analysis to become part of the automated development pipeline and supports policy-based deployment governance.

3.9 Blockchain Integration

3.9.1 viem

viem is a TypeScript library that enables applications to interact with Ethereum and other EVM-compatible blockchain networks through RPC interfaces.

In ChainOps, viem is selected to support blockchain communication during smart contract deployment and transaction management. It enables the platform to submit deployment transactions, retrieve transaction information, track confirmations, and access relevant blockchain data.

The integration supports maintaining deployment records, including transaction hashes, deployed contract addresses, network information, and transaction statuses.

This allows ChainOps to connect validated smart contract revisions with their corresponding blockchain deployment activities.

3.10 Application Deployment Technologies

The ChainOps deployment architecture separates frontend hosting from backend services and background pipeline execution.

3.10.1 Vercel

Vercel is a cloud-based deployment platform designed for hosting modern web applications.

In ChainOps, Vercel is selected for deploying the SvelteKit frontend. It supports automated application deployment and provides infrastructure for delivering the web interface to users.

Using Vercel simplifies frontend hosting and allows the development team to manage interface updates independently from backend and worker services.

3.10.2 Linux VPS and Docker Compose

A Linux Virtual Private Server (VPS) is selected to host the ChainOps backend API and background workers. It provides a configurable server environment suitable for managing backend services and execution infrastructure.

Docker Compose is used to define and manage the required application services through container configurations. These may include the NestJS API, background workers, and Redis.

This deployment approach enables the backend components to operate within a coordinated environment while supporting the execution of automated smart contract pipelines.

3.11 Version Control and Collaborative Development

3.11.1 GitHub Monorepo

GitHub is selected as the version control and collaborative development platform for ChainOps. The project adopts a monorepository structure in which the frontend, backend, shared components, and relevant configurations are maintained within a single GitHub repository.

The monorepo approach supports:

- Centralized source code management.
- Collaboration among the five development modules.
- Branching, merging, and pull request reviews.
- Shared configuration and dependency management.
- Version tracking and coordinated development.

In addition to hosting the ChainOps application source code, GitHub integration is a core feature of the proposed system. Developers can connect their own smart contract repositories to ChainOps, enabling automated pipeline execution through GitHub webhooks.

This supports both internal project collaboration and the platform's main objective of integrating GitHub-based smart contract development workflows.

3.12 Summary

This chapter discussed the technologies selected for developing the ChainOps platform and their roles in supporting its main functionalities. SvelteKit and TypeScript are used for frontend development, while NestJS provides modular backend services. Neon PostgreSQL and Prisma support structured data management, with object storage used for generated artifacts. Redis and BullMQ enable asynchronous processing, and Docker provides isolated execution environments for smart contract pipelines. Hardhat and Slither support automated compilation, testing, and security analysis, while viem enables blockchain communication. Vercel, Linux VPS, Docker Compose, and GitHub provide the necessary deployment and collaborative development infrastructure. Together, these technologies establish a foundation for building a secure, scalable, and integrated Smart Contract DevSecOps and Operations Platform.

Chapter 4 - Our Approach

4.1 Introduction

The approach adopted for **ChainOps** focuses on developing a centralized Smart Contract DevSecOps and Operations Platform that integrates various stages of the smart contract development lifecycle into a unified environment. Rather than developing separate tools for compilation, testing, security analysis, deployment, and monitoring, the proposed system connects existing technologies and automates their interactions through a structured workflow. This approach aims to reduce manual effort, improve security, and maintain end-to-end traceability throughout smart contract development and operations.

The system follows a modular architecture consisting of five main components: Pipeline and Execution, Blockchain Deployment and Contract Registry, Monitoring and Alerts, Security and Governance, and Platform Access and AI Assistance. Each component manages specific responsibilities while communicating with other components to support the complete development lifecycle. The proposed technology stack includes SvelteKit with TypeScript for frontend development, NestJS with TypeScript for backend services, and Neon PostgreSQL with Prisma ORM for data management. Redis and BullMQ support background job processing, while Docker provides isolated execution environments. Hardhat and Slither are

integrated for smart contract compilation, testing, and security analysis, respectively, with viem enabling blockchain interactions.

The initial approach involves analyzing the smart contract development lifecycle, identifying the limitations of existing tools, and defining the functional requirements of the proposed system. Based on these requirements, system workflows, user interactions, and module dependencies are designed to establish a clear understanding of how ChainOps operates. UML diagrams, database models, and user interface prototypes are used during the design process to represent system behavior, data relationships, and interactions between components. These activities guide the development of a secure, scalable, and user-friendly platform.

4.2 User-Centric Design

ChainOps is designed to support different users involved in smart contract development and management. Role-based access control ensures that users can access relevant information and perform operations according to their assigned permissions.

4.2.1 Developers

Developers are the primary users of the ChainOps platform. The system provides a centralized interface for connecting GitHub repositories, monitoring pipeline execution, reviewing compilation and testing results, examining security findings, and requesting smart contract deployments.

Developers can also access deployment histories, contract information, monitoring alerts, and AI-assisted explanations of operational records. This approach reduces the need to switch between multiple tools and improves visibility into the development lifecycle.

4.2.2 Workspace Owners and Administrators

Workspace owners and administrators are responsible for managing collaborative development environments and controlling access to platform resources.

Their main responsibilities include:

- Creating and managing workspaces and projects.
- Managing workspace members and assigning roles.
- Configuring project environments and access permissions.
- Managing permitted workspace-level approval settings.
- Reviewing authorized operational information and audit records.

These functionalities provide an organized and secure environment for development teams while maintaining control over sensitive operations.

4.2.3 Authorized Reviewers

Authorized reviewers are responsible for examining deployment requests and making approval decisions according to established security and governance policies.

Reviewers can examine validation results, security findings, release policy decisions, and deployment information before approving or rejecting eligible requests. This approach ensures that sensitive deployment activities follow defined authorization and governance procedures.

4.3 Functional Process Flows

The ChainOps platform processes various inputs from users, connected development tools, and blockchain networks to generate structured results and automate smart contract lifecycle activities.

4.3.1 Repository Integration and Pipeline Execution

- **Input:** Developers connect GitHub repositories and provide relevant project configurations. Source code changes are identified through configured GitHub webhooks or manual pipeline requests.
- **Process:** ChainOps validates incoming events, identifies the corresponding commit revision, and initiates an automated pipeline. Background workers retrieve the required source code into temporary isolated Docker containers and execute smart contract compilation and automated testing using Hardhat.
- **Output:** The system generates pipeline execution statuses, compilation results, test reports, logs, and relevant build artifacts.

This process reduces repetitive manual tasks while maintaining traceability between source code revisions and validation results.

4.3.2 Security Analysis and Policy Validation

- **Input:** Smart contract source revisions, compilation results, testing outcomes, and configured security policies.
- **Process:** The system integrates an established analyzer such as Slither to identify potential vulnerabilities. Security findings are collected and evaluated alongside build and testing results against predefined release policies.
- **Output:** Structured security findings, validation reports, and policy decisions indicating whether a revision can proceed, must be blocked, or requires additional review.

This approach improves security management by incorporating automated vulnerability analysis and policy enforcement into the development workflow.

4.3.3 Deployment Approval and Blockchain Deployment

- **Input:** Validated smart contract revisions, deployment requests, target network configurations, and approval decisions.
- **Process:** ChainOps verifies applicable validation and authorization requirements. Deployment requests requiring approval are reviewed by authorized users. Once the necessary conditions are satisfied, the system submits the deployment transaction to a configured EVM-compatible blockchain network and monitors its status.
- **Output:** Deployment transaction records, confirmation statuses, deployed contract addresses, and updated Contract Registry information.

This workflow ensures controlled deployment while maintaining a relationship between source code revisions, validation records, approvals, and deployed contracts.

4.3.4 Blockchain Monitoring and Alert Generation

- **Input:** Registered smart contract addresses, blockchain events, transaction activities, and configured monitoring rules.
- **Process:** The monitoring component retrieves selected blockchain events and relevant transaction information. Event data is decoded using contract ABI information and evaluated against configured monitoring conditions.
- **Output:** Structured monitoring records, contract activity summaries, and alerts when specified conditions are detected.

This process provides developers with improved visibility into deployed smart contract activities and enables the identification of operational issues.

4.3.5 Notification and Dashboard Management

- **Input:** Pipeline events, security findings, deployment updates, approval decisions, monitoring alerts, and user notification preferences.
- **Process:** ChainOps identifies authorized recipients, checks notification preferences, and generates appropriate in-app or email notifications. The global dashboard retrieves and combines authorized operational summaries from the platform's functional components.
- **Output:** Operational notifications, a centralized notification center, and dashboard summaries displaying relevant pipeline, deployment, security, and monitoring information.

This functionality improves communication and enables users to access important operational information through a unified interface.

4.3.6 AI-Assisted Operational Query Processing

- **Input:** Natural language questions submitted by users regarding pipeline failures, security findings, deployment statuses, policy decisions, or monitoring alerts.
- **Process:** The AI Assistant verifies the user's authorization, retrieves relevant operational records from permitted project contexts, and uses an integrated Large Language Model (LLM) API to generate contextual explanations.
- **Output:** Read-only, AI-generated explanations based on authorized operational information.

The AI Assistant is restricted from performing privileged activities such as approving deployments, modifying security policies, or executing blockchain transactions. This approach improves accessibility to complex technical information while maintaining security boundaries.

4.4 Software Process Model

An iterative **Agile development approach** is proposed for ChainOps to support the development and integration of its five functional modules. Since the project involves multiple technologies, external tool integrations, and interconnected workflows, an iterative approach allows the team to develop individual components, evaluate their functionality, and refine the system throughout the development process.

The project can be organized into short development iterations focusing on manageable tasks such as requirement analysis, system design, authentication, repository integration, pipeline execution, security validation, deployment, monitoring, and AI assistance. Each iteration includes development, testing, and review activities to identify technical issues and improve implementation quality.

Development responsibilities are distributed among five team members, with each member primarily responsible for one functional module. However, requirement analysis, system design, integration, testing, and documentation remain shared responsibilities. Version control and collaborative development can be managed through GitHub, while task tracking and regular progress discussions support coordination between team members.

The iterative development approach is suitable for ChainOps because it allows individual modules to be developed incrementally while supporting continuous integration and refinement. It also helps the team manage technical dependencies, identify integration challenges early, and maintain progress toward the overall project objectives.

4.5 Summary

This chapter presented the proposed development approach for ChainOps, emphasizing modular architecture, user-centric design, automated workflows, and secure integration of existing smart contract development tools. The main user roles, functional process flows, and

development methodology were discussed, highlighting how the platform aims to improve efficiency, security, and lifecycle traceability. The following chapter focuses on the analysis and design of the proposed system, including software requirements, system architecture, and UML models.

Chapter 5 - Analysis and Design

5.1 Introduction

This chapter presents the analysis and design of ChainOps, a Smart Contract DevSecOps and Operations Platform designed to integrate and automate smart contract development processes. The system analysis focuses on identifying functional and non-functional requirements, user roles, system interactions, and the relationships between the five major modules. The design phase emphasizes a modular architecture, secure access control, efficient data management, and seamless communication between system components. Unified Modeling Language (UML) diagrams, including Use Case, Activity, Sequence, and Class diagrams, are used to represent system functionalities and workflows, while the Enhanced Entity-Relationship (EER) diagram illustrates the database structure and relationships. These design models provide a structured foundation for system implementation and integration.

5.2 Use case diagram

The use case diagrams illustrate the functional requirements of the ChainOps platform by representing the interactions between different actors and the system. They identify the major functionalities available to users, including developers, workspace administrators, and authorized reviewers. These diagrams provide a clear understanding of user responsibilities, access permissions, and the overall scope of the system.

https://drive.google.com/file/d/1X9FIXNBVGB13WKKNXfoyCegdq2X7HIJd/view?usp=sharing

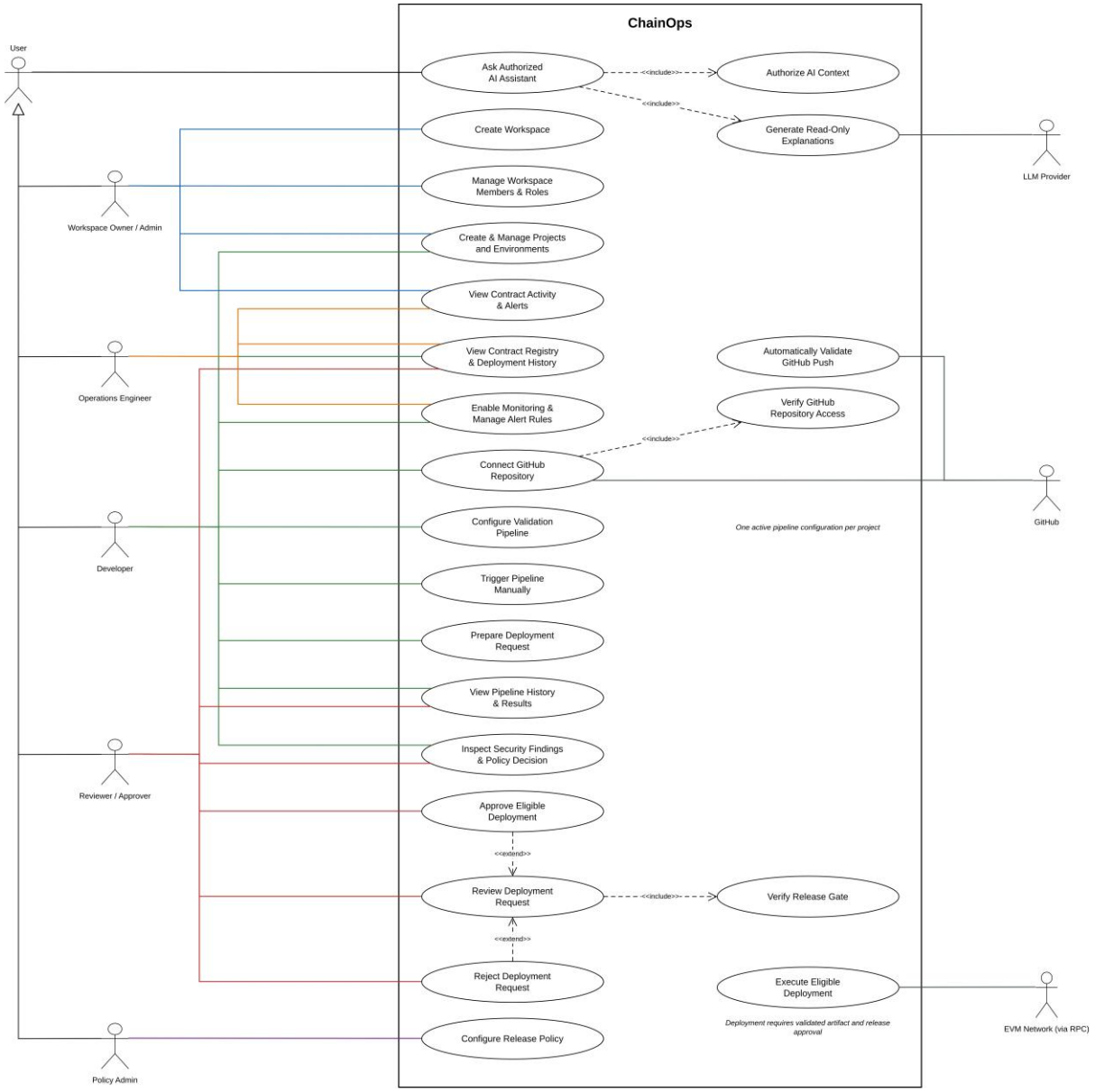


Figure 5.2.1

5.3 Activity diagram

The activity diagrams represent the workflows and operational processes involved in the ChainOps platform. They illustrate the sequence of activities, decision points, and interactions between different system components. These diagrams help explain how major processes are executed, including alternative paths, validation conditions, and possible outcomes.

https://drive.google.com/file/d/1bb2CD1qyauuWFur7UJeqhwVlM2b4KT5J/view?usp=sharing

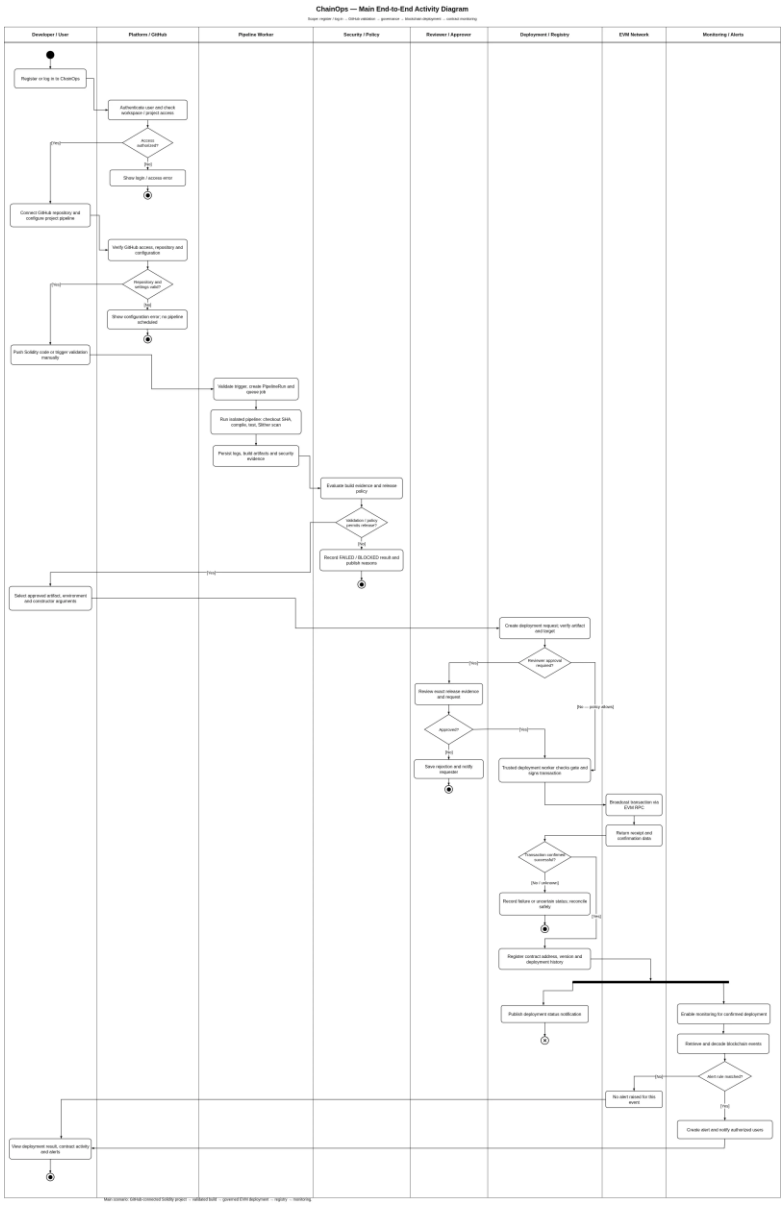


Figure 5.3.1

5.4 Sequence diagram

The sequence diagrams illustrate the interactions and communication between users, system components, and external services within ChainOps. They represent the chronological order of messages exchanged during specific operations, demonstrating how different components collaborate to complete a process. These diagrams help identify system responsibilities and clarify the flow of information between components.

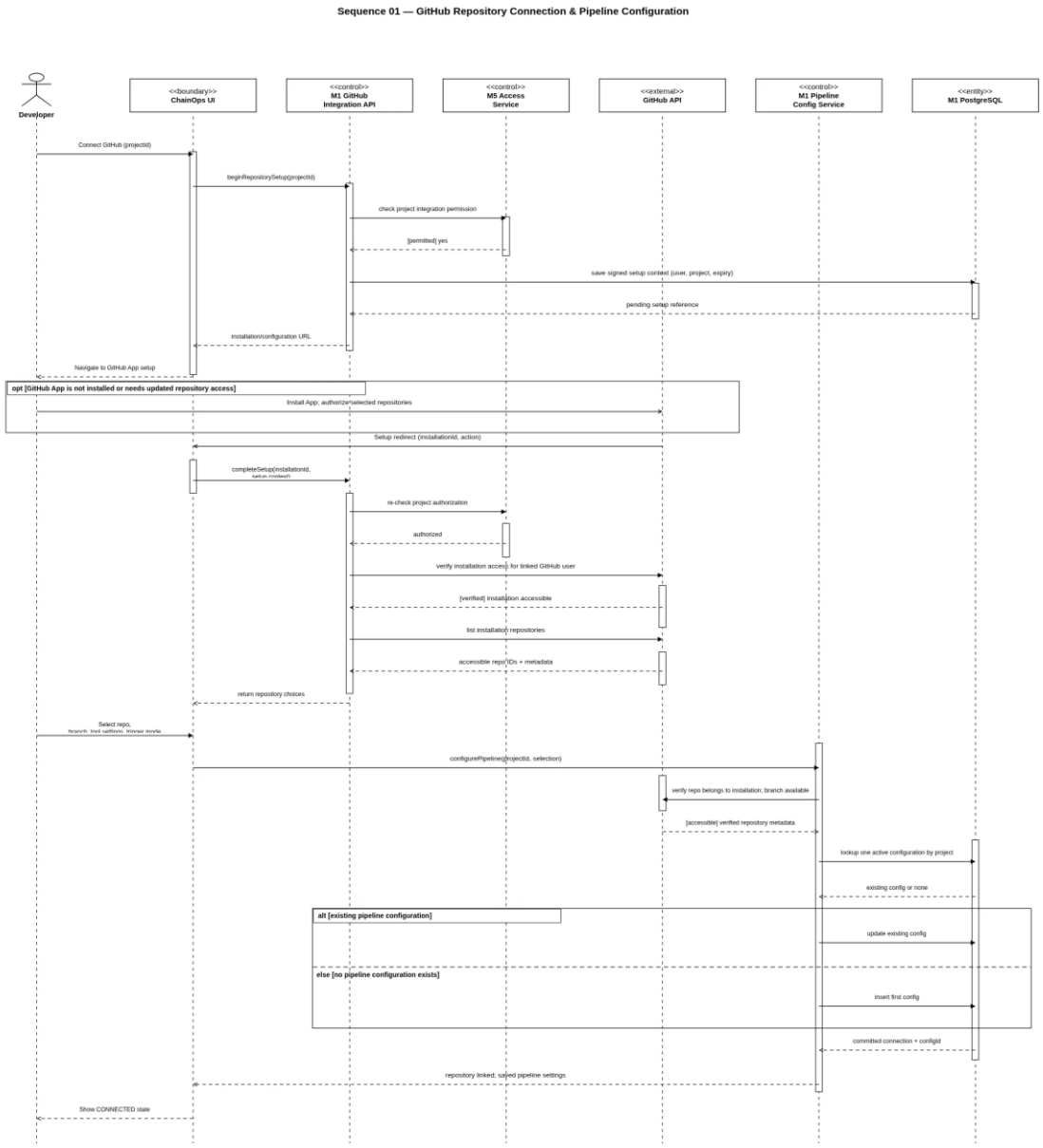

Sequence 01 — GitHub Repository Connection & Pipeline Configuration
Developer
<<boundary>>
ChainOps UI
<<control>>
M1 GitHub
Integration API
<<control>>
M5 Access
Service
<<external>>
GitHub API
<<control>>
M1 Pipeline
Config Service
<<entity>>
M1 PostgreSQL
Connect GitHub (projectId)
beginRepositorySetup(projectId)
check project integration permission
[permitted] yes
save signed setup context (user, project, expiry)
pending setup reference
installation/configuration URL
Navigate to GitHub App setup
opt [GitHub App is not installed or needs updated repository access]
Install App; authorize selected repositories
Setup redirect (installationId, action)
completeSetup(installationId,
re-check project authorization
authorized
verify installation access for linked GitHub user
[verified] installation accessible
list installation repositories
accessible repo IDs + metadata
return repository choices
Select repo,
configurePipeline(projectId, selection)
verify repo belongs to installation; branch available
[accessible] verified repository metadata
lookup one active configuration by project
existing config or none
alt [existing pipeline configuration]
update existing config
else [no pipeline configuration exists]
insert first config
committed connection + configId
repository linked; saved pipeline settings
Show CONNECTED state

Figure 5.4.1

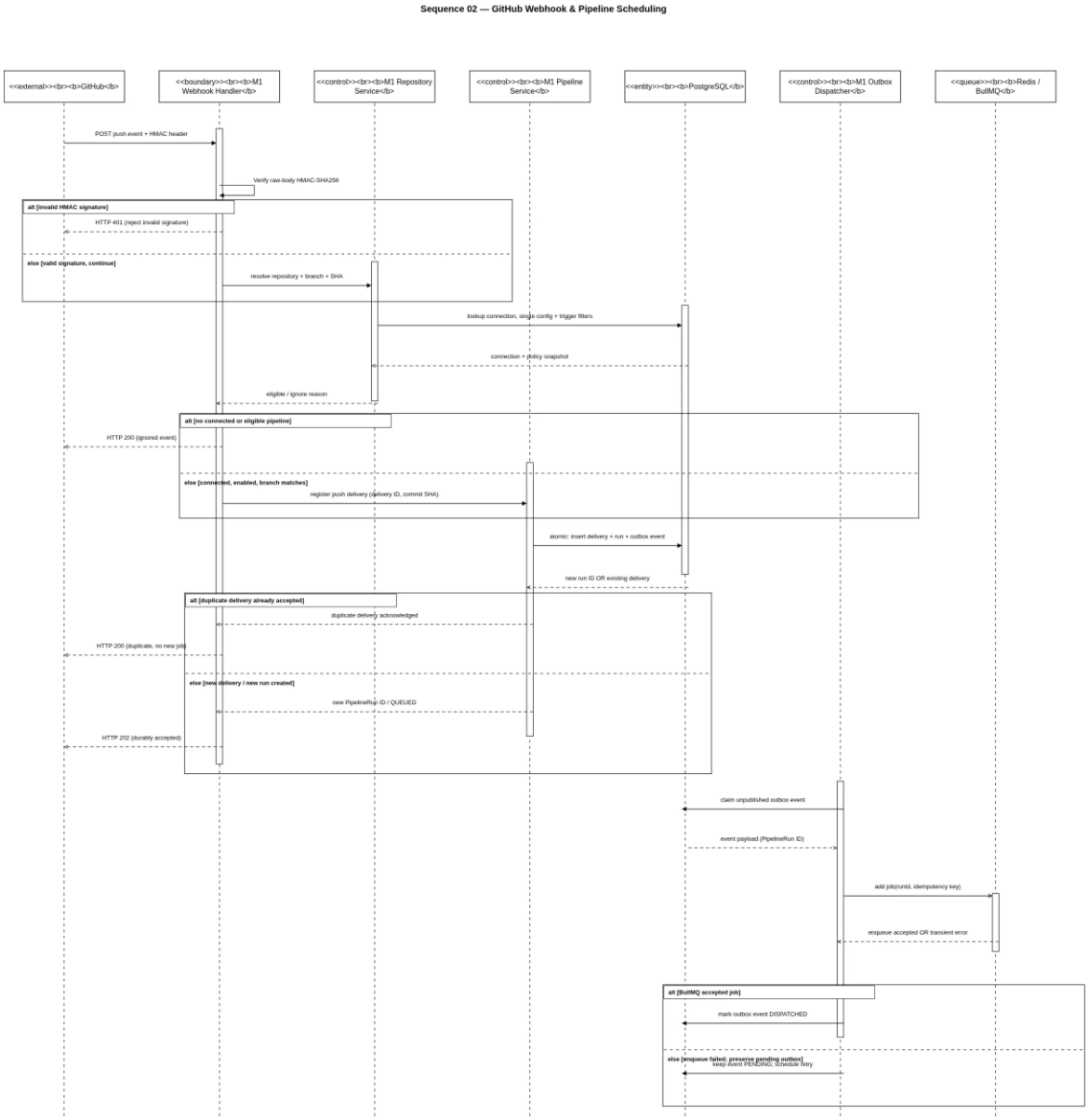

Sequence 02 — GitHub Webhook & Pipeline Scheduling
<<external>>
GitHub
<<boundary>>
M1 Webhook Handler
<<control>>
M1 Repository Service
<<control>>
M1 Pipeline Service
<<entity>>
PostgreSQL
<<control>>
M1 Outbox Dispatcher
<<queue>>
Redis / BullMQ
POST push event + HMAC header
Verify raw-body HMAC-SHA256
alt [invalid HMAC signature]
HTTP 401 (reject invalid signature)
else [valid signature, continue]
resolve repository + branch + SHA
lookup connection, single config + trigger filters
connection + policy snapshot
eligible / ignore reason
alt [no connected or eligible pipeline]
HTTP 200 (ignored event)
else [connected, enabled, branch matches]
register push delivery (delivery ID, commit SHA)
atomic: insert delivery + run + outbox event
new run ID OR existing delivery
alt [duplicate delivery already accepted]
duplicate delivery acknowledged
HTTP 200 (duplicate, no new job)
else [new delivery / new run created]
new PipelineRun ID / QUEUED
HTTP 202 (durably accepted)
claim unpublished outbox event
event payload (PipelineRun ID)
add job(runId, idempotency key)
enqueue accepted OR transient error
alt [BullMQ accepted job]
mark outbox event DISPATCHED
else [enqueue failed; preserve pending outbox]
keep event PENDING; schedule retry

Figure 5.4.2

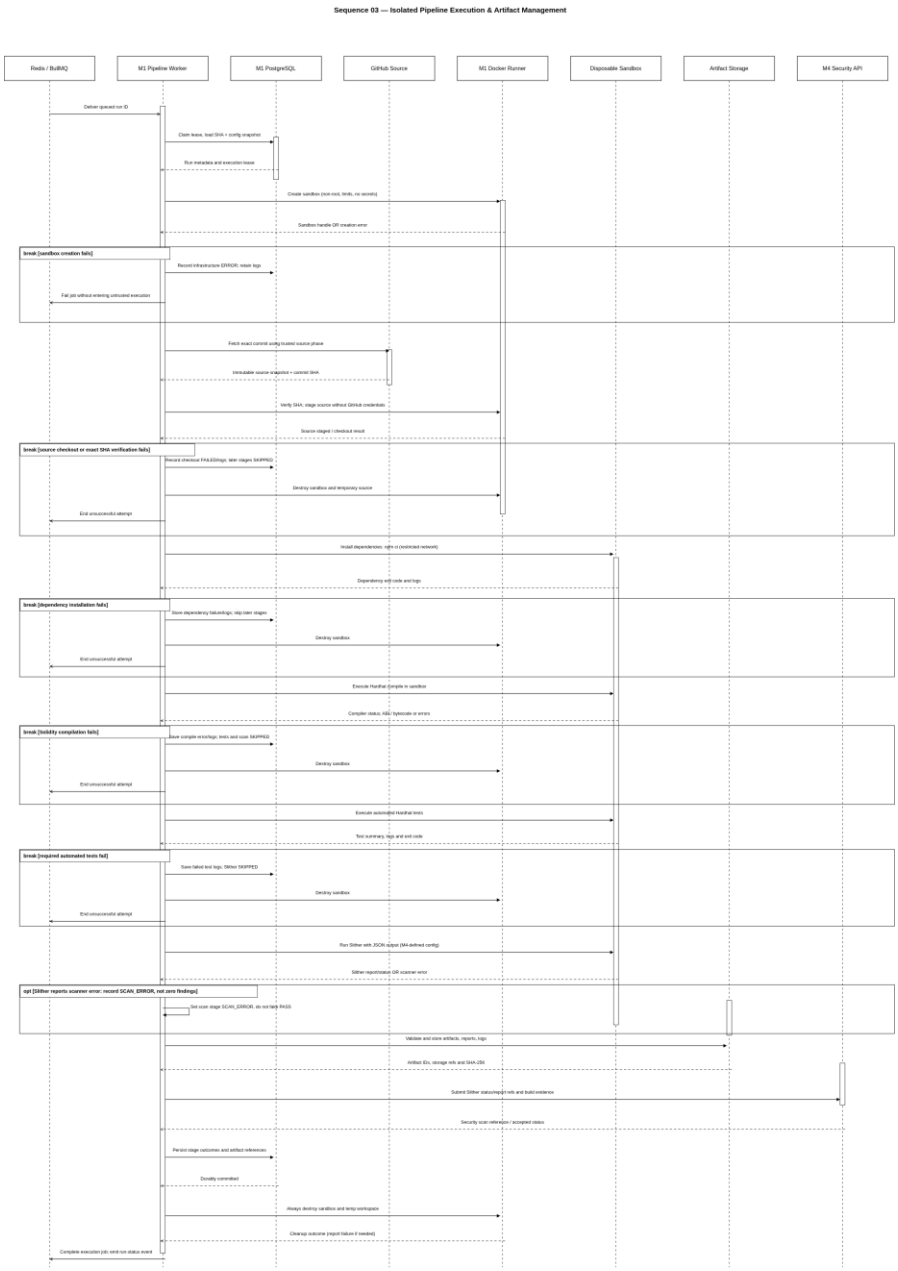

Sequence 03 — Isolated Pipeline Execution & Artifact Management
Redis / BullMQ
M1 Pipeline Worker
M1 PostgreSQL
GitHub Source
M1 Docker Runner
Disposable Sandbox
Artifact Storage
M4 Security API
Deliver queued run ID
Claim lease, load SHA + config snapshot
Run metadata and execution lease
Create sandbox (non-root, limits, no secrets)
Sandbox handle OR creation error
break [sandbox creation fails]
Record infrastructure ERROR; retain logs
Fail job without entering untrusted execution
Fetch exact commit using trusted source phase
Immutable source snapshot + commit SHA
Verify SHA; stage source without GitHub credentials
Source staged / checkout result
break [source checkout or exact SHA verification fails]
Record checkout FAILED/logs; later stages SKIPPED
Destroy sandbox and temporary source
End unsuccessful attempt
Install dependencies; npm ci (restricted network)
Dependency exit code and logs
break [dependency installation fails]
Store dependency failure/logs; skip later stages
Destroy sandbox
End unsuccessful attempt
Execute Hardhat compile in sandbox
Compiler status, ABIs, bytecode or errors
break [Solidity compilation fails]
Save compile errors/logs; tests and scan SKIPPED
Destroy sandbox
End unsuccessful attempt
Execute automated Hardhat tests
Test summary, logs and exit code
break [required automated tests fail]
Save failed test logs; Slither SKIPPED
Destroy sandbox
End unsuccessful attempt
Run Slither with JSON output (M4-defined config)
Slither report/status OR scanner error
opt [Slither reports scanner error; record SCAN_ERROR, not zero findings]
Set scan stage SCAN_ERROR; do not fake PASS
Validate and store artifacts, reports, logs
Artifact IDs, storage refs and SHA-256
Submit Slither status/report refs and build evidence
Security scan reference / accepted status
Persist stage outcomes and artifact references
Durably committed
Always destroy sandbox and temp workspace
Cleanup outcome (report failure if needed)
Complete execution job; emit run status event

Figure 5.4.3

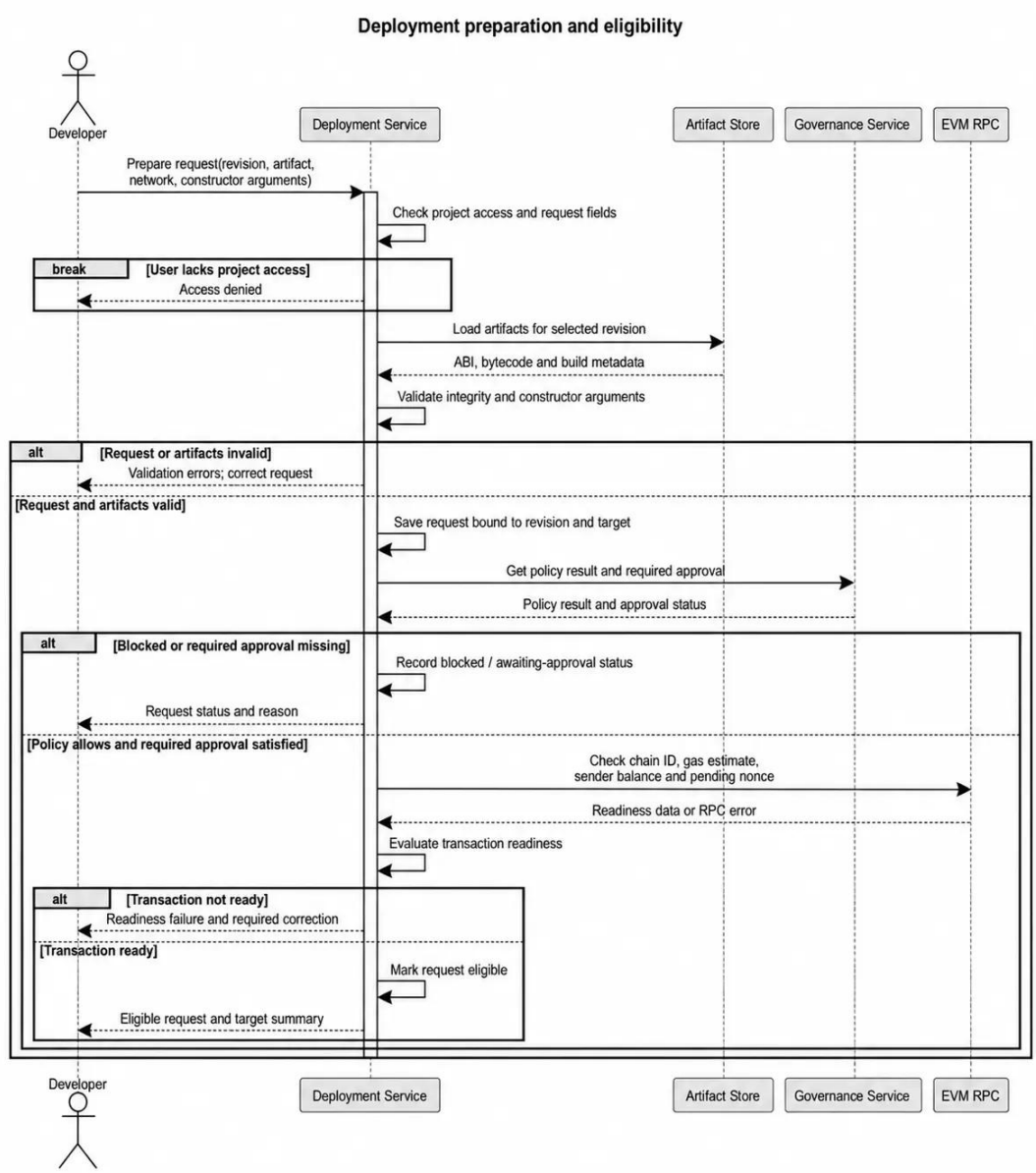

Deployment preparation and eligibility
Developer
Deployment Service
Artifact Store
Governance Service
EVM RPC
Prepare request(revision, artifact, network, constructor arguments)
Check project access and request fields
break
[User lacks project access]
Access denied
Load artifacts for selected revision
ABI, bytecode and build metadata
Validate integrity and constructor arguments
alt
[Request or artifacts invalid]
Validation errors; correct request
[Request and artifacts valid]
Save request bound to revision and target
Get policy result and required approval
Policy result and approval status
alt
[Blocked or required approval missing]
Record blocked / awaiting-approval status
Request status and reason
[Policy allows and required approval satisfied]
Check chain ID, gas estimate, sender balance and pending nonce
Readiness data or RPC error
Evaluate transaction readiness
alt
[Transaction not ready]
Readiness failure and required correction
[Transaction ready]
Mark request eligible
Eligible request and target summary
Developer
Deployment Service
Artifact Store
Governance Service
EVM RPC

Figure 5.4.4

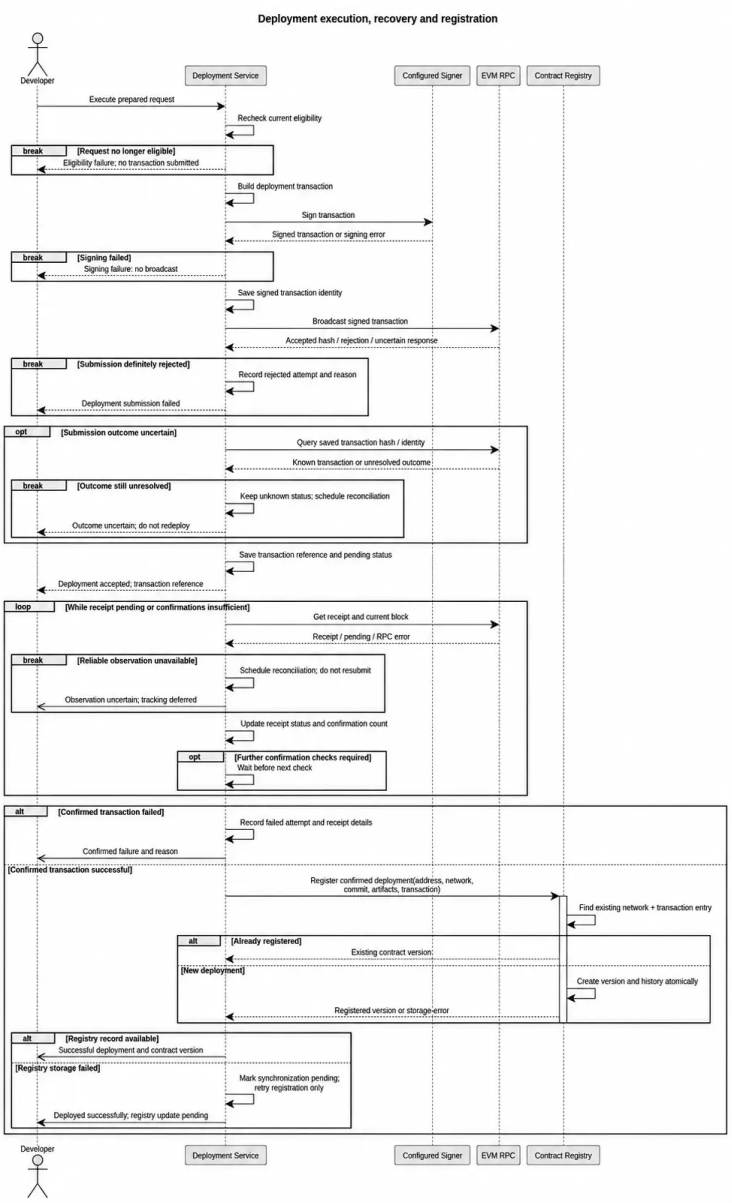

Deployment execution, recovery and registration
Developer
Deployment Service
Configured Signer
EVM RPC
Contract Registry
Execute prepared request
Recheck current eligibility
break
[Request no longer eligible]
Eligibility failure; no transaction submitted
Build deployment transaction
Sign transaction
Signed transaction or signing error
break
[Signing failed]
Signing failure: no broadcast
Save signed transaction identity
Broadcast signed transaction
Accepted hash / rejection / uncertain response
break
[Submission definitely rejected]
Record rejected attempt and reason
Deployment submission failed
opt
[Submission outcome uncertain]
Query saved transaction hash / identity
Known transaction or unresolved outcome
break
[Outcome still unresolved]
Keep unknown status; schedule reconciliation
Outcome uncertain; do not redeploy
Save transaction reference and pending status
Deployment accepted; transaction reference
loop
[While receipt pending or confirmations insufficient]
Get receipt and current block
Receipt / pending / RPC error
break
[Reliable observation unavailable]
Schedule reconciliation; do not resubmit
Observation uncertain; tracking deferred
Update receipt status and confirmation count
opt
[Further confirmation checks required]
Wait before next check
alt
[Confirmed transaction failed]
Record failed attempt and receipt details
Confirmed failure and reason
[Confirmed transaction successful]
Register confirmed deployment(address, network, commit, artifacts, transaction)
Find existing network + transaction entry
alt
[Already registered]
Existing contract version
[New deployment]
Create version and history atomically
Registered version or storage-error
alt
[Registry record available]
Successful deployment and contract version
[Registry storage failed]
Mark synchronization pending; retry registration only
Deployed successfully; registry update pending
Developer
Deployment Service
Configured Signer
EVM RPC
Contract Registry

Figure 5.4.5

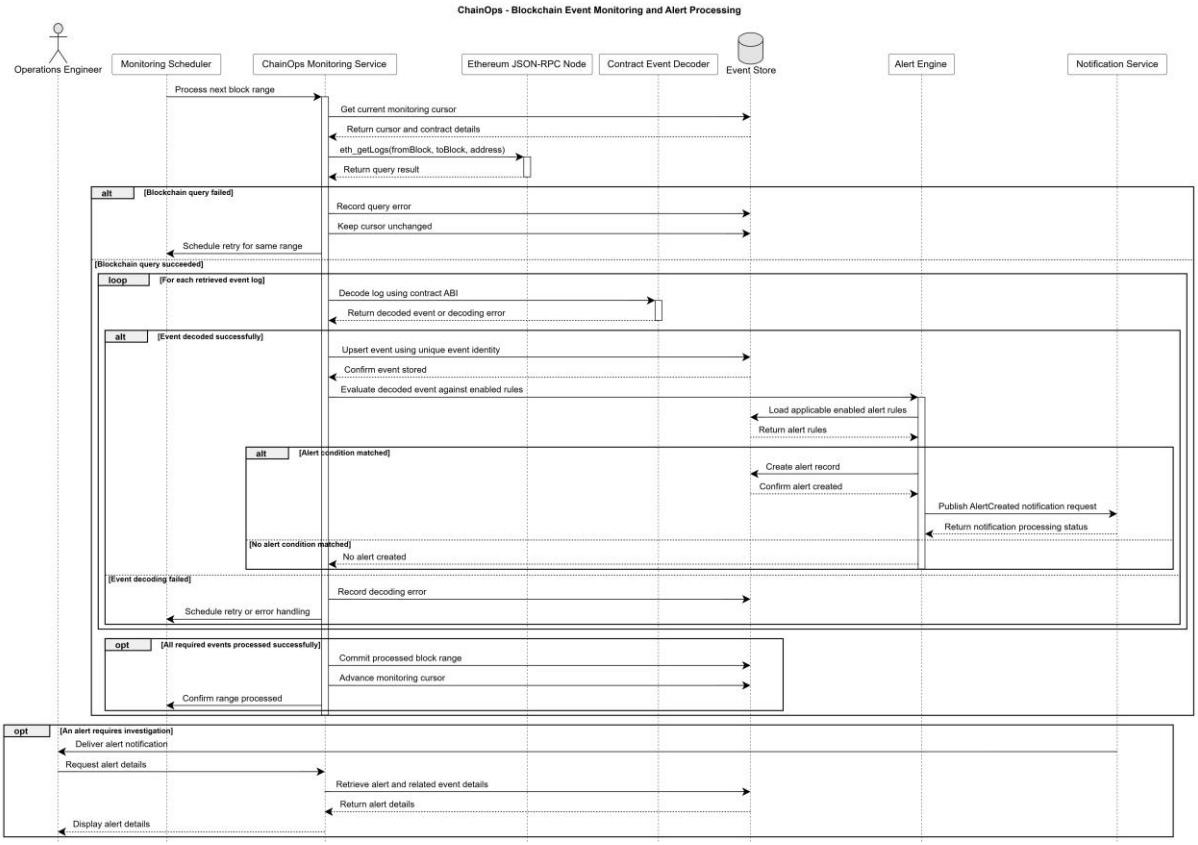

ChainOps - Blockchain Event Monitoring and Alert Processing
Operations Engineer
Monitoring Scheduler
ChainOps Monitoring Service
Ethereum JSON-RPC Node
Contract Event Decoder
Event Store
Alert Engine
Notification Service
Process next block range
Get current monitoring cursor
Return cursor and contract details
eth_getLogs(fromBlock, toBlock, address)
Return query result
alt
[Blockchain query failed]
Record query error
Keep cursor unchanged
Schedule retry for same range
[Blockchain query succeeded]
loop
[For each retrieved event log]
Decode log using contract ABI
Return decoded event or decoding error
alt
[Event decoded successfully]
Upsert event using unique event identity
Confirm event stored
Evaluate decoded event against enabled rules
Load applicable enabled alert rules
Return alert rules
alt
[Alert condition matched]
Create alert record
Confirm alert created
Publish AlertCreated notification request
Return notification processing status
[No alert condition matched]
No alert created
[Event decoding failed]
Record decoding error
Schedule retry or error handling
opt
[All required events processed successfully]
Commit processed block range
Advance monitoring cursor
Confirm range processed
opt
[An alert requires investigation]
Deliver alert notification
Request alert details
Retrieve alert and related event details
Return alert details
Display alert details

Figure 5.4.6

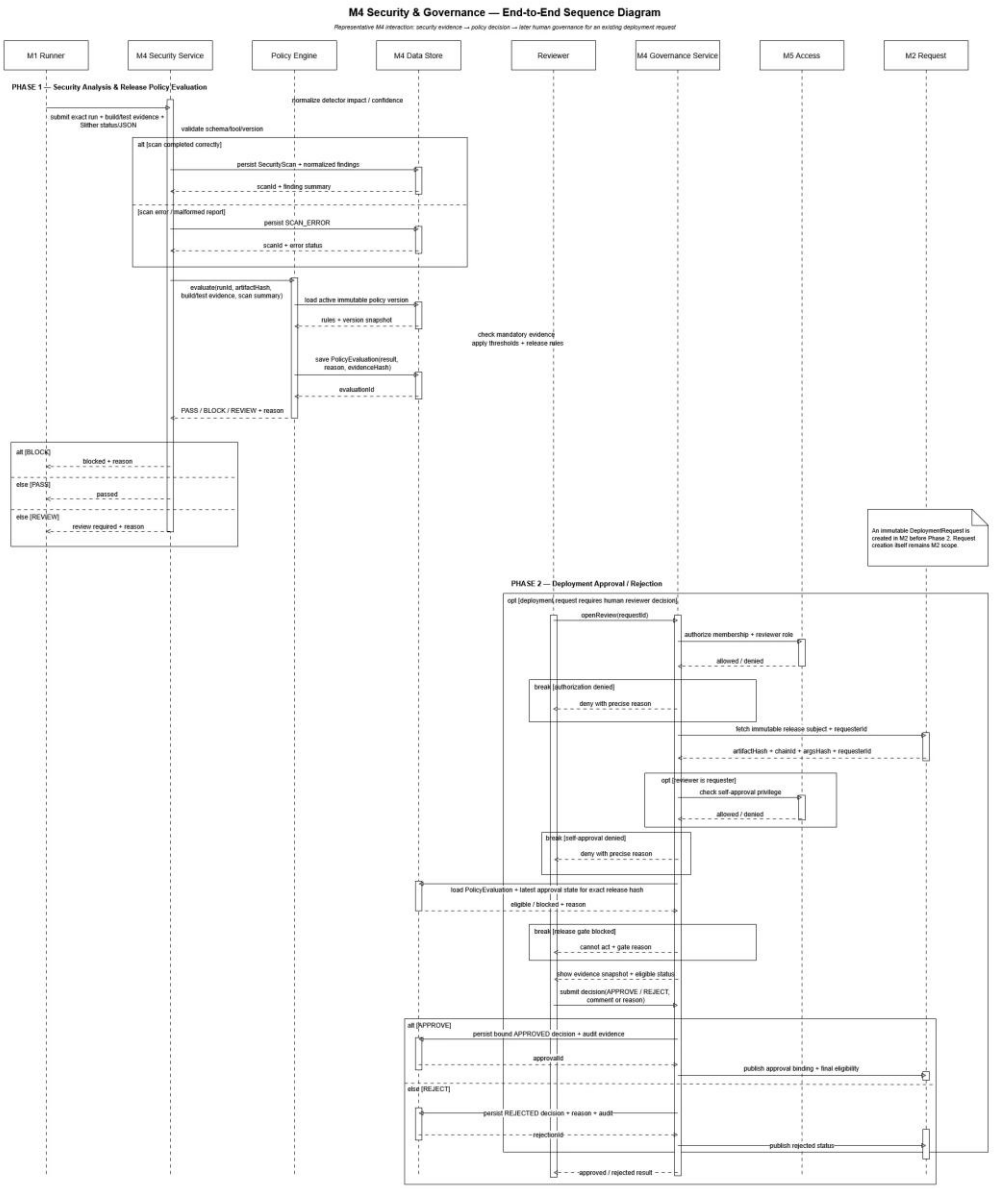
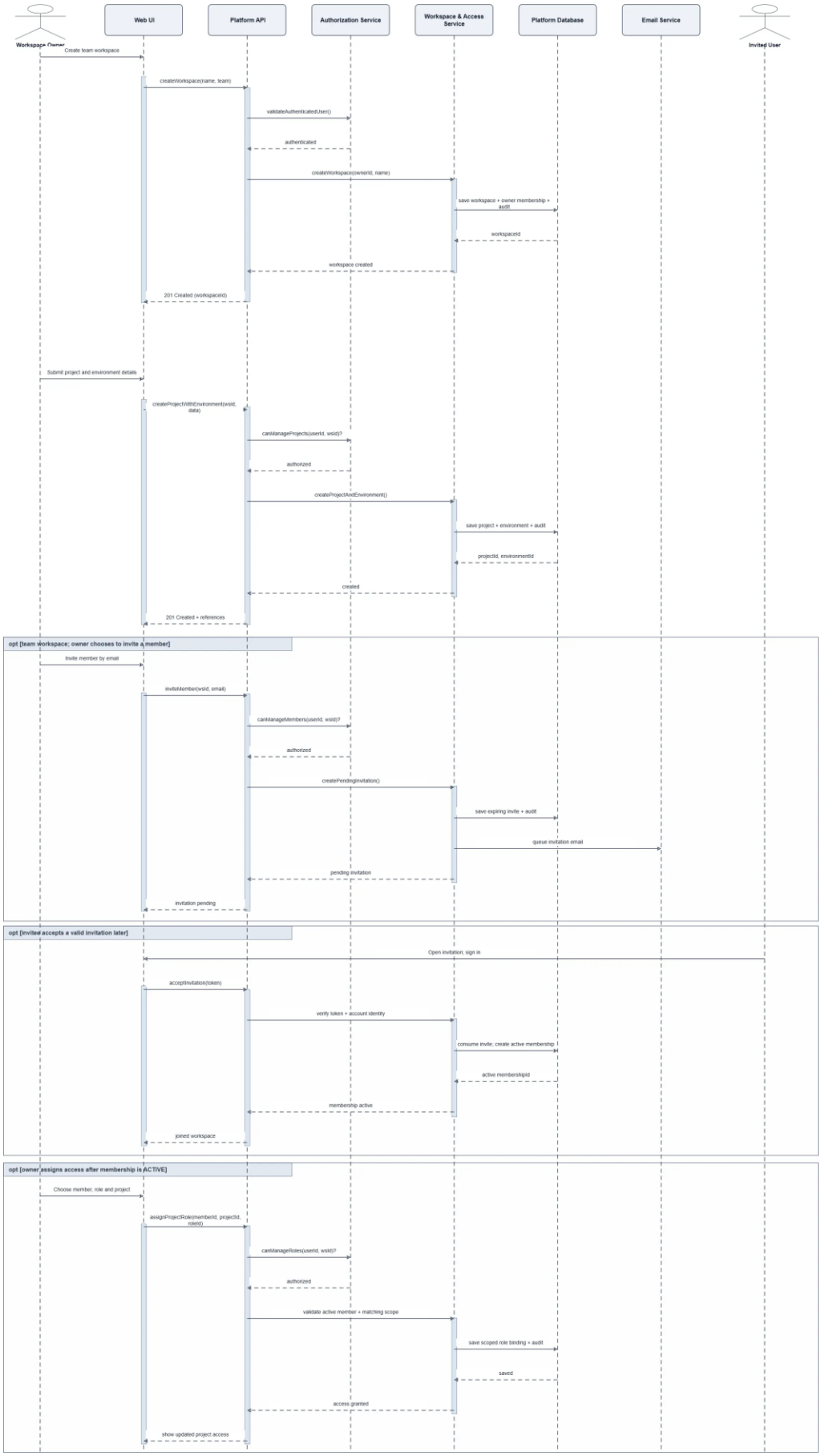

M4 Security & Governance — End-to-End Sequence Diagram
Representative M4 interaction: security evidence → policy decision → later human governance for an existing deployment request
M1 Runner
M4 Security Service
Policy Engine
M4 Data Store
Reviewer
M4 Governance Service
M5 Access
M2 Request
PHASE 1 — Security Analysis & Release Policy Evaluation
normalize detector impact / confidence
submit exact run + build/test evidence + Slither status/JSON
validate schema/tool/version
alt [scan completed correctly]
persist SecurityScan + normalized findings
scanId + finding summary
[scan error / malformed report]
persist SCAN_ERROR
scanId + error status
evaluate(runId, artifactHash, build/test evidence, scan summary)
load active immutable policy version
rules + version snapshot
check mandatory evidence
apply thresholds + release rules
save PolicyEvaluation(result, reason, evidenceHash)
evaluationId
PASS / BLOCK / REVIEW + reason
alt [BLOCK]
blocked + reason
else [PASS]
passed
else [REVIEW]
review required + reason
An immutable DeploymentRequest is created in M2 before Phase 2. Request creation itself remains M2 scope.
PHASE 2 — Deployment Approval / Rejection
opt [deployment request requires human reviewer decision]
openReview(requestId)
authorize membership + reviewer role
allowed / denied
break [authorization denied]
deny with precise reason
fetch immutable release subject + requesterId
artifactHash + chainId + argsHash + requesterId
opt [reviewer is requester]
check self-approval privilege
allowed / denied
break [self-approval denied]
deny with precise reason
load PolicyEvaluation + latest approval state for exact release hash
eligible / blocked + reason
break [release gate blocked]
cannot act + gate reason
show evidence snapshot + eligible status
submit decision(APPROVE / REJECT, comment or reason)
alt [APPROVE]
persist bound APPROVED decision + audit evidence
approvalId
publish approval binding + final eligibility
else [REJECT]
persist REJECTED decision + reason + audit
rejectionId
publish rejected status
approved / rejected result

Figure 5.4.7

ChainOps - Workspace, Project & Access Management

Single success scenario; authentication assumed; invitation acceptance precedes role grant



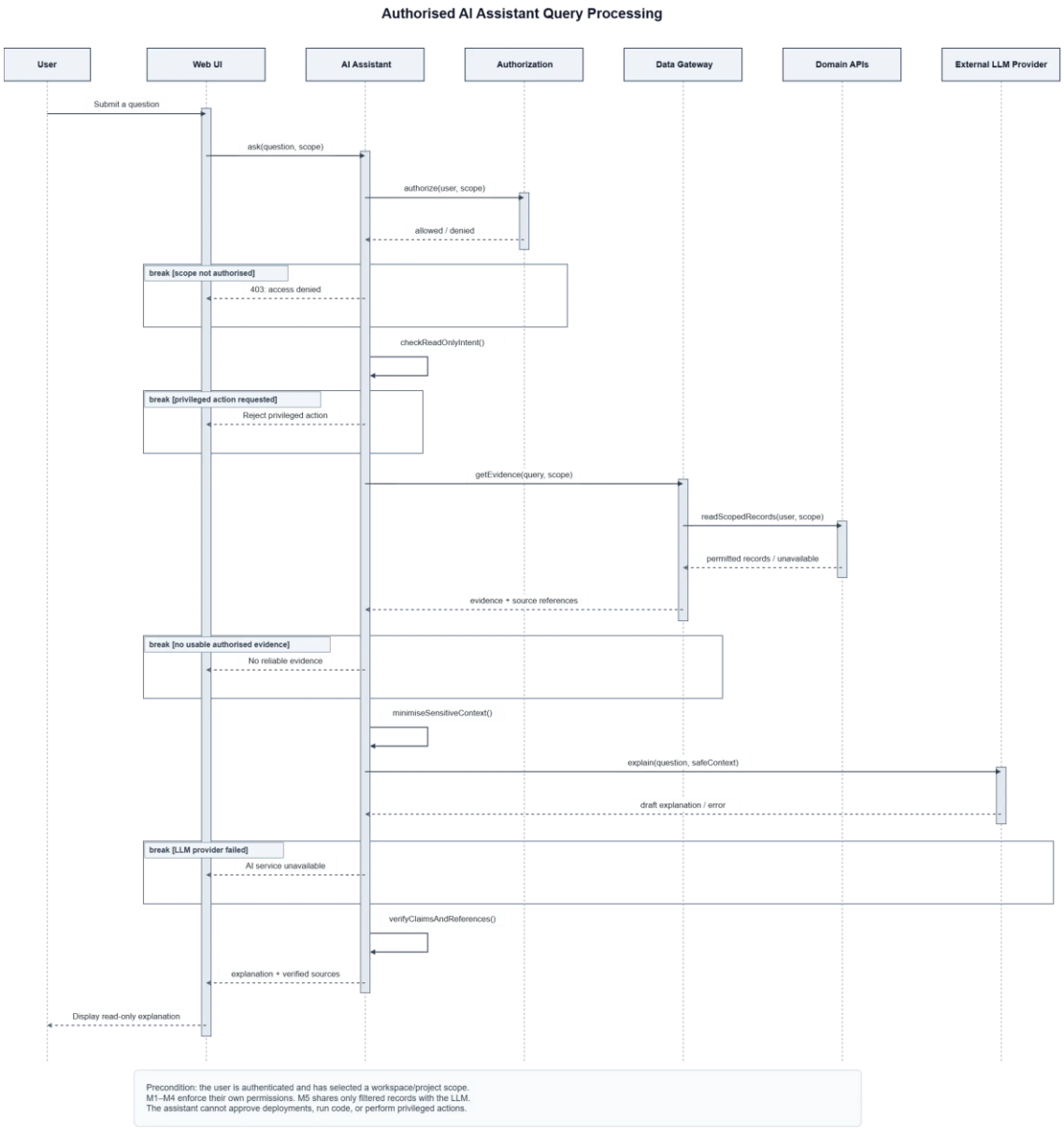

Authorised AI Assistant Query Processing
User
Web UI
AI Assistant
Authorization
Data Gateway
Domain APIs
External LLM Provider
Submit a question
ask(question, scope)
authorize(user, scope)
allowed / denied
break [scope not authorised]
403: access denied
checkReadOnlyIntent()
break [privileged action requested]
Reject privileged action
getEvidence(query, scope)
readScopedRecords(user, scope)
permitted records / unavailable
evidence + source references
break [no usable authorised evidence]
No reliable evidence
minimiseSensitiveContext()
explain(question, safeContext)
draft explanation / error
break [LLM provider failed]
AI service unavailable
verifyClaimsAndReferences()
explanation + verified sources
Display read-only explanation
Precondition: the user is authenticated and has selected a workspace/project scope.
M1–M4 enforce their own permissions. M5 shares only filtered records with the LLM.
The assistant cannot approve deployments, run code, or perform privileged actions.

Figure 5.4.9

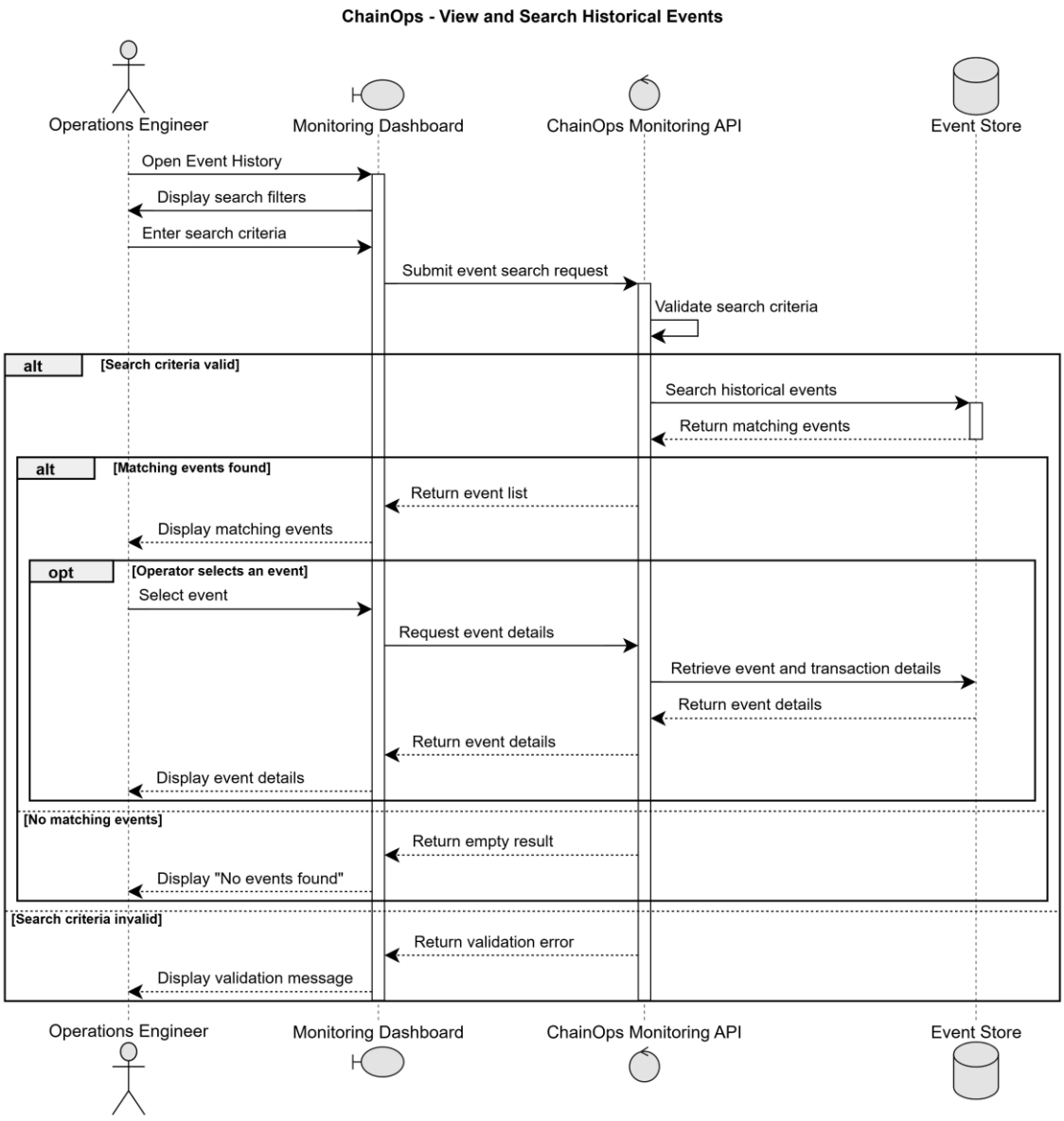

ChainOps - View and Search Historical Events
Operations Engineer
Monitoring Dashboard
ChainOps Monitoring API
Event Store
Open Event History
Display search filters
Enter search criteria
Submit event search request
Validate search criteria
alt
[Search criteria valid]
Search historical events
Return matching events
alt
[Matching events found]
Return event list
Display matching events
opt
[Operator selects an event]
Select event
Request event details
Retrieve event and transaction details
Return event details
Return event details
Display event details
[No matching events]
Return empty result
Display "No events found"
[Search criteria invalid]
Return validation error
Display validation message
Operations Engineer
Monitoring Dashboard
ChainOps Monitoring API
Event Store

Figure 5.4.10

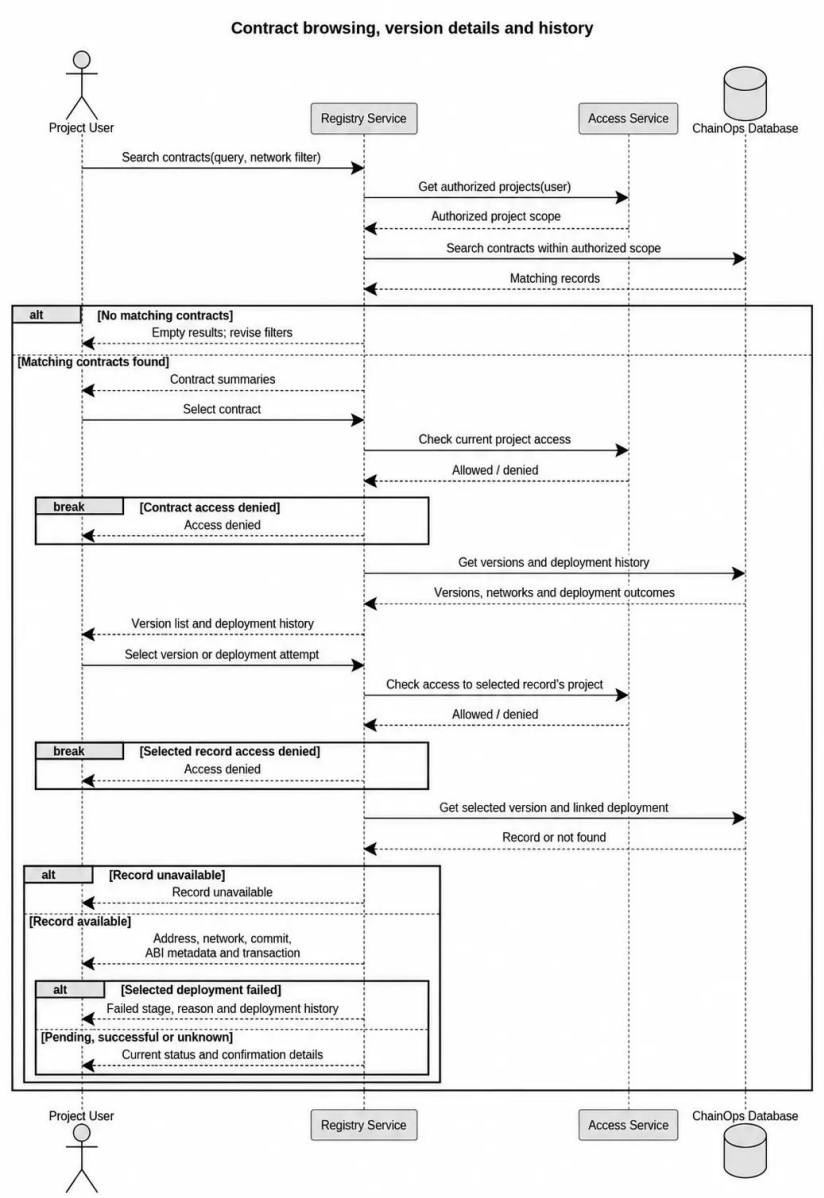
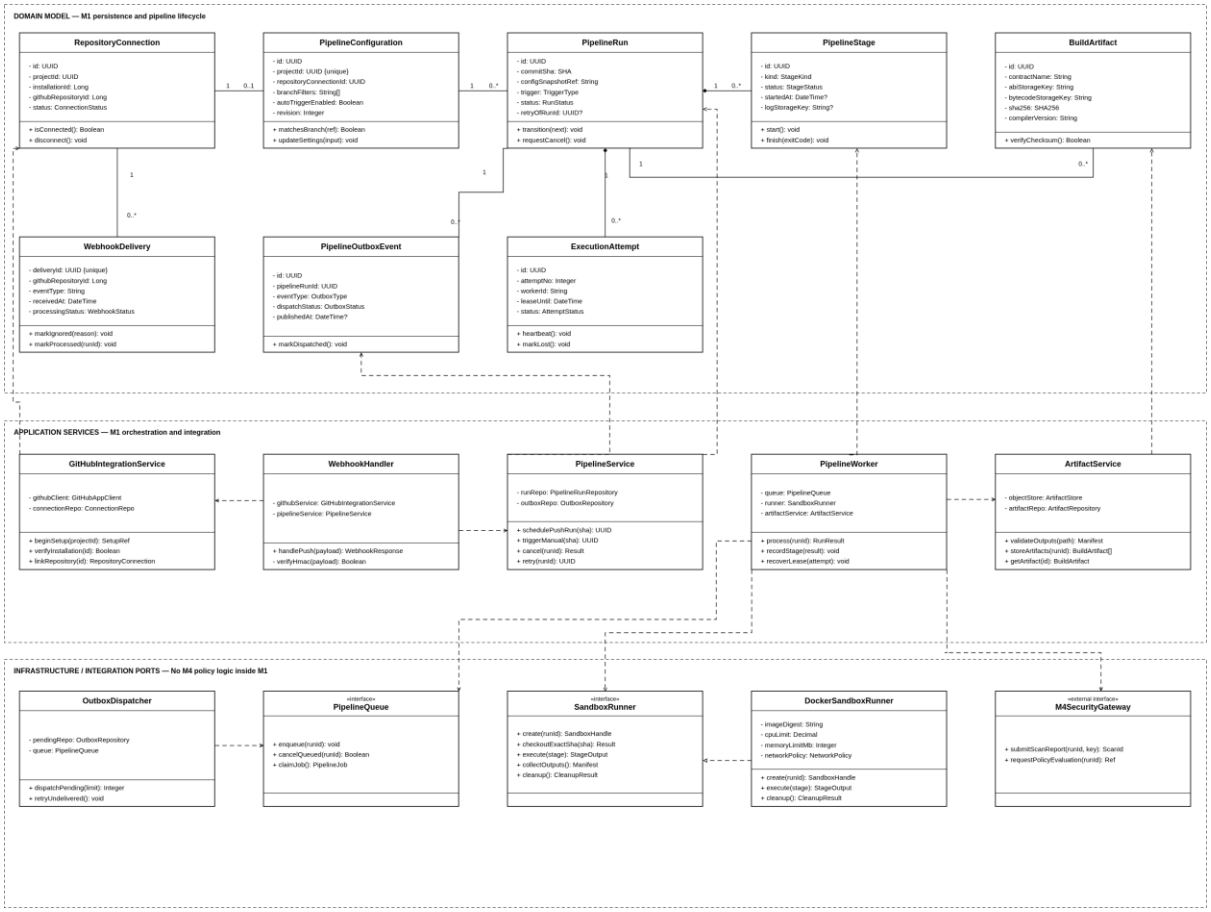


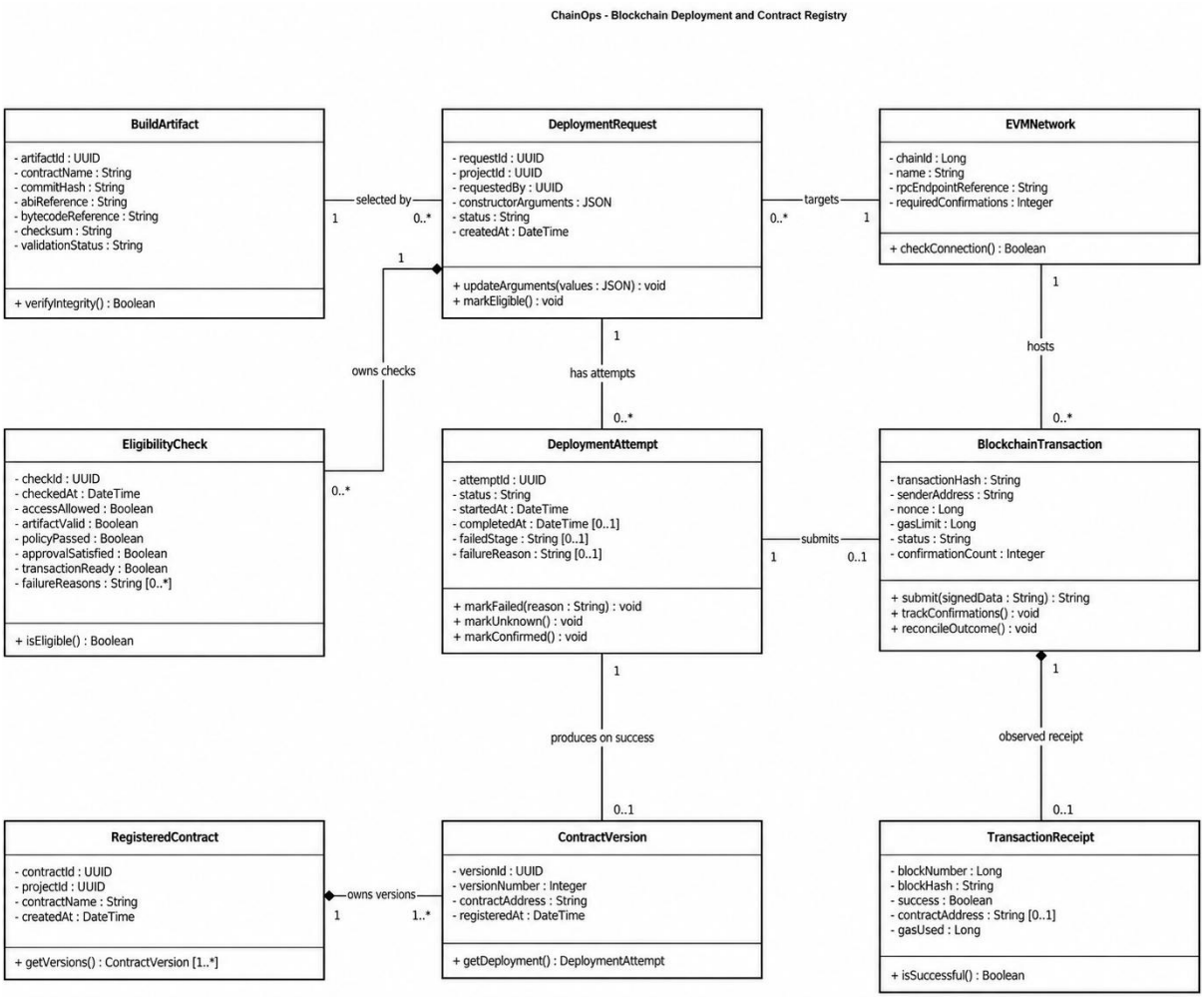
Figure 5.4.11

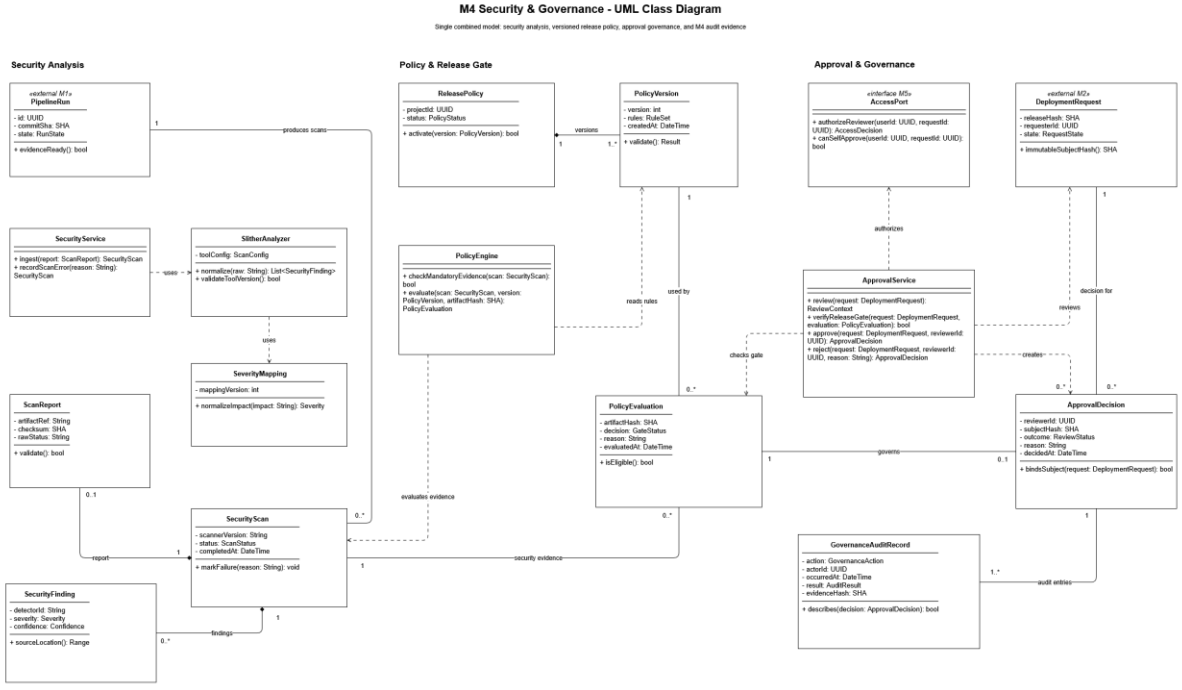
5.5 Class diagram

The class diagram represents the structural design of the ChainOps platform by illustrating its main classes, attributes, methods, and relationships. It demonstrates how different classes are organized and connected through associations, inheritance, aggregation, and composition where applicable. This diagram provides a foundation for understanding the object-oriented structure and relationships between the major system components.

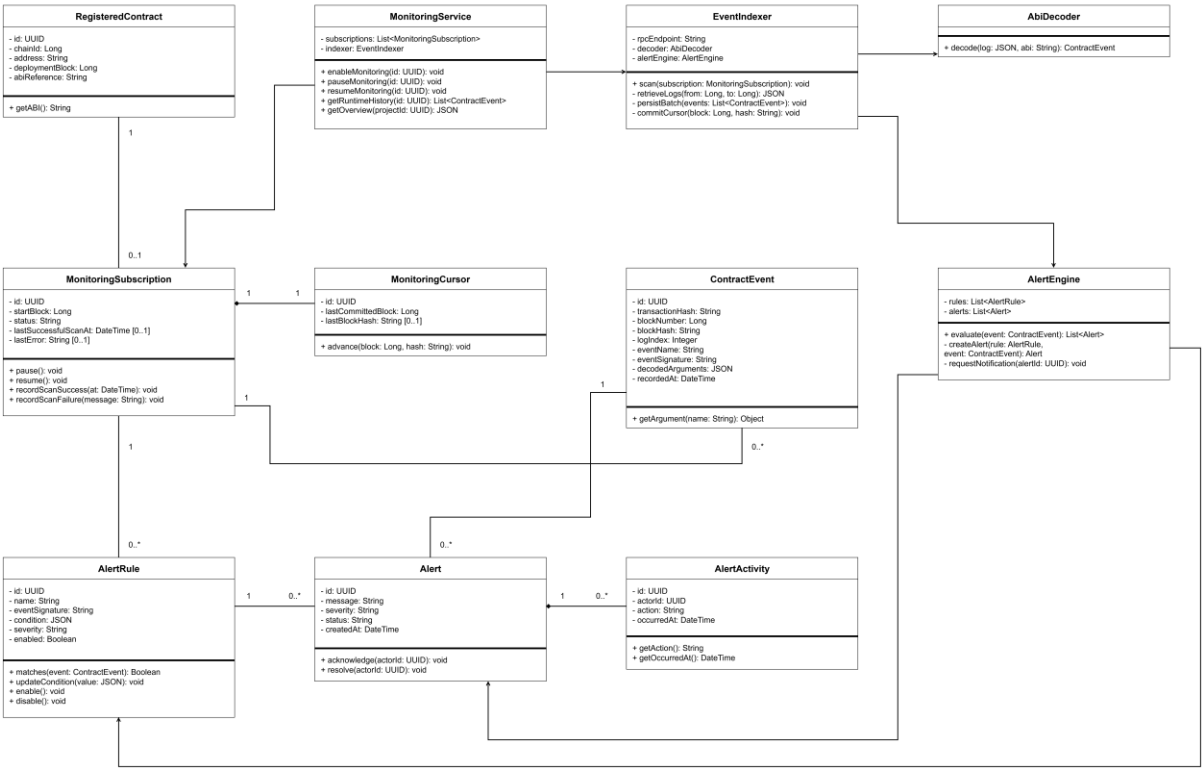
UML Class Diagram — Domain entities, application services and infrastructure ports

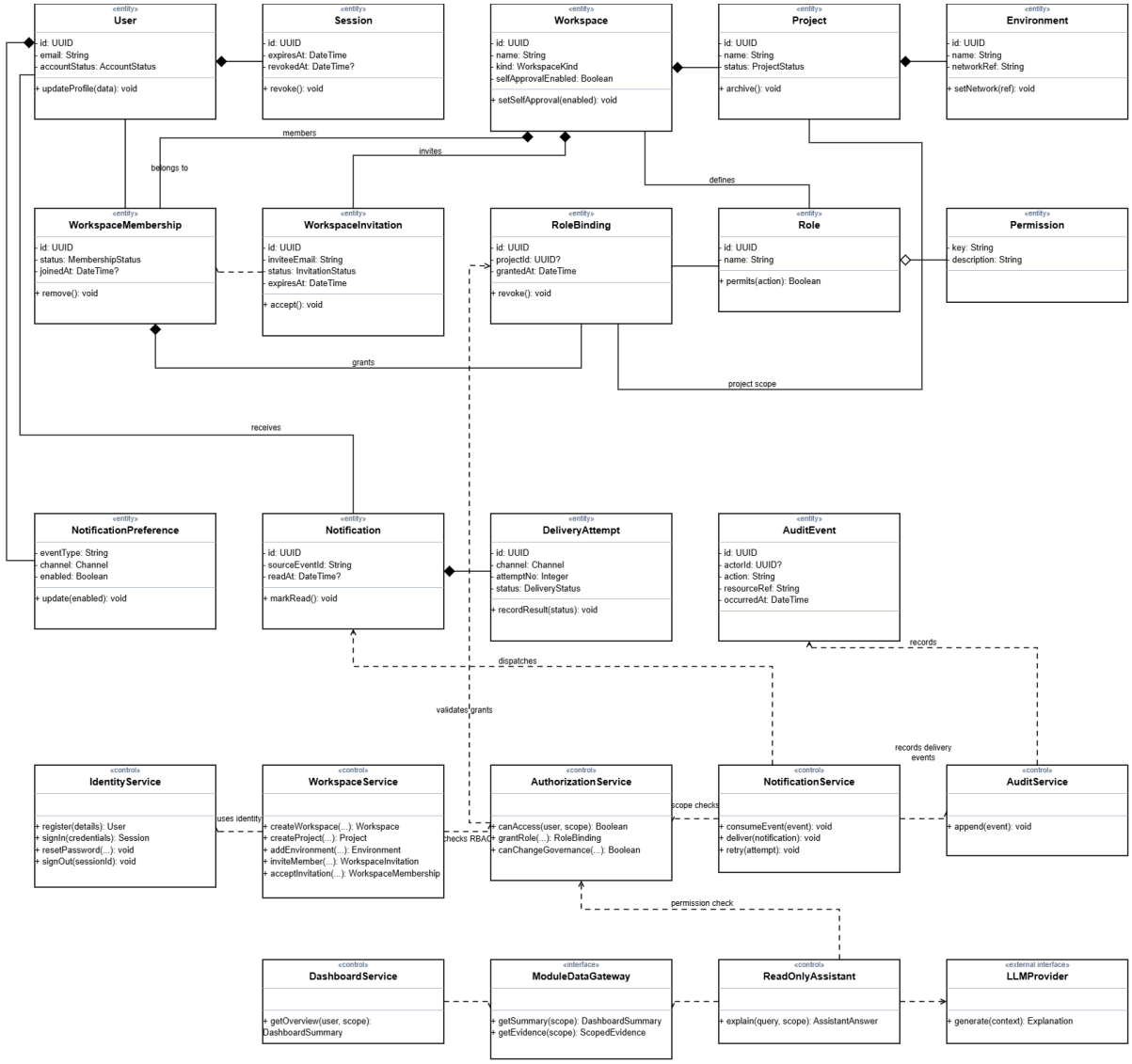



ChainOps - Blockchain Deployment and Contract Registry
BuildArtifact
- artifactId : UUID
- contractName : String
- commitSha : String
- abiReference : String
- bytecodeReference : String
- checksum : String
- validationStatus : String
+ verifyIntegrity() : Boolean
selected by
1
0..*
DeploymentRequest
- requestId : UUID
- projectId : UUID
- requestedBy : UUID
- constructorArguments : JSON
- status : String
- createdAt : DateTime
+ updateArguments(values : JSON) : void
+ markEligible() : void
targets
0..*
1
EVMNetwork
- chainId : Long
- name : String
- rpcEndpointReference : String
- requiredConfirmations : Integer
+ checkConnection() : Boolean
1
hosts
0..*
1
owns checks
0..*
1
has attempts
0..*
EligibilityCheck
- checkId : UUID
- checkedAt : DateTime
- accessAllowed : Boolean
- artifactValid : Boolean
- policyPassed : Boolean
- approvalSatisfied : Boolean
- transactionReady : Boolean
- failureReasons : String [0..*]
+ isEligible() : Boolean
DeploymentAttempt
- attemptId : UUID
- status : String
- startedAt : DateTime
- completedAt : DateTime [0..1]
- failedStage : String [0..1]
- failureReason : String [0..1]
+ markFailed(reason : String) : void
+ markUnknown() : void
+ markConfirmed() : void
submits
1
0..1
BlockchainTransaction
- transactionHash : String
- senderAddress : String
- nonce : Long
- gasLimit : Long
- status : String
- confirmationCount : Integer
+ submit(signedData : String) : String
+ trackConfirmations() : void
+ reconcileOutcome() : void
1
produces on success
0..1
1
observed receipt
0..1
RegisteredContract
- contractId : UUID
- projectId : UUID
- contractName : String
- createdAt : DateTime
+ getVersions() : ContractVersion [1..*]
owns versions
1
1..*
ContractVersion
- versionId : UUID
- versionNumber : Integer
- contractAddress : String
- registeredAt : DateTime
+ getDeployment() : DeploymentAttempt
TransactionReceipt
- blockNumber : Long
- blockHash : String
- success : Boolean
- contractAddress : String [0..1]
- gasUsed : Long
+ isSuccessful() : Boolean


M4 Security & Governance - UML Class Diagram
Single combined model: security analysis, versioned release policy, approval governance, and M4 audit evidence
Security Analysis
Policy & Release Gate
Approval & Governance
«external M1»
PipelineRun
- id: UUID
- commitSha: SHA
- state: RunState
+ evidenceReady(): bool
produces scans
ReleasePolicy
- projectId: UUID
- status: PolicyStatus
+ activate(version: PolicyVersion): bool
versions
PolicyVersion
- version: Int
- rules: RuleSet
- createdAt: DateTime
+ validate(): Result
«interface M5»
AccessPort
+ authorizeReviewer(userId: UUID, requestId: UUID): AccessDecision
+ canSelfApprove(userId: UUID, requestId: UUID): bool
«external M2»
DeploymentRequest
- releaseHash: SHA
- requestedId: UUID
- state: RequestState
+ immutableSubjectHash(): SHA
authorizes
SecurityService
+ ingest(report: ScanReport): SecurityScan
+ recordScanError(reason: String): SecurityScan
uses
SlitherAnalyzer
- toolConfig: ScanConfig
+ normalize(raw: String): List<SecurityFinding>
+ validateToolVersion(): bool
uses
SeverityMapping
- mappingVersion: Int
+ normalizeImpact(impact: String): Severity
PolicyEngine
+ checkMandatoryEvidence(scan: SecurityScan): bool
+ evaluate(scan: SecurityScan, version: PolicyVersion, artifactHash: SHA): PolicyEvaluation
reads rules
used by
ApprovalService
+ review(request: DeploymentRequest): ReviewContext
+ verifyReleaseGate(request: DeploymentRequest, evaluation: PolicyEvaluation): bool
+ approve(request: DeploymentRequest, reviewerId: UUID): ApprovalDecision
+ reject(request: DeploymentRequest, reviewerId: UUID, reason: String): ApprovalDecision
decision for
reviews
creates
checks gate
ScanReport
- artifactRef: String
- checksum: SHA
- rawStatus: String
+ validate(): bool
report
SecurityScan
- scannerVersion: String
- status: ScanStatus
- completedAt: DateTime
+ markFailure(reason: String): void
evaluates evidence
security evidence
PolicyEvaluation
- artifactHash: SHA
- decision: GateStatus
- reason: String
- evaluatedAt: DateTime
+ isEligible(): bool
governs
ApprovalDecision
- reviewerId: UUID
- subjectHash: SHA
- outcome: ReviewStatus
- reason: String
- decidedAt: DateTime
+ bindsSubject(request: DeploymentRequest): bool
SecurityFinding
- detectorId: String
- severity: Severity
- confidence: Confidence
+ sourceLocation(): Range
findings
GovernanceAuditRecord
- action: GovernanceAction
- actorId: UUID
- occurredAt: DateTime
- result: AuditResult
- evidenceHash: SHA
+ describes(decision: ApprovalDecision): bool
audit entries

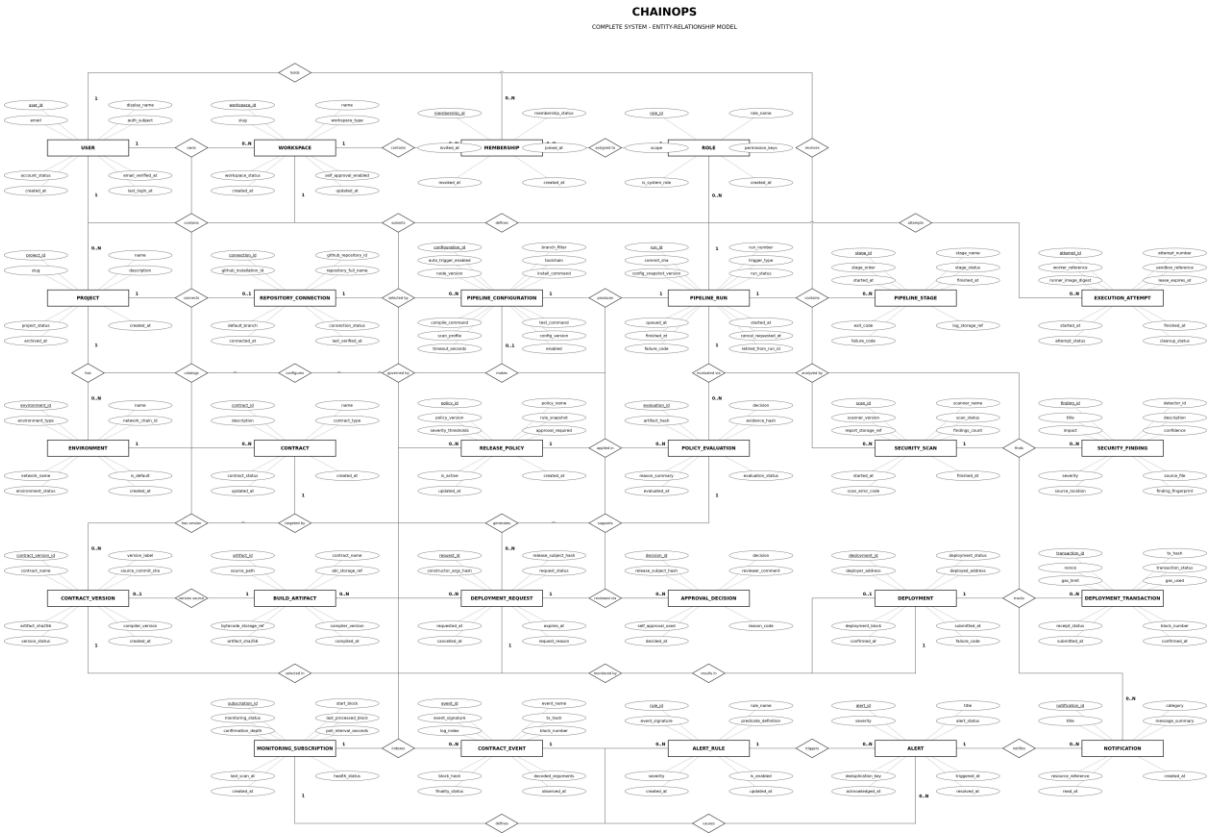
ChainOps - Monitoring, Events & Alerts | Final Proposed Class Diagram





5.6 ER diagram

The Entity-Relationship (ER) diagram illustrates the database structure of ChainOps by identifying the main entities, their attributes, and the relationships between them. It represents how essential information, such as user accounts, projects, pipeline executions, security findings, and deployment records, is organized and connected. This diagram supports the development of a structured relational database while maintaining data consistency and integrity.



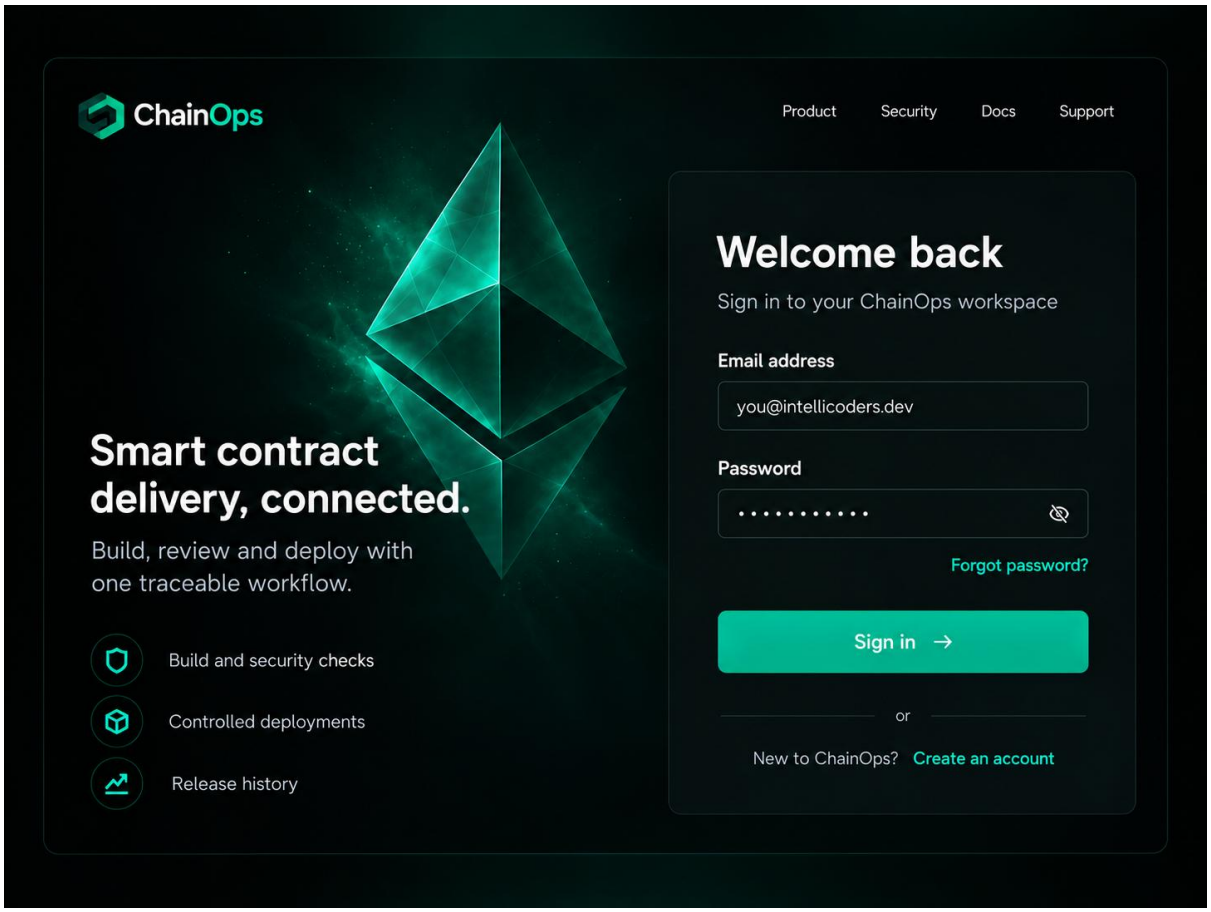
https://drive.google.com/file/d/13-yzIiSn16G8BCJO6oOFODWr_Y778vRV/view?usp=sharing

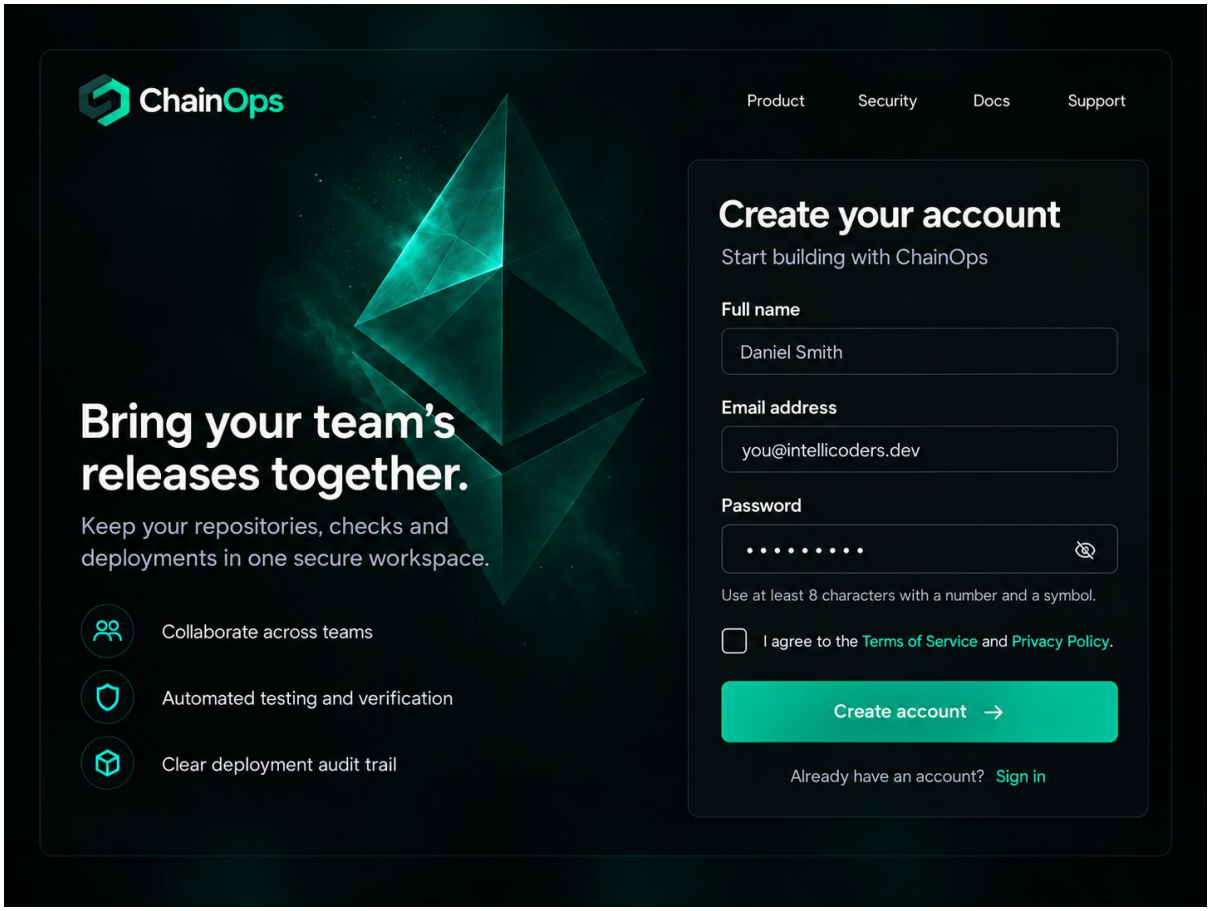
5.7 Summary

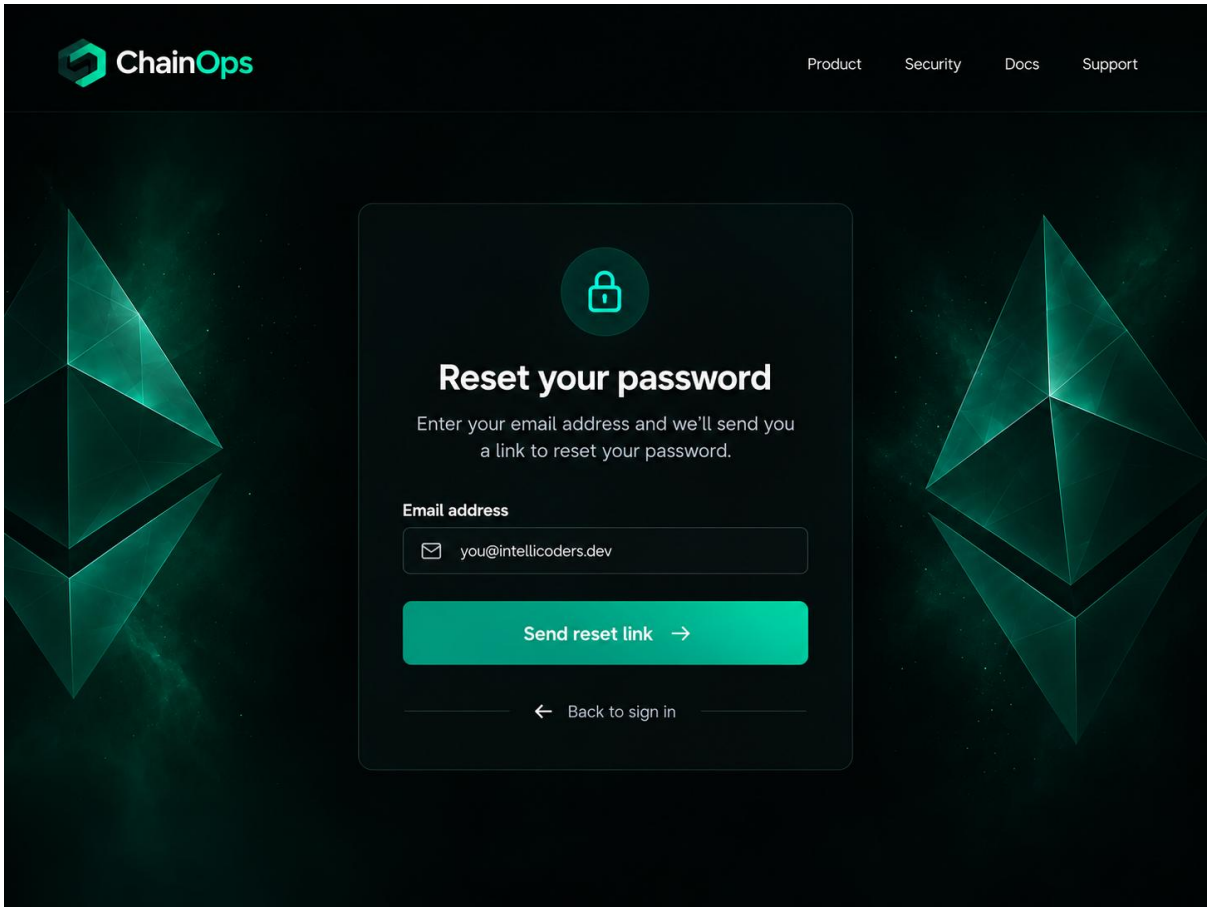
This chapter discussed the analysis and design of the ChainOps platform, including system requirements, user interactions, functional workflows, and database relationships. The proposed architecture and design models establish a clear understanding of the system structure and interactions between its major components. These specifications provide a foundation for the implementation phase, supporting consistency, security, and maintainability throughout the development process.

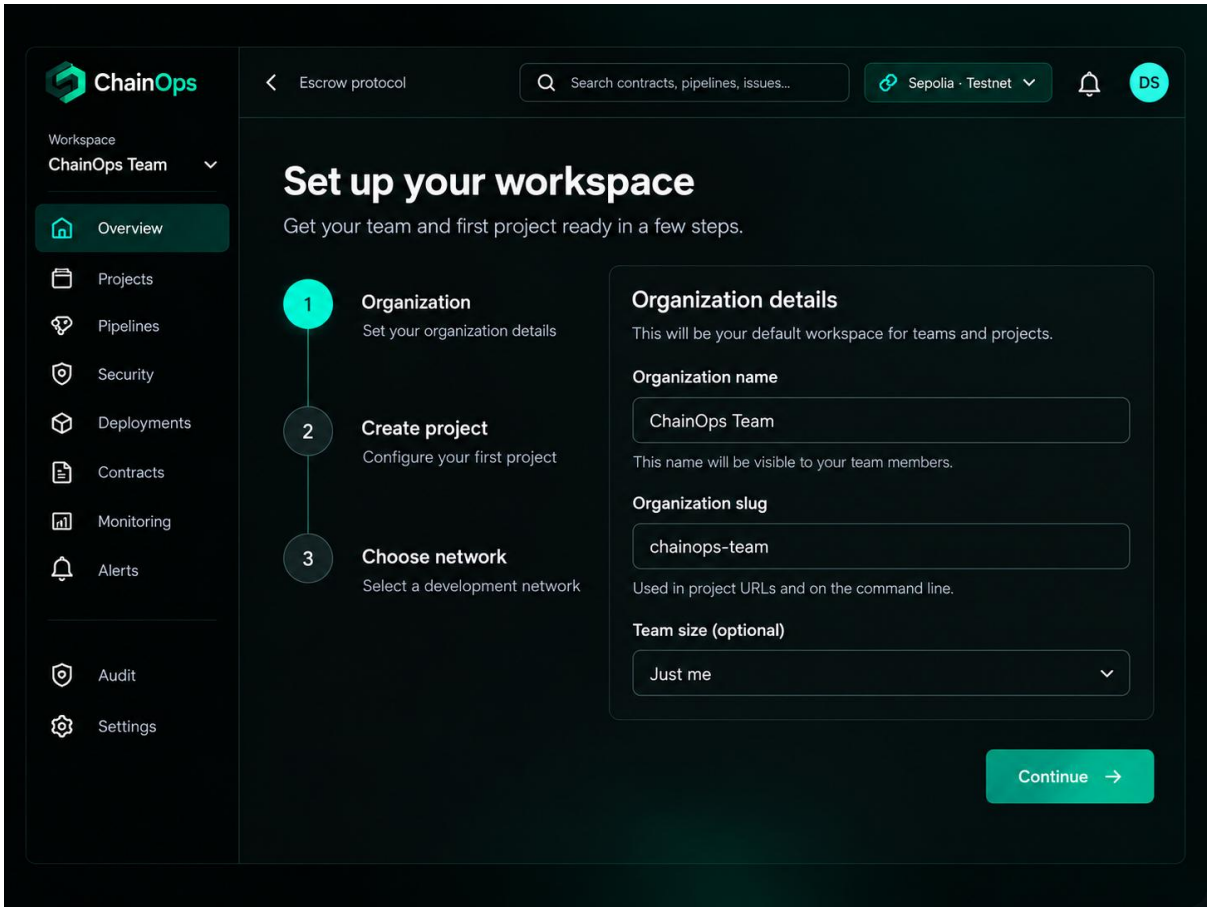
5.8 UI Design

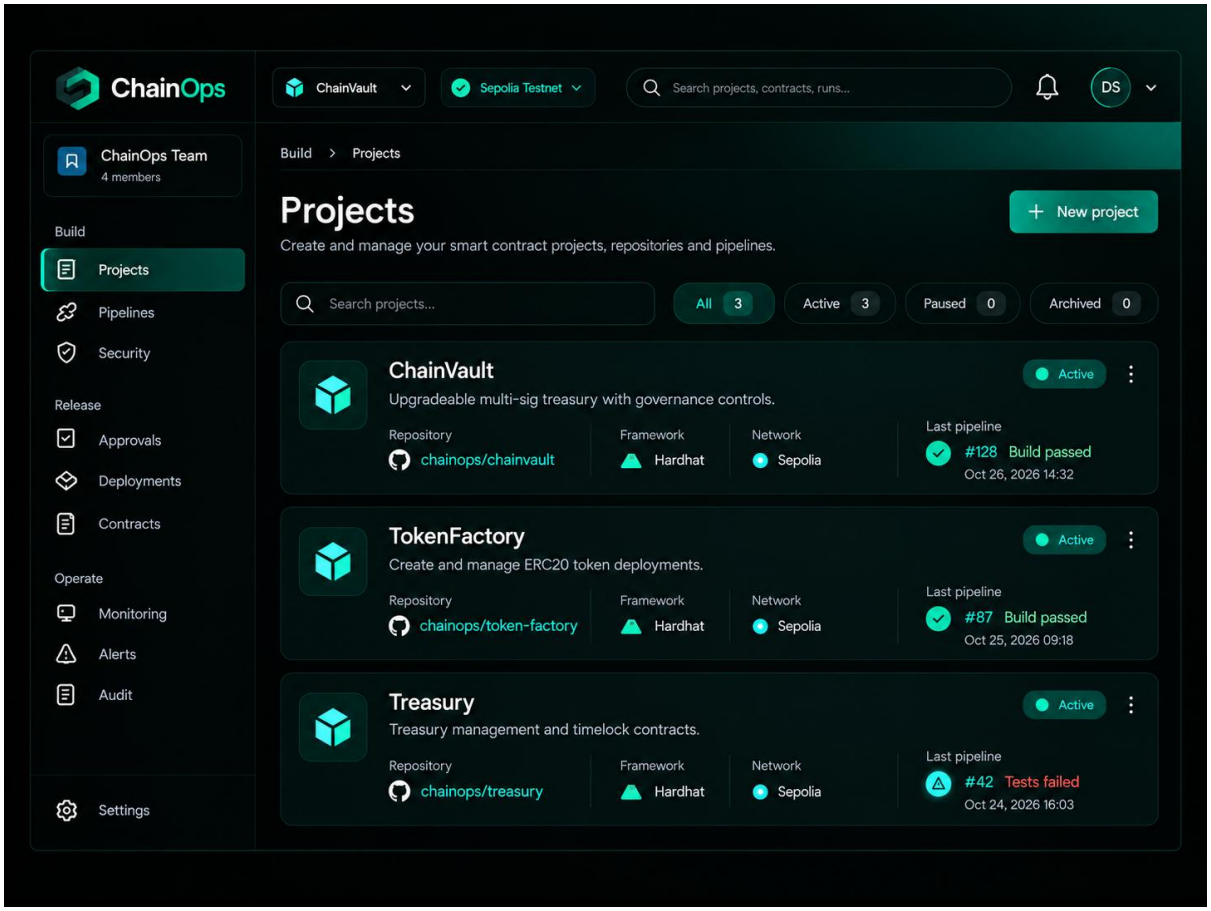
https://www.figma.com/design/wr9kWmDlC4csbPMYLH9vGt?node-id=157-10507

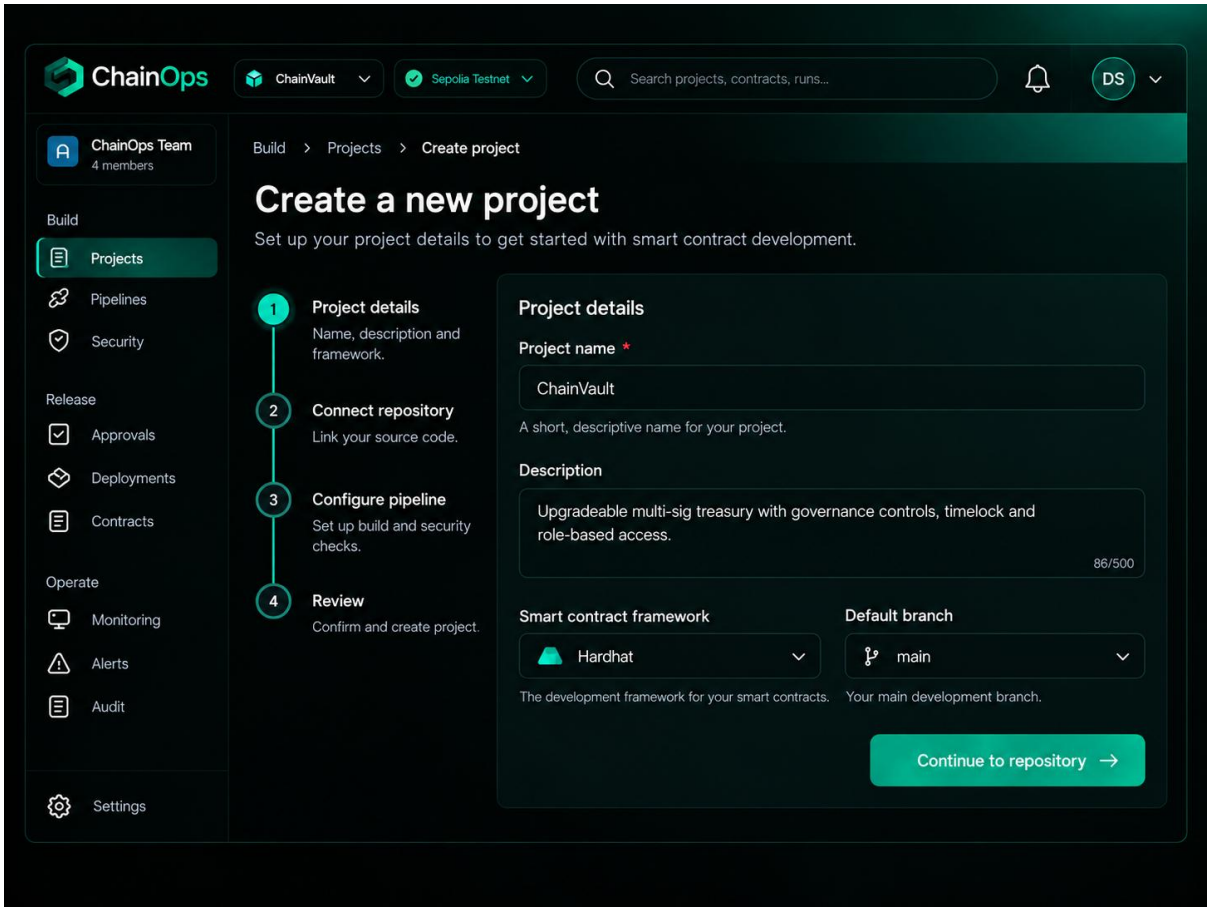


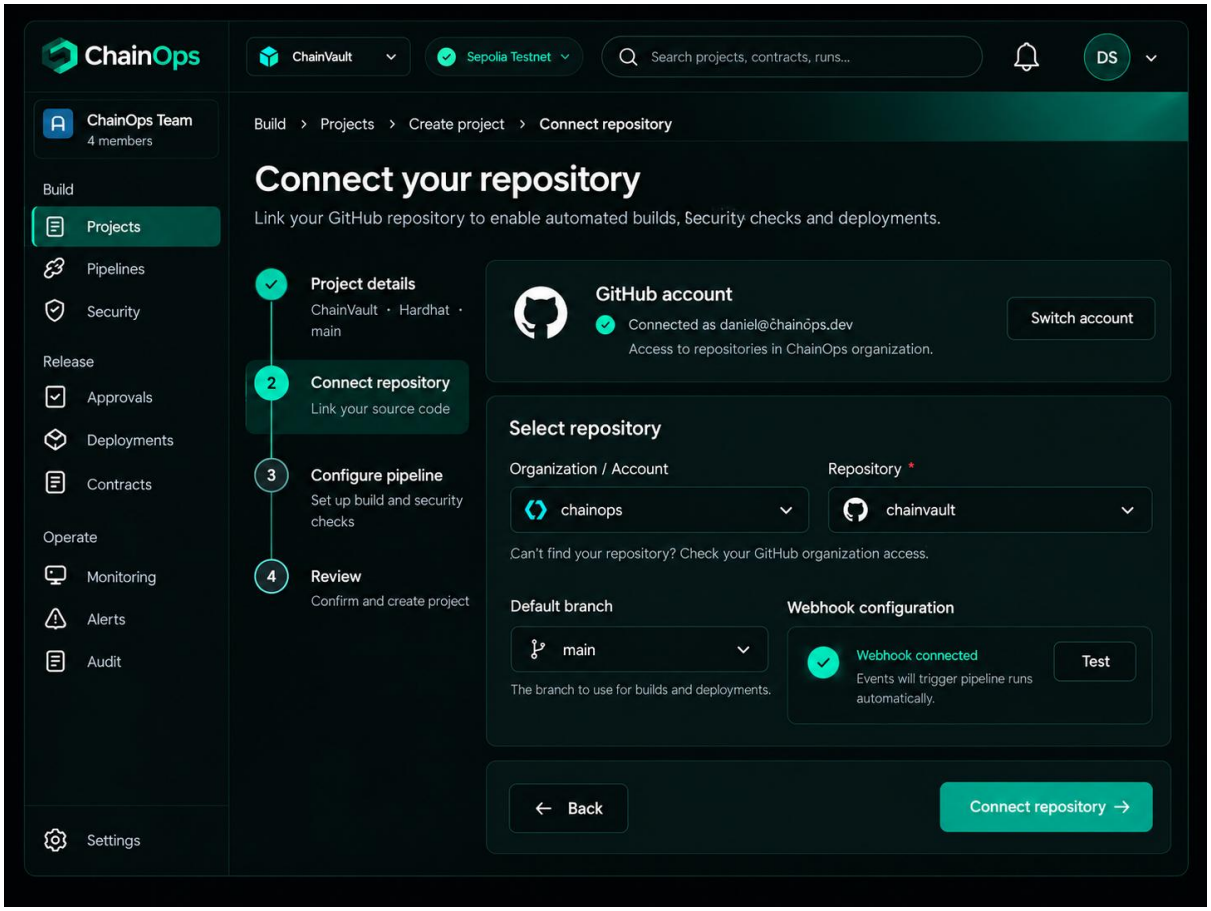

ChainOps
Product
Security
Docs
Support
Bring your team's releases together.
Keep your repositories, checks and deployments in one secure workspace.
Collaborate across teams
Automated testing and verification
Clear deployment audit trail
Create your account
Start building with ChainOps
Full name
Daniel Smith
Email address
you@intellicoders.dev
Password
Use at least 8 characters with a number and a symbol.
I agree to the Terms of Service and Privacy Policy.
Create account
Already have an account? Sign in

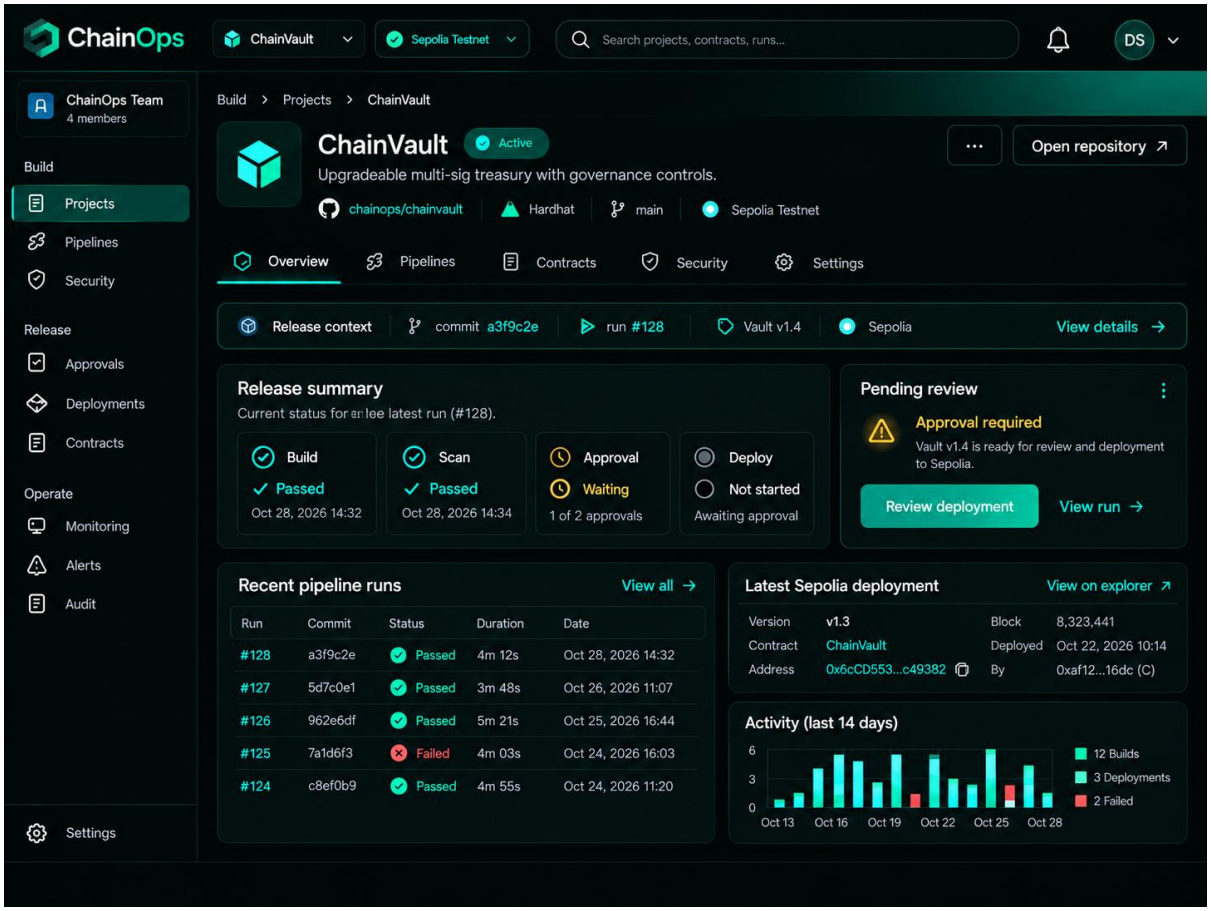

ChainOps
Product
Security
Docs
Support
Reset your password
Enter your email address and we'll send you a link to reset your password.
Email address
you@intellicoders.dev
Send reset link
Back to sign in

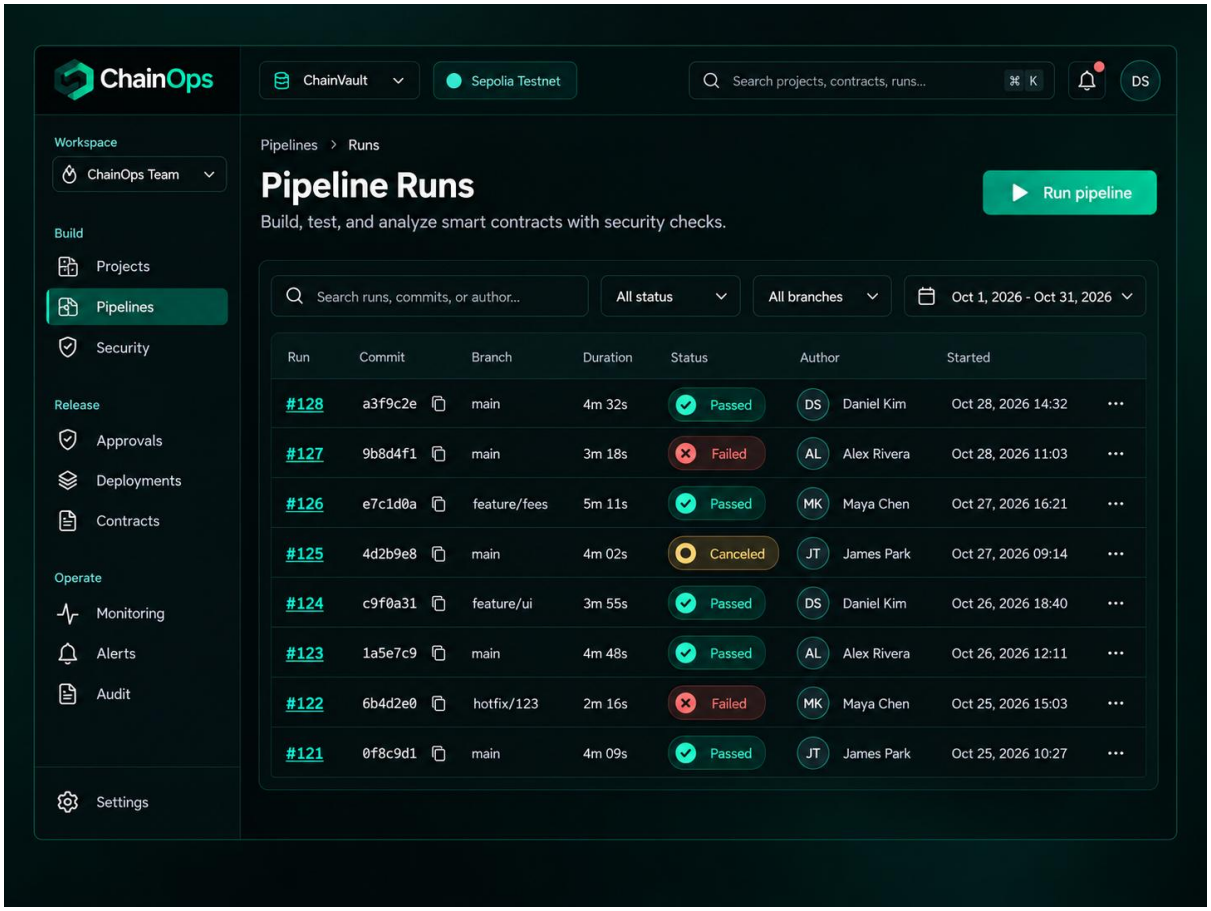

ChainOps
Escrow protocol
Search contracts, pipelines, issues...
Sepolia · Testnet
DS
Workspace
ChainOps Team
Overview
Projects
Pipelines
Security
Deployments
Contracts
Monitoring
Alerts
Audit
Settings
Set up your workspace
Get your team and first project ready in a few steps.
1
Organization
Set your organization details
2
Create project
Configure your first project
3
Choose network
Select a development network
Organization details
This will be your default workspace for teams and projects.
Organization name
ChainOps Team
This name will be visible to your team members.
Organization slug
chainops-team
Used in project URLs and on the command line.
Team size (optional)
Just me
Continue

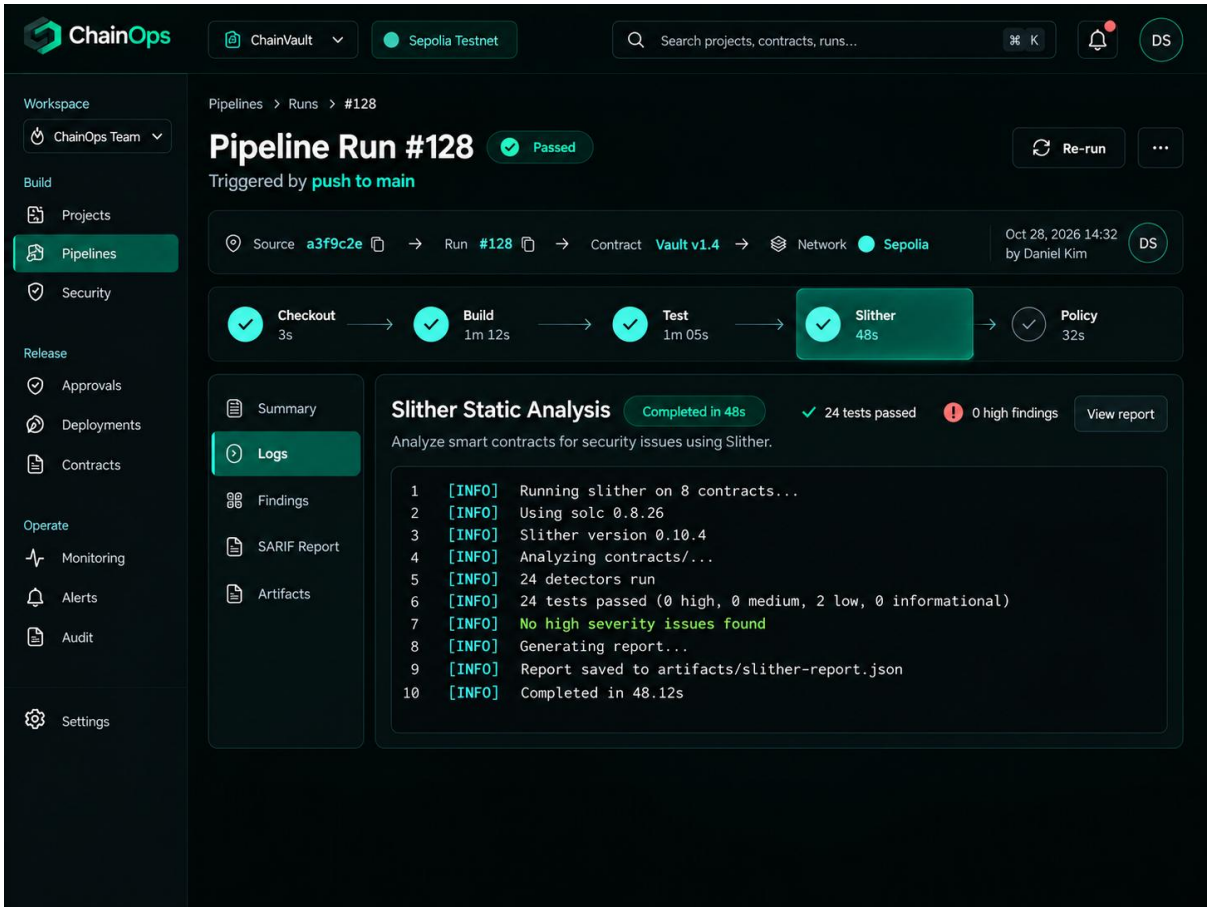

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
DS
ChainOps Team
4 members
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Build > Projects
Projects
Create and manage your smart contract projects, repositories and pipelines.
New project
Search projects...
All 3
Active 3
Paused 0
Archived 0
ChainVault
Upgradeable multi-sig treasury with governance controls.
Active
Repository
chainops/chainvault
Framework
Hardhat
Network
Sepolia
Last pipeline
#128 Build passed
Oct 26, 2026 14:32
TokenFactory
Create and manage ERC20 token deployments.
Active
Repository
chainops/token-factory
Framework
Hardhat
Network
Sepolia
Last pipeline
#87 Build passed
Oct 25, 2026 09:18
Treasury
Treasury management and timelock contracts.
Active
Repository
chainops/treasury
Framework
Hardhat
Network
Sepolia
Last pipeline
#42 Tests failed
Oct 24, 2026 16:03

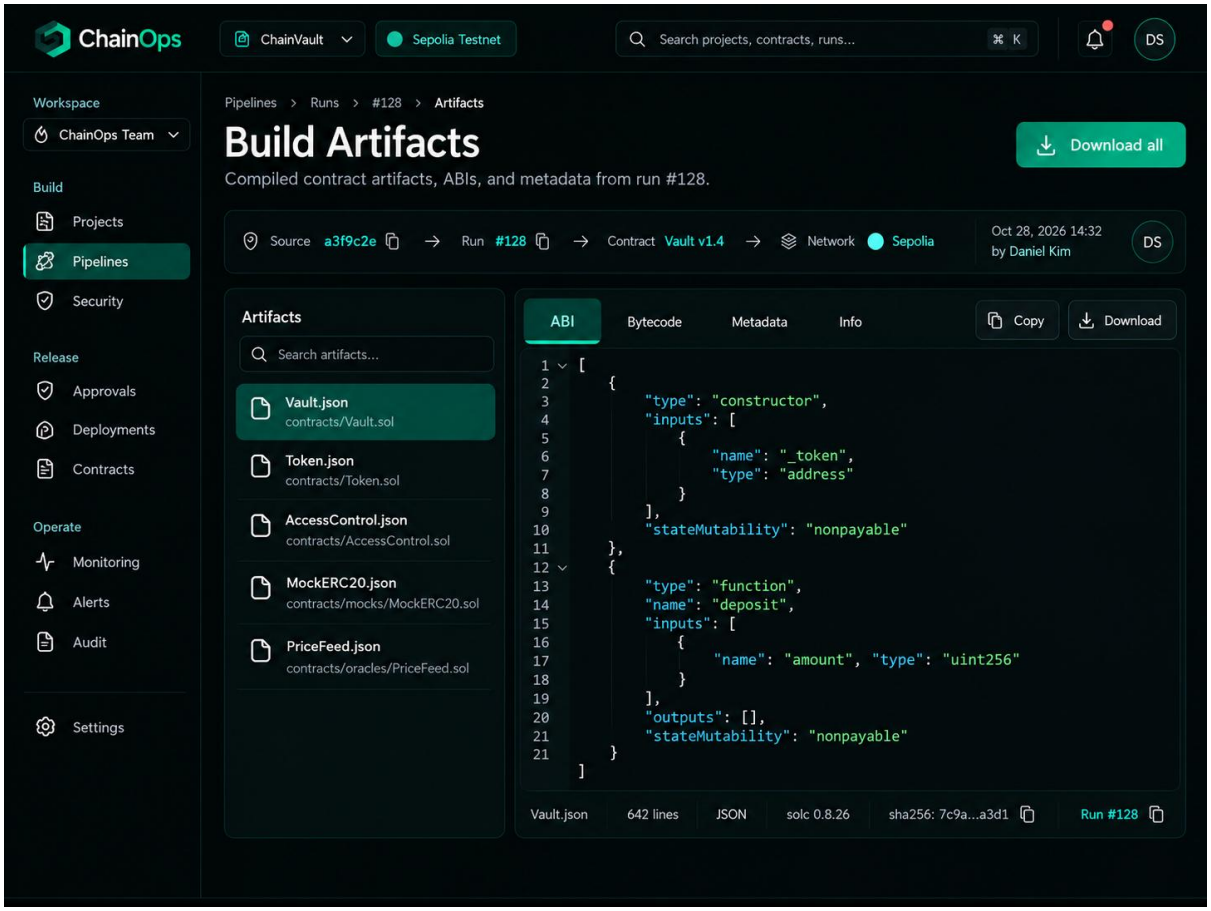

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
DS
ChainOps Team
4 members
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Build > Projects > Create project
Create a new project
Set up your project details to get started with smart contract development.
1 Project details
Name, description and framework.
2 Connect repository
Link your source code.
3 Configure pipeline
Set up build and security checks.
4 Review
Confirm and create project.
Project details
Project name *
ChainVault
A short, descriptive name for your project.
Description
Upgradeable multi-sig treasury with governance controls, timelock and role-based access.
86/500
Smart contract framework
Hardhat
The development framework for your smart contracts.
Default branch
main
Your main development branch.
Continue to repository

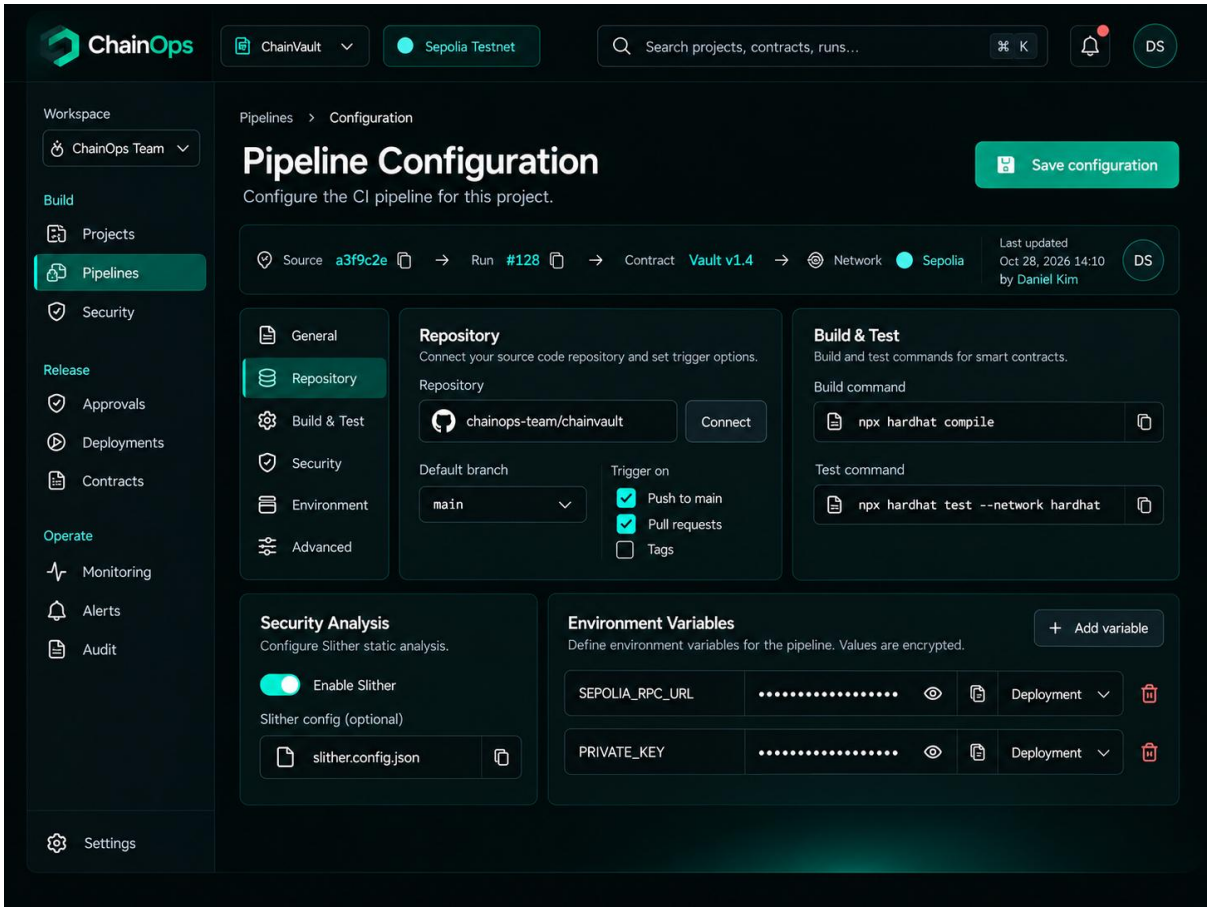

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
DS
ChainOps Team
4 members
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Build > Projects > Create project > Connect repository
Connect your repository
Link your GitHub repository to enable automated builds, Security checks and deployments.
Project details
ChainVault · Hardhat · main
2 Connect repository
Link your source code
3 Configure pipeline
Set up build and security checks
4 Review
Confirm and create project
GitHub account
Connected as daniel@chainops.dev
Access to repositories in ChainOps organization.
Switch account
Select repository
Organization / Account
chainops
Repository *
chainvault
Can't find your repository? Check your GitHub organization access.
Default branch
main
The branch to use for builds and deployments.
Webhook configuration
Webhook connected
Events will trigger pipeline runs automatically.
Test
Back
Connect repository

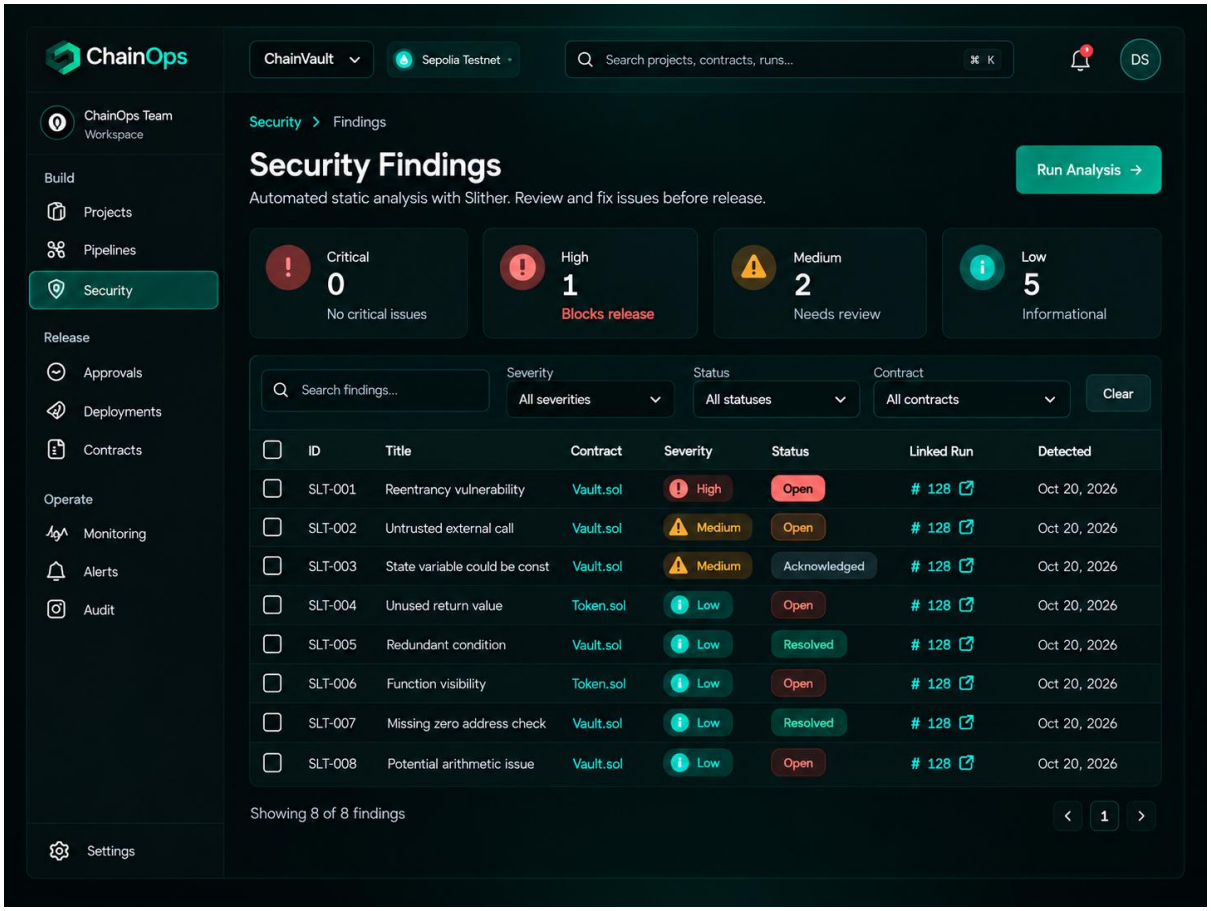

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
DS
ChainOps Team
4 members
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Build > Projects > ChainVault
ChainVault
Active
Upgradeable multi-sig treasury with governance controls.
chainops/chainvault
Hardhat
main
Sepolia Testnet
Open repository
Overview
Pipelines
Contracts
Security
Settings
Release context
commit a3f9c2e
run #128
Vault v1.4
Sepolia
View details
Release summary
Current status for the latest run (#128).
Build
Passed
Oct 28, 2026 14:32
Scan
Passed
Oct 28, 2026 14:34
Approval
Waiting
1 of 2 approvals
Deploy
Not started
Awaiting approval
Pending review
Approval required
Vault v1.4 is ready for review and deployment to Sepolia.
Review deployment
View run
Recent pipeline runs
View all
Run
Commit
Status
Duration
Date
#128
a3f9c2e
Passed
4m 12s
Oct 28, 2026 14:32
#127
5d7c0e1
Passed
3m 48s
Oct 26, 2026 11:07
#126
962e6df
Passed
5m 21s
Oct 25, 2026 16:44
#125
7a1d6f3
Failed
4m 03s
Oct 24, 2026 16:03
#124
c8ef0b9
Passed
4m 55s
Oct 24, 2026 11:20
Latest Sepolia deployment
View on explorer
Version
v1.3
Block
8,323,441
Contract
ChainVault
Deployed
Oct 22, 2026 10:14
Address
0x6cCD553...c49382
By
0xaf12...16dc (C)
Activity (last 14 days)
6
3
0
Oct 13
Oct 16
Oct 19
Oct 22
Oct 25
Oct 28
12 Builds
3 Deployments
2 Failed

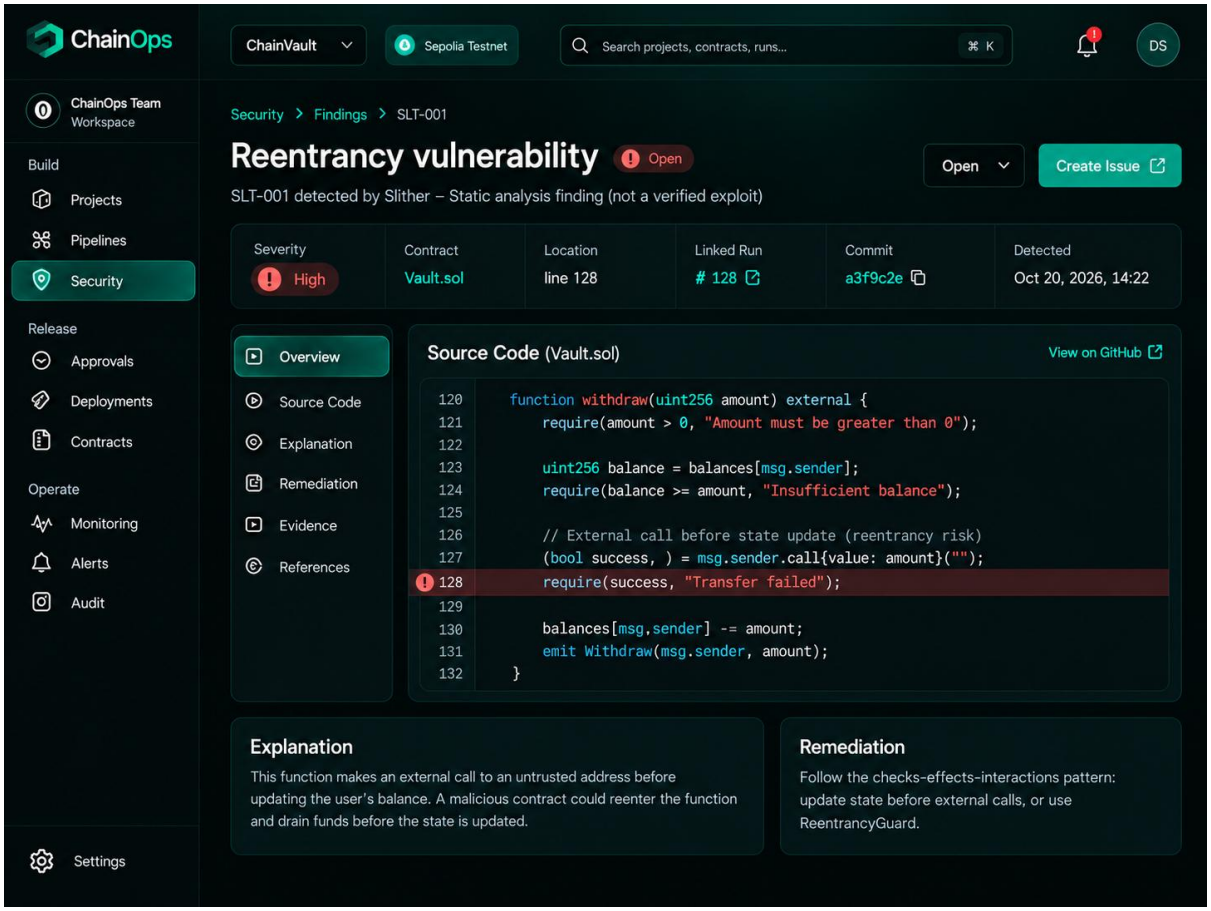

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
Workspace
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Pipelines > Runs
Pipeline Runs
Build, test, and analyze smart contracts with security checks.
Run pipeline
Search runs, commits, or author...
All status
All branches
Oct 1, 2026 - Oct 31, 2026
Run
Commit
Branch
Duration
Status
Author
Started
#128
a3f9c2e
main
4m 32s
Passed
DS
Daniel Kim
Oct 28, 2026 14:32
#127
9b8d4f1
main
3m 18s
Failed
AL
Alex Rivera
Oct 28, 2026 11:03
#126
e7c1d0a
feature/fees
5m 11s
Passed
MK
Maya Chen
Oct 27, 2026 16:21
#125
4d2b9e8
main
4m 02s
Canceled
JT
James Park
Oct 27, 2026 09:14
#124
c9f0a31
feature/ui
3m 55s
Passed
DS
Daniel Kim
Oct 26, 2026 18:40
#123
1a5e7c9
main
4m 48s
Passed
AL
Alex Rivera
Oct 26, 2026 12:11
#122
6b4d2e0
hotfix/123
2m 16s
Failed
MK
Maya Chen
Oct 25, 2026 15:03
#121
0f8c9d1
main
4m 09s
Passed
JT
James Park
Oct 25, 2026 10:27

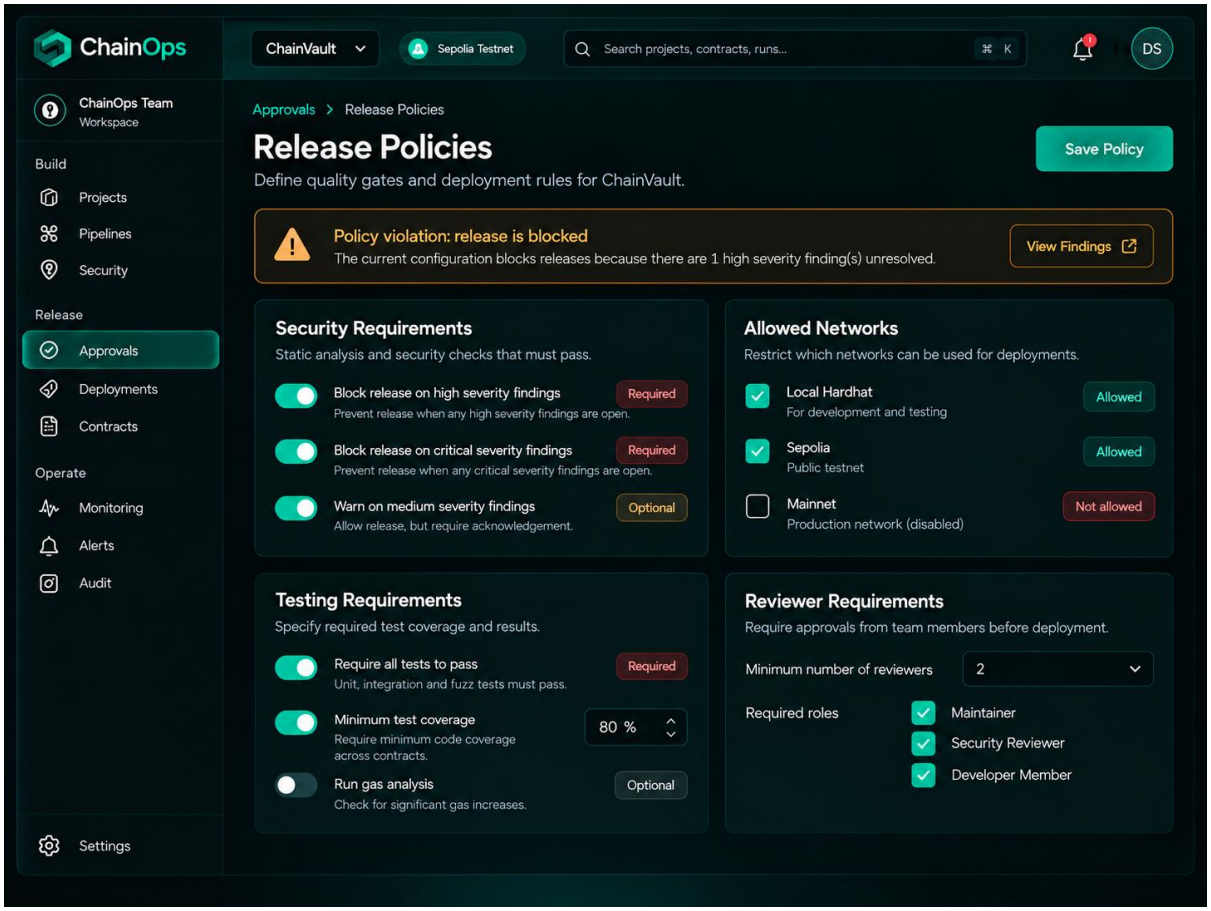

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
Workspace
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Pipelines > Runs > #128
Pipeline Run #128
Passed
Re-run
Triggered by push to main
Source a3f9c2e → Run #128 → Contract Vault v1.4 → Network Sepolia
Oct 28, 2026 14:32 by Daniel Kim
Checkout 3s
Build 1m 12s
Test 1m 05s
Slither 48s
Policy 32s
Summary
Logs
Findings
SARIF Report
Artifacts
Slither Static Analysis
Completed in 48s
24 tests passed
0 high findings
View report
Analyze smart contracts for security issues using Slither.
1 [INFO] Running slither on 8 contracts...
2 [INFO] Using solc 0.8.26
3 [INFO] Slither version 0.10.4
4 [INFO] Analyzing contracts/...
5 [INFO] 24 detectors run
6 [INFO] 24 tests passed (0 high, 0 medium, 2 low, 0 informational)
7 [INFO] No high severity issues found
8 [INFO] Generating report...
9 [INFO] Report saved to artifacts/slither-report.json
10 [INFO] Completed in 48.12s

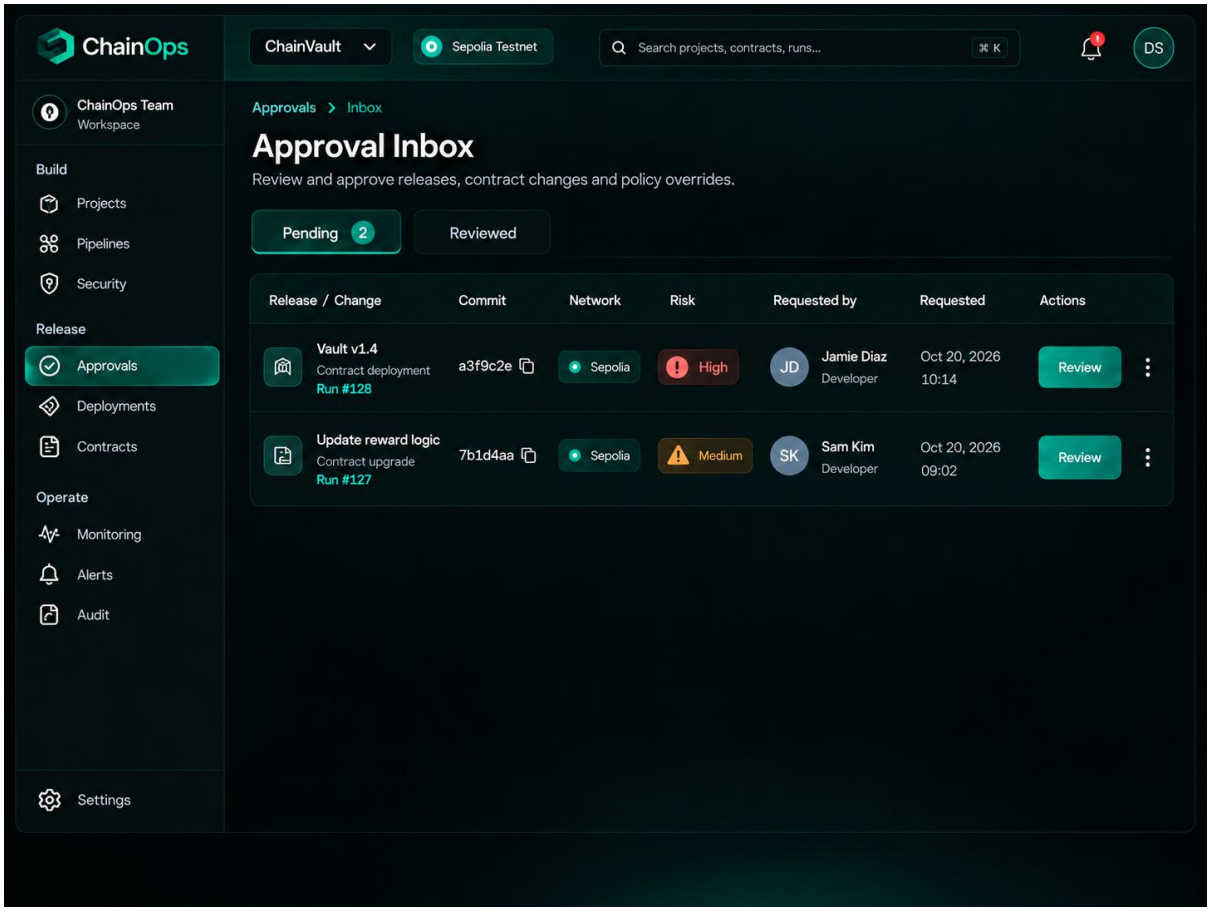

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
Workspace
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Pipelines > Runs > #128 > Artifacts
Build Artifacts
Download all
Compiled contract artifacts, ABIs, and metadata from run #128.
Source a3f9c2e → Run #128 → Contract Vault v1.4 → Network Sepolia
Oct 28, 2026 14:32 by Daniel Kim
Artifacts
Search artifacts...
Vault.json contracts/Vault.sol
Token.json contracts/Token.sol
AccessControl.json contracts/AccessControl.sol
MockERC20.json contracts/mocks/MockERC20.sol
PriceFeed.json contracts/oracles/PriceFeed.sol
ABI
Bytecode
Metadata
Info
Copy
Download
[
{
"type": "constructor",
"inputs": [
{
"name": "_token",
"type": "address"
}
],
"stateMutability": "nonpayable"
},
{
"type": "function",
"name": "deposit",
"inputs": [
{
"name": "amount", "type": "uint256"
}
],
"outputs": [],
"stateMutability": "nonpayable"
}
]
Vault.json 642 lines JSON solc 0.8.26 sha256: 7c9a...a3d1 Run #128

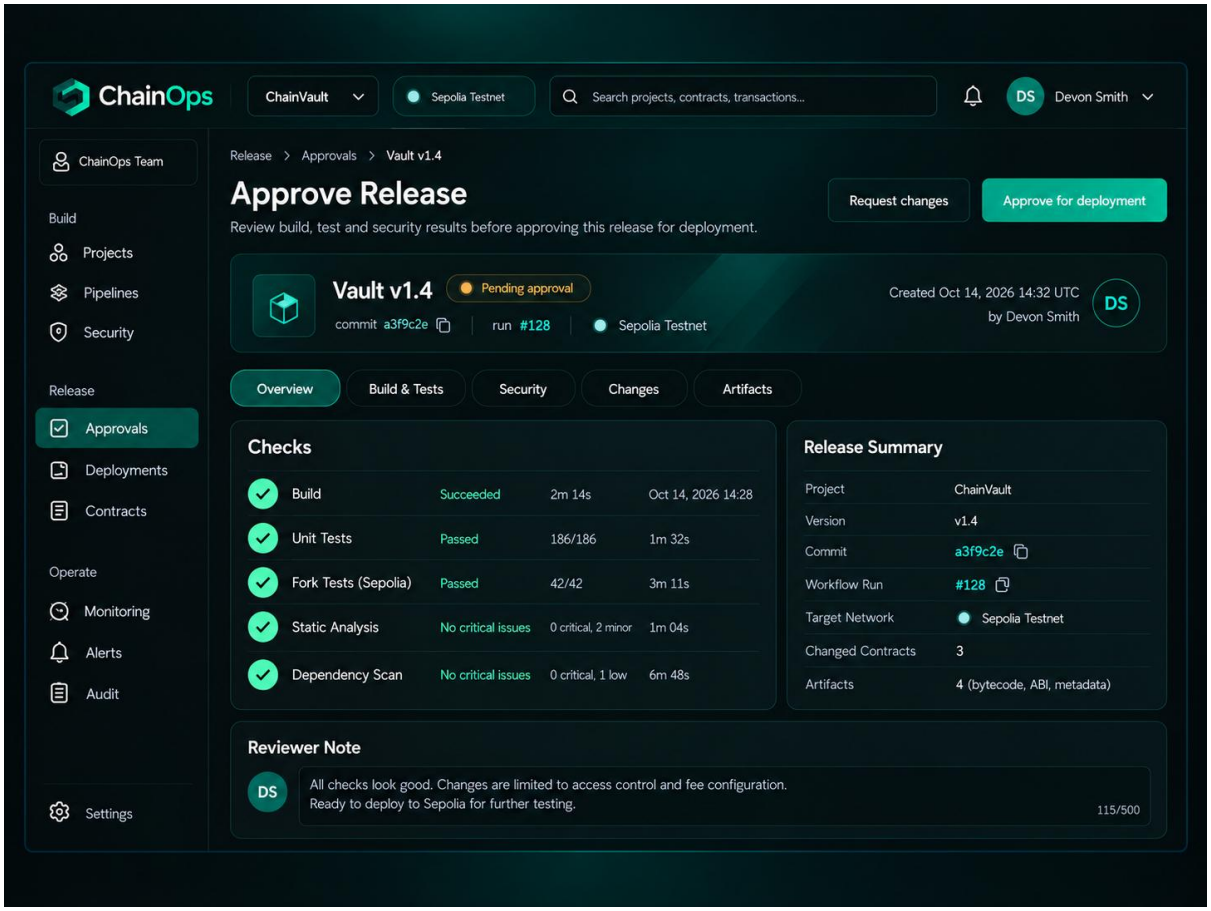

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
Workspace
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Pipelines > Configuration
Pipeline Configuration
Configure the CI pipeline for this project.
Save configuration
Source a3f9c2e → Run #128 → Contract Vault v1.4 → Network Sepolia
Last updated Oct 28, 2026 14:10 by Daniel Kim
General
Repository
Build & Test
Security
Environment
Advanced
Repository
Connect your source code repository and set trigger options.
Repository
chainops-team/chainvault
Connect
Default branch
main
Trigger on
Push to main
Pull requests
Tags
Build & Test
Build and test commands for smart contracts.
Build command
npx hardhat compile
Test command
npx hardhat test --network hardhat
Security Analysis
Configure Slither static analysis.
Enable Slither
Slither config (optional)
slither.config.json
Environment Variables
Define environment variables for the pipeline. Values are encrypted.
Add variable
SEPOLIA_RPC_URL
PRIVATE_KEY
Deployment

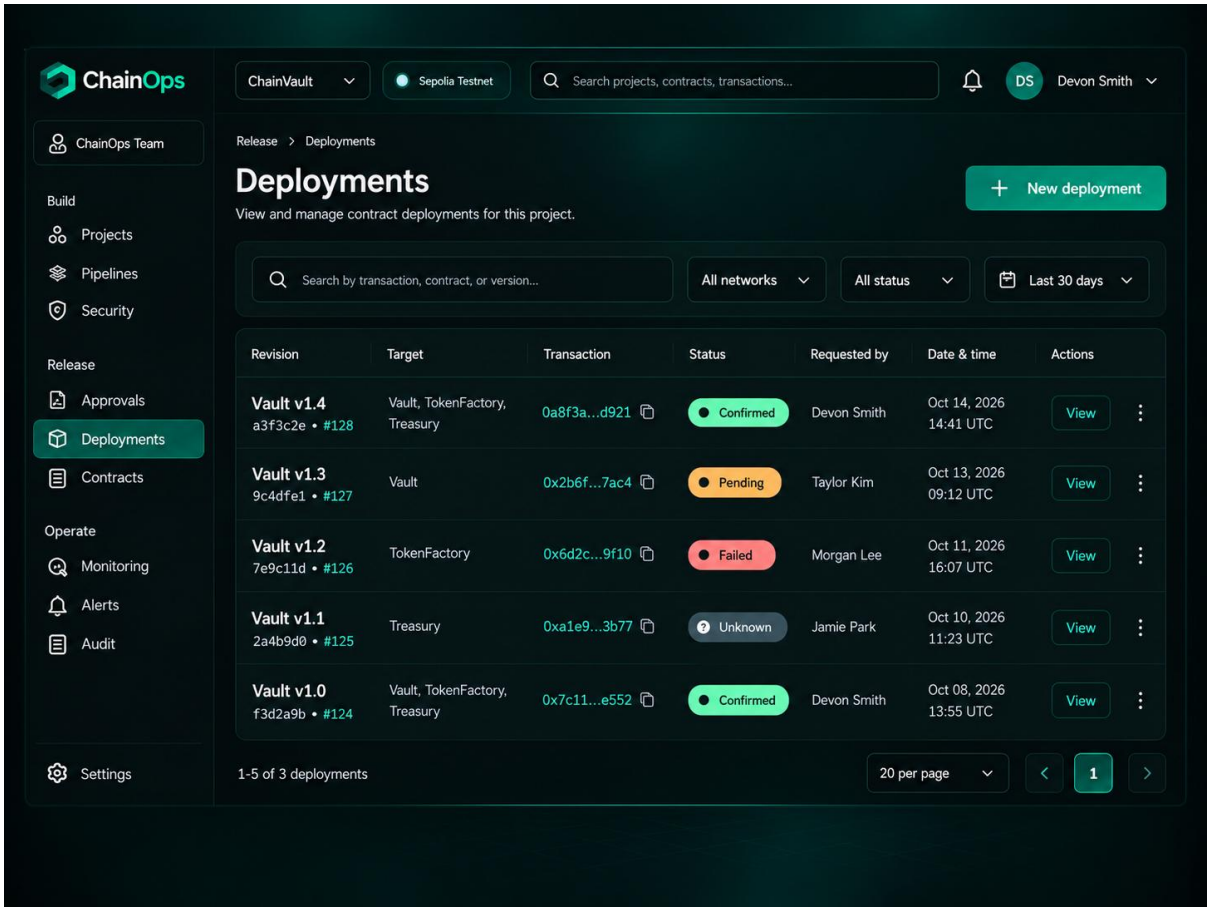

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
ChainOps Team
Workspace
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Security > Findings
Security Findings
Automated static analysis with Slither. Review and fix issues before release.
Run Analysis
Critical 0 No critical issues
High 1 Blocks release
Medium 2 Needs review
Low 5 Informational
Search findings...
Severity
All severities
Status
All statuses
Contract
All contracts
Clear
ID Title Contract Severity Status Linked Run Detected
SLT-001 Reentrancy vulnerability Vault.sol High Open # 128 Oct 20, 2026
SLT-002 Untrusted external call Vault.sol Medium Open # 128 Oct 20, 2026
SLT-003 State variable could be const Vault.sol Medium Acknowledged # 128 Oct 20, 2026
SLT-004 Unused return value Token.sol Low Open # 128 Oct 20, 2026
SLT-005 Redundant condition Vault.sol Low Resolved # 128 Oct 20, 2026
SLT-006 Function visibility Token.sol Low Open # 128 Oct 20, 2026
SLT-007 Missing zero address check Vault.sol Low Resolved # 128 Oct 20, 2026
SLT-008 Potential arithmetic issue Vault.sol Low Open # 128 Oct 20, 2026
Showing 8 of 8 findings
1

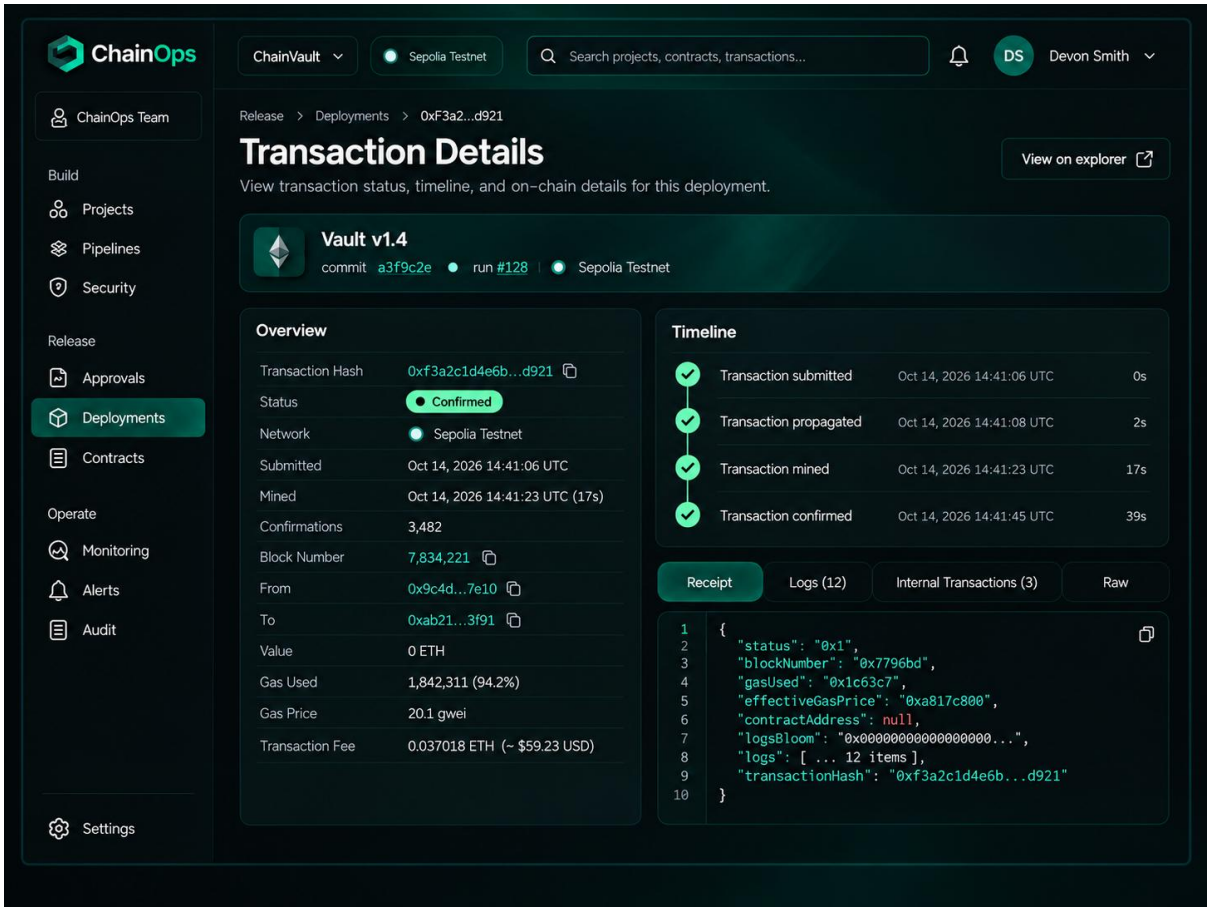

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
ChainOps Team
Workspace
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Security > Findings > SLT-001
Reentrancy vulnerability
Open
SLT-001 detected by Slither – Static analysis finding (not a verified exploit)
Open
Create Issue
Severity
High
Contract
Vault.sol
Location
line 128
Linked Run
128
Commit
a3f9c2e
Detected
Oct 20, 2026, 14:22
Overview
Source Code
Explanation
Remediation
Evidence
References
Source Code (Vault.sol)
View on GitHub
120 function withdraw(uint256 amount) external {
121 require(amount > 0, "Amount must be greater than 0");
122
123 uint256 balance = balances[msg.sender];
124 require(balance >= amount, "Insufficient balance");
125
126 // External call before state update (reentrancy risk)
127 (bool success,) = msg.sender.call{value: amount}("");
128 require(success, "Transfer failed");
129
130 balances[msg.sender] -= amount;
131 emit Withdraw(msg.sender, amount);
132 }
Explanation
This function makes an external call to an untrusted address before updating the user's balance. A malicious contract could reenter the function and drain funds before the state is updated.
Remediation
Follow the checks-effects-interactions pattern: update state before external calls, or use ReentrancyGuard.

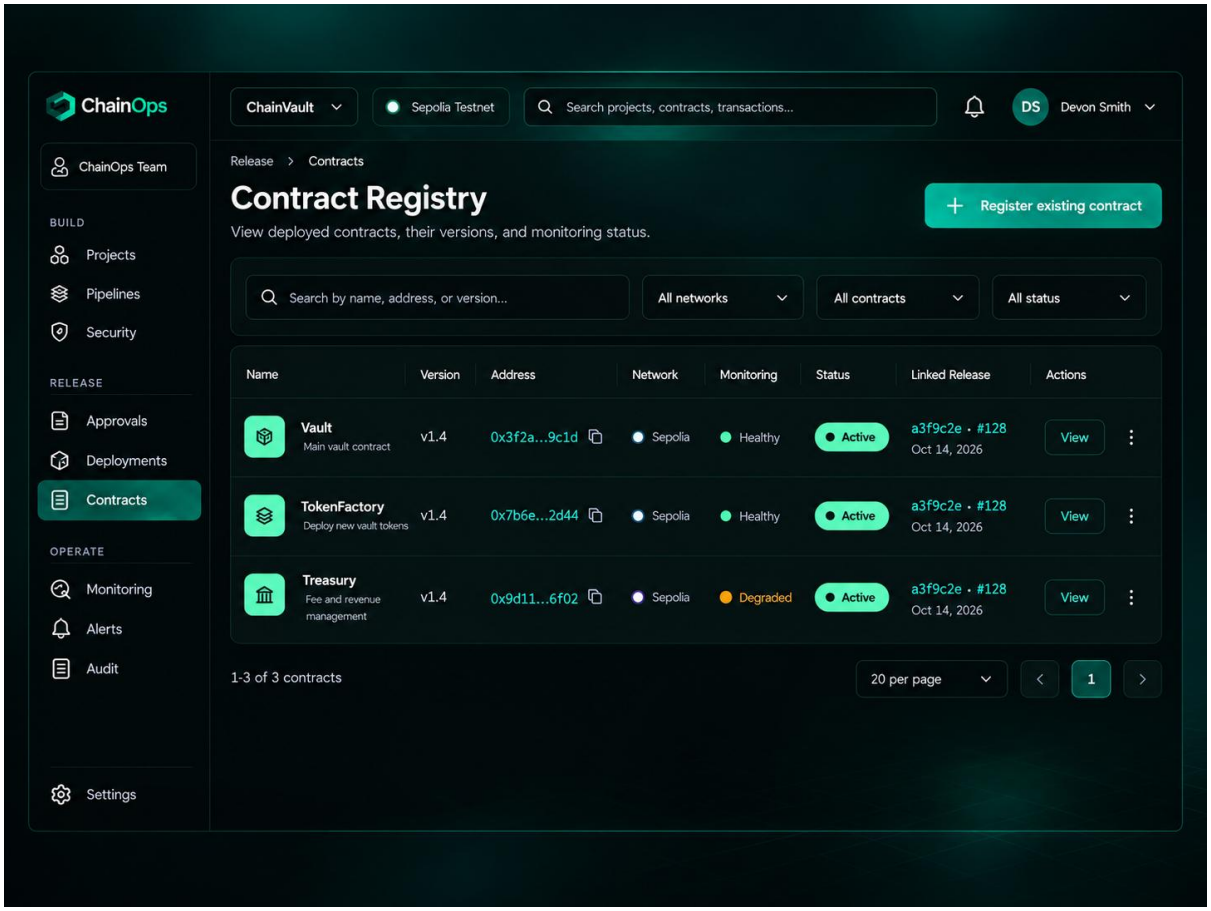

ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
ChainOps Team
Workspace
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Approvals > Release Policies
Release Policies
Define quality gates and deployment rules for ChainVault.
Save Policy
Policy violation: release is blocked
The current configuration blocks releases because there are 1 high severity finding(s) unresolved.
View Findings
Security Requirements
Static analysis and security checks that must pass.
Block release on high severity findings
Required
Prevent release when any high severity findings are open.
Block release on critical severity findings
Required
Prevent release when any critical severity findings are open.
Warn on medium severity findings
Optional
Allow release, but require acknowledgement.
Allowed Networks
Restrict which networks can be used for deployments.
Local Hardhat
For development and testing
Allowed
Sepolia
Public testnet
Allowed
Mainnet
Production network (disabled)
Not allowed
Testing Requirements
Specify required test coverage and results.
Require all tests to pass
Required
Unit, integration and fuzz tests must pass.
Minimum test coverage
80 %
Require minimum code coverage across contracts.
Run gas analysis
Optional
Check for significant gas increases.
Reviewer Requirements
Require approvals from team members before deployment.
Minimum number of reviewers
2
Required roles
Maintainer
Security Reviewer
Developer Member


ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, runs...
⌘ K
DS
ChainOps Team
Workspace
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Approvals > Inbox
Approval Inbox
Review and approve releases, contract changes and policy overrides.
Pending 2
Reviewed
Release / Change
Commit
Network
Risk
Requested by
Requested
Actions
Vault v1.4
Contract deployment
Run #128
a3f9c2e
Sepolia
High
JD
Jamie Diaz
Developer
Oct 20, 2026
10:14
Review
Update reward logic
Contract upgrade
Run #127
7b1d4aa
Sepolia
Medium
SK
Sam Kim
Developer
Oct 20, 2026
09:02
Review


ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, transactions...
DS
Devon Smith
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Release > Approvals > Vault v1.4
Approve Release
Review build, test and security results before approving this release for deployment.
Request changes
Approve for deployment
Vault v1.4
Pending approval
commit a3f9c2e
run #128
Sepolia Testnet
Created Oct 14, 2026 14:32 UTC
by Devon Smith
DS
Overview
Build & Tests
Security
Changes
Artifacts
Checks
Build
Succeeded
2m 14s
Oct 14, 2026 14:28
Unit Tests
Passed
186/186
1m 32s
Fork Tests (Sepolia)
Passed
42/42
3m 11s
Static Analysis
No critical issues
0 critical, 2 minor
1m 04s
Dependency Scan
No critical issues
0 critical, 1 low
6m 48s
Release Summary
Project
ChainVault
Version
v1.4
Commit
a3f9c2e
Workflow Run
#128
Target Network
Sepolia Testnet
Changed Contracts
3
Artifacts
4 (bytecode, ABI, metadata)
Reviewer Note
DS
All checks look good. Changes are limited to access control and fee configuration.
Ready to deploy to Sepolia for further testing.
115/500


ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, transactions...
DS
Devon Smith
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Release > Deployments
Deployments
View and manage contract deployments for this project.
New deployment
Search by transaction, contract, or version...
All networks
All status
Last 30 days
Revision
Target
Transaction
Status
Requested by
Date & time
Actions
Vault v1.4
a3f3c2e • #128
Vault, TokenFactory, Treasury
0a8f3a...d921
Confirmed
Devon Smith
Oct 14, 2026 14:41 UTC
View
Vault v1.3
9c4dfe1 • #127
Vault
0x2b6f...7ac4
Pending
Taylor Kim
Oct 13, 2026 09:12 UTC
View
Vault v1.2
7e9c11d • #126
TokenFactory
0x6d2c...9f10
Failed
Morgan Lee
Oct 11, 2026 16:07 UTC
View
Vault v1.1
2a4b9d0 • #125
Treasury
0xa1e9...3b77
Unknown
Jamie Park
Oct 10, 2026 11:23 UTC
View
Vault v1.0
f3d2a9b • #124
Vault, TokenFactory, Treasury
0x7c11...e552
Confirmed
Devon Smith
Oct 08, 2026 13:55 UTC
View
1-5 of 3 deployments
20 per page
1


ChainOps
ChainVault
Sepolia Testnet
Search projects, contracts, transactions...
DS
Devon Smith
ChainOps Team
Build
Projects
Pipelines
Security
Release
Approvals
Deployments
Contracts
Operate
Monitoring
Alerts
Audit
Settings
Release > Deployments > 0xF3a2...d921
Transaction Details
View transaction status, timeline, and on-chain details for this deployment.
View on explorer
Vault v1.4
commit a3f9c2e • run #128 • Sepolia Testnet
Overview
Transaction Hash 0xf3a2c1d4e6b...d921
Status Confirmed
Network Sepolia Testnet
Submitted Oct 14, 2026 14:41:06 UTC
Mined Oct 14, 2026 14:41:23 UTC (17s)
Confirmations 3,482
Block Number 7,834,221
From 0x9c4d...7e10
To 0xab21...3f91
Value 0 ETH
Gas Used 1,842,311 (94.2%)
Gas Price 20.1 gwei
Transaction Fee 0.037018 ETH (~ $59.23 USD)
Timeline
Transaction submitted Oct 14, 2026 14:41:06 UTC 0s
Transaction propagated Oct 14, 2026 14:41:08 UTC 2s
Transaction mined Oct 14, 2026 14:41:23 UTC 17s
Transaction confirmed Oct 14, 2026 14:41:45 UTC 39s
Receipt
Logs (12)
Internal Transactions (3)
Raw
{
"status": "0x1",
"blockNumber": "0x7796bd",
"gasUsed": "0x1c63c7",
"effectiveGasPrice": "0xa817c800",
"contractAddress": null,
"logsBloom": "0x00000000000000000...",
"logs": [... 12 items],
"transactionHash": "0xf3a2c1d4e6b...d921"
}



Chapter 6 - Implementation

6.1 Introduction

This chapter presents the implementation approach and development progress of the ChainOps platform during the interim phase. The implementation focuses on transforming the proposed system design into functional components using the selected technologies and development tools. The platform follows a modular architecture consisting of Pipeline and Execution, Blockchain Deployment and Contract Registry, Monitoring and Alerts, Security and Governance, and Platform Access and AI Assistance. Technologies such as SvelteKit or Next.js, Node.js, PostgreSQL, and established blockchain development tools are considered for implementing the proposed functionalities. This chapter describes the implementation activities, technical configurations, and integration approaches undertaken during the interim period, providing a foundation for further development and testing.

6.2 Current Implementation Status

At the interim stage, the development of ChainOps is primarily focused on completing the system design and preparing the foundation for implementation. The main activities include analyzing system requirements, developing UML diagrams, designing database relationships, and creating user interface prototypes.

The system has been divided into five major functional modules: Pipeline and Execution, Blockchain Deployment and Contract Registry, Monitoring and Alerts, Security and Governance, and Platform Access and AI Assistance. These modules have been analyzed to identify their responsibilities, interactions, and dependencies.

The completed design activities provide a structured understanding of how the proposed platform should operate. Although the functional implementation of the modules has not yet been completed, the system design and user interface prototypes establish a foundation for the upcoming development phase.

6.3 User Interface and User Experience Design

The user interface and user experience (UI/UX) design is an important part of the ChainOps development process. Since the platform involves complex smart contract development activities, the interface must present technical information in an organized, understandable, and accessible manner.

The UI/UX design focuses on providing a centralized environment where developers and other authorized users can interact with the major functionalities of the platform. The designs emphasize intuitive navigation, consistent visual components, organized information presentation, and efficient access to operational records.

The proposed interface is organized around the following functional areas.

6.3.1 Authentication and Workspace Management

The authentication and workspace management interface is designed to provide users with access to the ChainOps platform and its project management functionalities.

The proposed screens support user registration, login, workspace selection, project management, and related account operations. The interface is designed to distinguish functionalities according to user roles and permissions.

These designs establish how users will navigate between their authorized workspaces, projects, and deployment environments.

6.3.2 Global Dashboard

The global dashboard is designed to provide users with a centralized overview of relevant activities within ChainOps.

The proposed dashboard presents information related to pipeline execution, security findings, deployment activities, and monitoring alerts. Information is intended to be displayed through organized interface components, allowing users to identify important operational updates and navigate to detailed records.

The dashboard design emphasizes clear information organization and easy navigation to reduce the complexity of managing multiple development activities.

6.3.3 Pipeline and Security Management Interfaces

The pipeline management interface is designed to allow developers to view automated pipeline execution information, including compilation results, testing outcomes, execution statuses, and related logs.

The security management interface is intended to display identified security findings, validation results, and release policy decisions. These interfaces provide a visual representation of the development and validation processes.

By presenting the results of different development activities through a centralized interface, ChainOps aims to improve operational visibility and simplify the interpretation of technical information.

6.3.4 Deployment and Monitoring Interfaces

The deployment management interface is designed to present smart contract deployment requests, approval information, transaction statuses, and registered contract details.

The monitoring interface focuses on displaying selected blockchain events, transaction activities, and generated alerts associated with deployed smart contracts.

These interfaces are intended to provide users with an organized view of deployment histories and post-deployment activities.

6.3.5 Notification and AI Assistant Interfaces

The notification interface is designed to present important updates related to pipeline executions, deployment decisions, security findings, and monitoring alerts.

Additionally, the proposed AI Assistant interface allows users to submit natural language questions regarding operational information. The design focuses on presenting AI-generated explanations in a clear and understandable format.

The AI Assistant is intended to operate with restricted access to authorized records, ensuring that sensitive operations remain under the control of authorized users.

6.4 Preparation for System Implementation

The system design provides a foundation for implementing ChainOps using the selected technology stack. The proposed architecture separates the frontend, backend, database, background processing, and blockchain integration components to support modular development.

The frontend is planned to be developed using SvelteKit with TypeScript, while NestJS with TypeScript will be used to implement backend API services. Neon PostgreSQL with Prisma ORM will support structured data management, including user accounts, projects, pipeline records, security findings, and deployment information.

Redis and BullMQ are selected to manage asynchronous pipeline execution, while Docker will provide isolated environments for processing smart contract projects. Hardhat and Slither will be integrated for compilation, testing, and security analysis, respectively. Blockchain communication and deployment operations will be supported through viem.

The completed UML diagrams and database design serve as references for translating the identified requirements into application components, backend services, and database models. These design artifacts will guide implementation and help maintain consistency across the five functional modules.

6.5 Planned Implementation and Integration

The upcoming development phase will focus on converting the completed system designs into functional software components. Development responsibilities are distributed among the five major modules, allowing team members to work on their assigned areas while coordinating shared functionalities.

The initial development activities will focus on establishing the frontend and backend structure, configuring database connectivity, and implementing authentication, workspace management, and role-based access control.

Subsequent activities will involve integrating GitHub repositories and webhooks, implementing background pipeline execution, and connecting smart contract compilation, testing, and security analysis tools.

The deployment and governance components will then be developed to support deployment requests, approval workflows, blockchain transactions, and contract registration. Monitoring, notifications, and AI-assisted operational guidance will be integrated to complete the proposed lifecycle management process.

Following module implementation, integration testing will be conducted to verify communication between components and ensure that the complete workflow operates according to the defined requirements.

6.6 Current Limitations and Next Steps

At the interim stage, ChainOps remains in the design and preparation phase. Although the UML diagrams and UI/UX designs have been completed, the proposed application functionalities have not yet undergone full software implementation or functional testing.

The next phase will focus on developing the frontend interfaces, backend services, database models, and integrations required to support the proposed system. Particular attention will be given to secure communication between modules, reliable pipeline execution, proper access control, and accurate lifecycle tracking.

The completed design artifacts will be used throughout development to guide implementation decisions and maintain consistency between the planned architecture and the final system.

6.7 Summary

This chapter outlined the implementation approach and development activities associated with the ChainOps platform. The implementation is organized around five interconnected modules, with an emphasis on modular development, secure integration, and efficient workflow management. The technical approaches discussed provide a basis for completing the remaining functionalities, integrating the individual modules, and conducting comprehensive system testing in the subsequent development phases.

Chapter 7 - Discussion

7.1 Introduction

The ChainOps project was initiated to address the fragmentation of tools and processes involved in smart contract development and operations. The proposed platform aims to integrate compilation, testing, security analysis, deployment, and monitoring into a centralized Smart Contract DevSecOps environment. At the interim stage, the project focuses on establishing the system requirements, modular architecture, functional workflows, and design foundations needed for implementation. This chapter discusses the preliminary

outcomes, compares the proposed solution with existing platforms, identifies the remaining development activities, and outlines the expected contributions of ChainOps.

7.2 Evaluation and Preliminary Results

At the interim stage, the evaluation primarily focuses on requirement analysis, system design, and the feasibility of integrating the proposed technologies. Comprehensive functional and integration testing will be conducted as the implementation progresses.

- **Requirement Analysis:** The major functional and non-functional requirements have been identified based on the smart contract development lifecycle. These include automated pipeline execution, security validation, deployment governance, blockchain monitoring, access control, and AI-assisted operational guidance.
- **System Design Evaluation:** The system has been divided into five interconnected modules, allowing responsibilities to be distributed among team members. UML diagrams and database models are being developed to represent user interactions, system workflows, and data relationships.
- **Technology Feasibility:** Technologies such as GitHub webhooks, Hardhat, Slither, PostgreSQL, and EVM-compatible blockchain networks have been identified as suitable components for the proposed architecture. Their integration and performance will require practical validation during implementation.
- **User Interface Design:** The proposed interface focuses on providing developers with centralized access to project management, pipeline statuses, security findings, deployment information, and monitoring activities. UI/UX prototypes will support usability evaluation and further design improvements.

These preliminary activities provide a foundation for implementation. Actual system performance, reliability, and functional correctness will be evaluated through testing once the relevant components are developed.

7.3 Comparison with Similar Works

As discussed in the literature review, existing platforms such as Tenderly, thirdweb, OpenZeppelin, and GitHub Actions offer valuable capabilities for blockchain development and operations. However, their primary functionalities focus on different aspects of the development lifecycle.

- **Integration:** While GitHub Actions supports automated development workflows and thirdweb provides smart contract deployment capabilities, ChainOps aims to connect compilation, testing, security validation, approval management, deployment, and monitoring within a unified workflow.
- **Security and Governance:** ChainOps incorporates security analysis, policy-based release validation, and role-based deployment approvals to establish a structured

governance process. These features aim to improve accountability and reduce the risk of unauthorized deployments.

- **Lifecycle Traceability:** ChainOps emphasizes maintaining relationships between GitHub commits, pipeline results, security findings, approval decisions, deployment transactions, and deployed contracts through centralized records.
- **AI Assistance:** The proposed AI Assistant provides contextual explanations of authorized operational information, including pipeline failures, security findings, deployment statuses, and monitoring alerts, without independently performing privileged operations.

The primary contribution of ChainOps is therefore its proposed integration of development automation, security governance, deployment management, and operational monitoring into a traceable smart contract lifecycle.

7.4 Further Work

The remaining development phases will focus on implementing, integrating, and testing the five major modules of ChainOps.

- **Platform and Access Management:** Implement authentication, workspace and project management, role-based permissions, notification services, and the restricted AI Assistant.
- **Pipeline and Execution:** Integrate GitHub repositories and webhooks, implement background workers, and automate smart contract compilation and testing.
- **Security and Governance:** Integrate security analysis tools, implement policy-based release validation, and develop deployment approval workflows.
- **Deployment and Contract Registry:** Implement smart contract deployment to configured EVM-compatible networks, transaction status tracking, and versioned contract records.
- **Monitoring and Alerts:** Develop blockchain event monitoring, event decoding, configurable alert rules, and monitoring dashboards.
- **System Integration and Testing:** Integrate all modules and conduct functional, security, integration, and usability testing to evaluate overall system reliability and performance.

Further improvements will focus on refining the user interface, optimizing system performance, and ensuring secure communication between components.

7.5 Conclusion

The interim phase of the ChainOps project establishes the foundation for developing a centralized Smart Contract DevSecOps and Operations Platform. The requirement analysis and system design activities define the major functionalities, module responsibilities, and interactions required to support the smart contract lifecycle. UML modeling, database design,

and user interface planning provide a structured basis for subsequent development and integration.

The proposed architecture emphasizes automation, security, traceability, and centralized operational visibility while utilizing established blockchain development tools. The remaining phases will focus on implementing the identified components, validating their integration, and evaluating the system against its objectives. ChainOps aims to deliver a secure, efficient, and user-friendly environment for managing smart contract development and post-deployment operations.

Chapter 8 - References

[1] BullMQ. (n.d.). *BullMQ documentation.*

https://docs.bullmq.io/

[2] Cloudflare. (n.d.). *Cloudflare R2 documentation.*

https://developers.cloudflare.com/r2/

[3] Docker, Inc. (n.d.). *Docker documentation.*

https://docs.docker.com/

[4] Docker, Inc. (n.d.). *Docker Compose documentation.*

https://docs.docker.com/compose/

[5] Ethereum.org. (n.d.). *Introduction to smart contracts.*

https://ethereum.org/developers/docs/smart-contracts/

[6] GitHub, Inc. (n.d.). *GitHub Actions documentation.*

https://docs.github.com/en/actions

[7] GitHub, Inc. (n.d.). *Webhooks documentation.*

https://docs.github.com/en/webhooks

[8] Microsoft. (n.d.). *TypeScript documentation.*

https://www.typescriptlang.org/docs/

[9] Neon. (n.d.). *Neon documentation.*

https://neon.com/docs/introduction

[10] Neon. (n.d.). *Neon Object Storage.*

https://neon.com/docs/storage/overview

[11] NestJS. (n.d.). *NestJS documentation.*

https://docs.nestjs.com/

[12] Nomic Foundation. (n.d.). *Getting started with Hardhat 3.*

https://hardhat.org/docs/getting-started

[13] OpenZeppelin. (n.d.). *OpenZeppelin documentation.*

https://docs.openzeppelin.com/

[14] PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation.*

https://www.postgresql.org/docs/

[15] Prisma. (n.d.). *Prisma documentation.*

https://www.prisma.io/docs

[16] Redis. (n.d.). *Redis documentation.*

https://redis.io/docs/latest/

[17] Solidity. (n.d.). *Solidity documentation.*

https://docs.soliditylang.org/en/latest/

[18] Svelte. (n.d.). *SvelteKit documentation: Introduction.*

https://svelte.dev/docs/kit/introduction

[19] Tenderly. (n.d.). *Tenderly documentation.*

https://docs.tenderly.co/introduction

[20] thirdweb. (n.d.). *thirdweb documentation.*

https://portal.thirdweb.com/

[21] Trail of Bits. (n.d.). *Slither: Static analyzer for Solidity and Vyper* [Software repository]. GitHub.

https://github.com/crytic/slither

[22] Vercel. (n.d.). *SvelteKit on Vercel.*

https://vercel.com/docs/frameworks/full-stack/sveltekit

[23] viem. (n.d.). *viem: TypeScript interface for Ethereum.*

https://viem.sh/

Appendix A - Individual Contribution

ChainOps — Individual Work Distribution

The ChainOps project is divided into five technical modules. Each member is responsible for implementing the frontend and backend functionality of their assigned module, while participating in shared system design, integration, and testing.

Member 1 — Pipeline & Execution

Overview: Develops the GitHub-integrated pipeline that automatically validates smart-contract source code. Manages pipeline execution, compilation, testing, and the presentation of results.

Frontend tasks

Design repository connection and pipeline configuration interfaces.

Build pipeline dashboard with run history and status indicators.

Develop pipeline run details showing stages, compilation and test results.

Display execution logs, errors, and options for manually triggering runs.

Backend tasks

Integrate GitHub repositories and securely handle webhook events.

Implement pipeline configurations, run creation, and state management.

Develop background job processing and isolated source-code execution.

Integrate smart-contract compilation and automated testing tools.

Store pipeline results, logs, and build artifact references for downstream modules.

Member 2 — Blockchain Deployment & Contract Registry

Overview: Handles the controlled deployment of validated smart contracts to EVM networks. Tracks blockchain transactions and maintains a searchable history of deployed contracts and versions.

Frontend tasks

Build deployment request and network selection interfaces.

Develop transaction status and confirmation tracking views.

Create the Contract Registry with contract details and versions.

Display deployment history, transaction references, and contract addresses.

Backend tasks

Validate deployment eligibility and consume approved deployment requests.

Integrate EVM RPC services for transaction preparation and submission.

Implement secure testnet signing, transaction tracking, and failure handling.

Register confirmed contracts with network, address, commit, and artifact metadata.

Provide APIs for deployment details, registry records, and version history.

Member 3 — Monitoring, Events & Alerts

Overview: Develops post-deployment monitoring for registered smart contracts. Collects blockchain activity, detects configured conditions, and presents events and alerts to users.

Frontend tasks

Build contract monitoring dashboard with activity summaries.

Create event and transaction activity history views.

Develop alert-rule creation and management interfaces.

Display generated alerts and their acknowledgement or resolution status.

Backend tasks

Implement blockchain event/log retrieval using RPC services.

Decode contract events using stored ABI information.

Persist and retrieve monitoring records while avoiding duplicate events.

Implement configurable alert-rule evaluation and alert generation.

Publish alert events to the shared notification service.

Member 4 — Security & Governance

Overview: Implements security validation, release policies, and deployment approval controls. Ensures that smart contracts satisfy defined checks before authorized deployment.

Frontend tasks

Build security findings dashboard with severity and scan details.

Develop release-policy configuration and result views.

Create the reviewer dashboard for pending deployment approvals.

Build approval/rejection interfaces with reasons and decision history.

Backend tasks

Integrate the smart-contract security analyzer and process scan results.

Normalize and store security findings linked to pipeline runs.

Implement release-policy evaluation and deployment eligibility gates.

Enforce reviewer permissions and approval/rejection workflows.

Record governance decisions and related audit information.

Member 5 — Platform, Access & AI Assistant

Overview: Develops shared platform functionality, including user access, workspace/project management, notifications, and a restricted AI assistant. Provides the common foundation used by all other modules.

Frontend tasks

Build registration, login, and account management interfaces.

Develop workspace, project, environment, and member management screens.

Create the main dashboard and shared application navigation.

Implement notification centre and user preferences interfaces.

Build the ChainOps AI Assistant chat interface.

Backend tasks

Implement authentication, session handling, and role-based access control.

Develop workspace, project, environment, and membership management APIs.

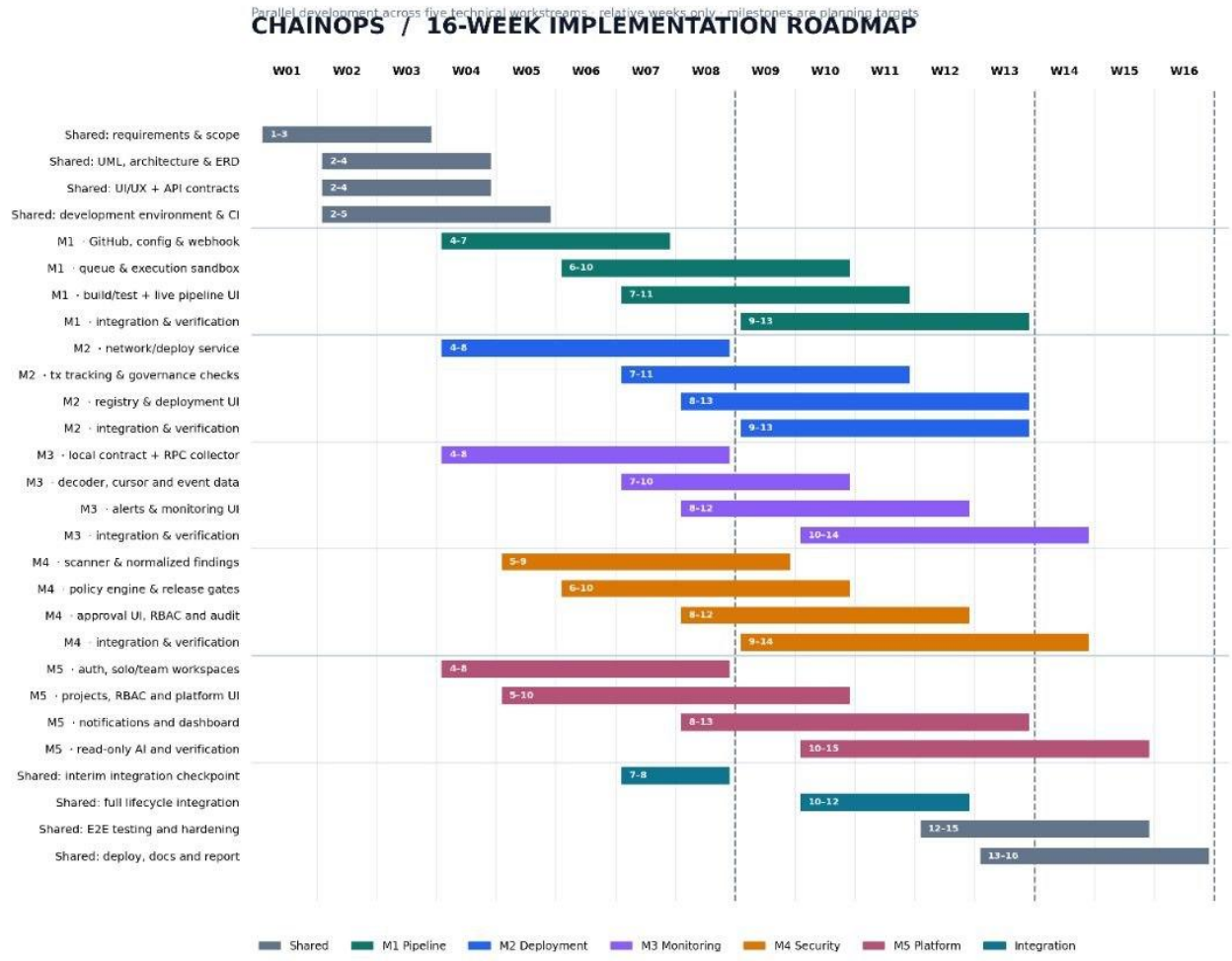
Enforce shared authorization rules across protected module endpoints.

Implement in-app notifications and notification preferences.

Integrate a read-oriented AI Assistant limited to authorized project information.

Shared team responsibilities: Requirements analysis, UML and database design coordination, UI/UX consistency, API integration, collaborative development, testing, deployment, and documentation are carried out jointly by all five members.

Appendix B – Project Timeline



Appendix C – Software Requirements Specification

Chapter 1 - Introduction

1.1 Introduction

This chapter establishes the purpose, scope and conventions of the ChainOps requirements specification. It identifies the intended audience and explains the sources used to define the proposed requirements baseline.

1.2 Purpose

ChainOps shall provide a web platform for solo developers and teams to take smart-contract code from GitHub through controlled compilation, tests and security analysis to blockchain deployment and operational monitoring. This SRS specifies externally observable behavior,

access boundaries, state rules and acceptance evidence. It is the baseline for use cases, architecture, database design, UI design and testing.

1.3 Document conventions

Shall denotes a mandatory obligation within the stated scope. Should denotes a recommended capability. Could denotes an optional capability. Must, Should and Could are delivery priorities; a conditional Must applies only when its optional parent capability is enabled. All priorities are proposed until baseline approval.

Existing FR and NFR identifiers from the supplied ChainOps details are preserved. Where the source priority is Must but its wording says should, the wording is normalized to shall. Added clarifications use CL identifiers and are proposed requirements, not evidence of previous agreement. AC identifiers are feature acceptance scenarios; VT identifiers are verification suites. TBD identifiers identify unresolved decisions.

An empty result is distinct from unavailable data. Approval is distinct from transaction success. Security scan completion is distinct from proof that a contract is secure. Times shall be stored in UTC and displayed with an explicit timezone.

1.4 Intended audience and reading suggestions

Table 1.1 presents intended audience and reading focus.

Table 1.1: Intended audience and reading focus

Audience	Reading focus
Team and supervisor	Scope, priorities, acceptance gates and Appendix C decisions
Developers and designers	Interfaces, features, business rules and logical data requirements
Testers and reviewers	Feature acceptance scenarios, quality criteria and Appendix B coverage
Operations and security reviewers	Isolation, signing boundaries, failure handling, monitoring and audit controls

1.5 Product scope

The proposed MVP includes authentication; personal and team workspaces; projects; GitHub integration; pipeline execution; compilation and tests; security gates; approvals; deployment tracking; a contract registry; monitoring; basic alert rules; in-app notifications; dashboards and audit records. Features marked Should or Could are not unconditional MVP commitments.

AI explanations are optional. Multi-ecosystem blockchain support, autoscaling, advanced anomaly detection, enterprise SSO, sophisticated AI remediation and complex multi-chain releases are future enhancements. ChainOps does not replace GitHub, guarantee vulnerability detection, provide an exchange or custodial wallet service, reverse confirmed transactions or guarantee external network availability.

1.6 References and source precedence

The supplied ChainOps requirements details [1] define the product content; the supplied example report [2] provides the original document structure. The complete source descriptions appear in References.No formal standards certification or compliance claim is made. The project guide named inside [1] is not treated as independently verified evidence. Team allocation is managed outside this product SRS because [1] describes its allocation as a proposal.

1.7 Summary

This chapter defined the purpose, conventions, audience and scope of the specification and identified its source documents. The next chapter describes the overall product and its operating context.

Chapter 2 - Overall description

2.1 Introduction

The preceding chapter defined the scope of the specification. This chapter describes the product context, user classes, operating environment, constraints and dependencies that shape the requirements.

2.2 Product perspective

ChainOps is an orchestration platform between a source-code host, isolated execution tools, reviewers and blockchain services. GitHub remains the canonical source repository. ChainOps retains configuration, run evidence, deployment and monitoring records; temporary source checkouts are removed after execution. Blockchain records remain independent of the application database.

Logical responsibilities are a browser UI, authenticated backend, durable store, asynchronous work queue, isolated runner, policy and approval service, deployment adapter and monitoring service. These are responsibility boundaries, not a commitment to separate microservices.

2.3 Product functions

- Create a personal or team workspace and authorize project access.
- Connect a permitted repository and configure a supported pipeline.
- Receive an authenticated repository event or manual trigger and bind a run to an exact commit.
- Compile, test and scan in isolation; persist evidence and evaluate mandatory gates.
- Bind a deployment request to immutable evidence and collect required approvals.
- Submit through an authorized signing mechanism and reconcile the transaction result.
- Register successful deployments, collect supported activity and evaluate alert rules.
- Present permitted results, notifications and an attributable audit history.

2.4 User classes and characteristics

Table 2.1 presents user classes and responsibilities.

Table 2.1: User classes and responsibilities

User class	Responsibility
Developer	Configures permitted projects and pipelines, investigates results and requests deployment.
Reviewer	Reviews exact deployment evidence and records approval or rejection when authorized.
Security reviewer	Examines findings and records security outcomes where that feature is enabled.
Operations engineer	Reviews contract activity, maintains permitted alert rules and acknowledges or resolves alerts.
Workspace owner or administrator	Manages workspace membership and privileged settings; approval permission is separately enforced.

Roles grant capabilities within authorized resources. One user may hold multiple roles. A role never grants implicit access to another workspace. External actors are GitHub, contract toolchains, security scanners, blockchain RPC services and networks, notification services and an optional AI provider.

2.5 Operating environment

Users access ChainOps through an agreed modern web browser over a protected connection. The server environment shall provide persistent data storage and background execution. Supported browsers, deployment capacity and hosting are recorded under TBD-08. Network access to GitHub and selected providers is required for their respective workflows; cached results must remain visibly dated during outages.

2.6 Design and implementation constraints

Untrusted repository code shall execute outside the main API process with enforced resource, filesystem and network boundaries. Source commit, configuration and artifact identity shall be retained for traceability. Deployment keys shall not be available to compilation or test scripts. The backend shall enforce authorization independently of UI controls.

Candidate technologies in [1] include React and TypeScript, Next.js or a React SPA, Node.js, PostgreSQL, an ORM, a queue and a restricted container runner. Solidity, Hardhat or Foundry and an EVM-compatible test network are candidates. These choices do not become approved dependencies merely by appearing here; exact selections and versions require Appendix C decisions.

2.7 User documentation

- Getting started guide for sign-in, workspaces and repository connection.
- Pipeline configuration guide including supported versions, limits and diagnostics.
- Deployment guide explaining evidence, approvals, signing and transaction states.
- Monitoring guide explaining event coverage, data freshness, rule configuration and alert handling.
- Administrator and operator guide for permissions, secrets, backups, restore and service failures.

2.8 Assumptions and dependencies

Users must have the necessary GitHub repository access and access to the chosen target network. Deployment depends on a configured signer, provider connectivity and sufficient test-network funds where fees apply. Event decoding depends on compatible contract metadata. External services may impose quotas or become unavailable. Scope is limited to the explicitly supported toolchain and network profiles.

2.9 Summary

This chapter identified the product functions, users, operating environment, constraints and dependencies. The next chapter specifies the interfaces through which users and external services interact with ChainOps.

Chapter 3 - External interface requirements

3.1 Introduction

The preceding chapter described the environment in which ChainOps operates. This chapter defines how users and external services interact with the platform, including interface behavior and communications.

3.2 User interfaces

Table 3.1 presents user interface screen groups.

Table 3.1: User interface screen groups

Screen group	Required content and behavior
Authentication and workspaces	Sign-in, applicable recovery, workspace selection, profile and session controls.
Projects and integration	Project list, settings, archive action, repository authorization and connection status.
Pipelines and security	Configuration, run timeline, stage logs, findings, artifacts and reasons for blocked eligibility.
Deployment and registry	Exact commit, artifact, network, inputs, approvals, transaction state, address and history.
Monitoring and alerts	Network and contract context, freshness, raw and decoded event history, rule editor and alert lifecycle.
Administration and audit	Membership and permissions, policy settings and authorized audit filtering.
Optional assistant	Contextual explanations clearly labelled as generated and linked to permitted evidence.

CL-UI-01 [Must] The interface shall distinguish loading, empty, denied, failed and stale states and shall show actionable validation errors.

CL-UI-02 [Must] Before approval or submission, the interface shall display the request identity, commit, artifact identity, target network and material deployment inputs.

3.3 Hardware interfaces

No specialized end-user hardware interface is specified. Users need a supported browser and connectivity. Server sizing and any hardware-backed signing integration depend on the selected deployment design. No hardware wallet support is implied by this SRS.

3.4 Software interfaces

Table 3.2 presents external software interfaces.

Table 3.2: External software interfaces

Interface	Inputs and outputs	Failure contract
GitHub	Installation and repository references; verified event payload; immutable commit reference.	Reject unauthentic events; deduplicate delivery; surface revoked access and provider limits.
Toolchain and scanner	Pinned configuration and source input; diagnostics, results, findings and artifacts.	Distinguish tool error from failed checks and a clean scan; mandatory incomplete checks block deployment.
Blockchain adapter	Network identity, signed transaction or authorized signing request; hash, receipt and logs.	Verify network identity; preserve ambiguous states; never infer success from submission.
Notification provider	Authorized recipient reference and redacted event message; available delivery status.	Retry within bounded policy without duplicating logical notifications.
Optional AI	Permitted, redacted evidence and user question; generated explanation.	Keep core workflow operational when AI is absent or unavailable.

CL-IF-01 [Must] API operations shall validate identity, workspace scope, input schema and state preconditions before mutation; errors shall return a stable code and safe diagnostic message.

CL-IF-02 [Must] Collection interfaces shall use bounded pagination and deterministic ordering; log and event responses shall include source context and collection time.

CL-IF-03 [Must] The integration contract shall document supported API versions, webhook event types, timeouts, retry limits and provider quota handling before implementation acceptance.

3.5 Communications interfaces

Browser and service communications shall use authenticated endpoints and transport protection. Webhook authenticity shall be verified against the received payload using the selected provider mechanism. Retries shall use bounded delays; state-changing calls shall use idempotency or explicit reconciliation. Exact transports for UI updates and blockchain subscriptions are implementation choices, not additional product commitments.

3.6 User stories

US-01 As a solo developer, I want a personal workspace so I can complete the lifecycle without an organization.

US-02 As a team developer, I want repository changes to run configured checks so I receive traceable feedback.

US-03 As a reviewer, I want exact deployment evidence so my decision applies to the intended release.

US-04 As a security reviewer, I want findings and policy outcomes separated so I can assess risks accurately.

US-05 As an operations engineer, I want decoded events and freshness indicators so I can investigate observed activity.

US-06 As an administrator, I want scoped permissions and attributable policy changes so access remains controlled.

US-07 As an authorized user, I want alert history and notifications so I can respond to operational conditions.

3.7 Summary

This chapter defined the user, hardware, software and communications interfaces and the supporting user stories. The next chapter describes the functional requirements and their acceptance scenarios.

Chapter 4 - System features

4.1 Introduction

The preceding chapter defined the external interfaces. This chapter specifies the platform features through identified requirements, stimulus and response sequences, exception handling and acceptance scenarios.

Each feature contains its purpose, stimulus and response sequence, failure behavior, identified requirements and an acceptance scenario. Source requirement identifiers are retained. Detailed test fixtures shall be recorded against every selected requirement before acceptance; scenario summaries here do not assert test execution.

4.2 Authentication and account management

Authenticate users and establish identity for protected actions. Priority: Must for core requirements; individual exceptions are identified below.

4.2.1 Stimulus and response

User selects sign-in; the selected identity mechanism validates the request; ChainOps establishes a session and opens authorized workspace content. Sign-out invalidates the active session as designed.

4.2.2 Exceptions and controls

Invalid, expired or missing identity denies protected access. Recovery is applicable only to the chosen authentication mechanism.

4.2.3 Functional requirements

FR-AUTH-01 [Must] The system shall allow users to register or sign in using the selected authentication mechanism.

FR-AUTH-02 [Must] The system shall maintain an authenticated user identity for each protected request.

FR-AUTH-03 [Must] The system shall allow users to view and update permitted profile settings.

FR-AUTH-04 [Must] The system shall allow users to sign out and invalidate their active session according to the chosen authentication design.

FR-AUTH-05 [Must] The system shall restrict unauthorized access to protected pages, API endpoints and resources.

FR-AUTH-06 [Should] The system should support account recovery if password-based authentication is selected.

FR-AUTH-07 [Should] The system should support stronger authentication for privileged users where supported by the selected identity provider.

4.2.4 AC-AUTH Acceptance scenario

Given a valid user, sign-in permits authorized access; sign-out prevents reuse of the invalidated session. Invalid credentials and cross-user resource access are denied.

4.3 Workspace and project management

Support personal ownership and scoped team collaboration. Priority: Must for core requirements; individual exceptions are identified below.

4.3.1 Stimulus and response

An authorized user creates a workspace or project; the system validates ownership and records it. An administrator invites members and grants resource-scoped permissions.

4.3.2 Exceptions and controls

Nonmembers cannot list, read or mutate workspace records. Archiving preserves retained evidence and blocks new work according to the archive rule.

4.3.3 Functional requirements

FR-WS-01 [Must] The system shall support personal workspaces.

FR-WS-02 [Must] The system shall support team workspaces with multiple members.

FR-WS-03 [Must] The system shall associate each project with exactly one owning workspace.

FR-WS-04 [Must] The system shall allow authorized users to create, view, update and archive projects.

FR-WS-05 [Must] The system shall allow authorized administrators to invite members and manage membership.

FR-WS-06 [Must] The system shall support role and permission assignment within team workspaces.

FR-WS-07 [Must] The system shall enforce workspace isolation when retrieving or modifying data.

FR-WS-08 [Must] The system shall record privileged workspace setting changes.

4.3.4 AC-WS Acceptance scenario

Create both workspace types, add a team member, exercise allowed actions and deny equivalent requests by a nonmember, including direct API requests.

4.4 GitHub integration and repository management

Connect permitted repositories and receive authenticated change events. Priority: Must for core requirements; individual exceptions are identified below.

4.4.1 Stimulus and response

User authorizes integration and chooses a permitted repository; the system links it to a project. A valid configured event is deduplicated and routed to pipeline creation.

4.4.2 Exceptions and controls

Invalid signatures are rejected before trusted processing. Revoked access and missing repositories show unavailable status. Duplicate delivery does not create a second logical run.

4.4.3 Functional requirements

FR-GH-01 [Must] The system shall allow a user to authorize ChainOps to access permitted GitHub repositories.

FR-GH-02 [Must] The system shall list repositories available to the authenticated GitHub installation or authorized user.

FR-GH-03 [Must] The system shall allow an authorized user to associate a repository with a ChainOps project.

FR-GH-04 [Must] The system shall store repository metadata and the relevant GitHub installation reference.

FR-GH-05 [Must] The system shall receive configured repository events through webhooks.

FR-GH-06 [Must] The system shall verify webhook authenticity before processing an event.

FR-GH-07 [Must] The system shall handle repeated webhook delivery without unintentionally creating duplicate pipeline runs.

FR-GH-08 [Must] The system shall allow authorized users to disconnect a repository integration.

FR-GH-09 [Must] The system shall handle revoked access, unavailable repositories and invalid events gracefully.

FR-GH-10 [Should] The system should allow users to select relevant branches and trigger conditions.

4.4.4 AC-GH Acceptance scenario

Connect a permitted repository, deliver a valid event twice and observe one logical run; forged delivery and revoked access must not produce unauthorized work.

4.5 Pipeline configuration and execution

Run controlled asynchronous validation for an exact source revision. Priority: Must for core requirements; individual exceptions are identified below.

4.5.1 Stimulus and response

Authorized user saves configuration; a webhook or manual action queues a run; the worker claims it, records stage progress and retains the terminal result and diagnostics.

4.5.2 Exceptions and controls

Timeout and failed mandatory stages block success. Cancellation is optional; cleanup is mandatory for every terminal path. Retries are distinguishable attempts.

4.5.3 Functional requirements

FR-PIPE-01 [Must] The system shall allow authorized users to create and update pipeline configurations.

FR-PIPE-02 [Must] The system shall associate a pipeline configuration with a project and repository.

FR-PIPE-03 [Must] The system shall support configurable build, test and security-check stages.

FR-PIPE-04 [Must] The system shall trigger configured pipeline runs when relevant repository events occur.

FR-PIPE-05 [Must] The system shall support a manual pipeline trigger for authorized users.

FR-PIPE-06 [Must] Each run shall record the triggering event, commit identifier, configuration version, start time, end time and status.

FR-PIPE-07 [Must] The system shall expose pipeline stage status and execution logs to authorized users.

FR-PIPE-08 [Must] The system shall distinguish queued, running, succeeded, failed, cancelled and timed-out runs.

FR-PIPE-09 [Must] The system shall enforce execution time and resource limits for pipeline jobs.

FR-PIPE-10 [Must] The system shall clean up temporary execution environments after completion or failure.

FR-PIPE-11 [Should] The system should support pipeline cancellation by authorized users.

FR-PIPE-12 [Must] The system shall retain historical run results for comparison and troubleshooting.

4.5.4 AC-PIPE Acceptance scenario

Trigger a run manually and by event, observe all status changes, force a timeout and verify limits, cleanup, retained logs and exact commit/configuration references.

4.6 Compilation and automated testing

Produce reproducible evidence and eligible build outputs for supported projects. Priority: Must for core requirements; individual exceptions are identified below.

4.6.1 Stimulus and response

Worker checks out the selected commit, invokes the configured compiler and tests, captures results and stores successful artifacts with their provenance.

4.6.2 Exceptions and controls

Unsupported toolchains or compilation/test errors produce explicit failures. Failed mandatory stages prevent deployment eligibility.

4.6.3 Functional requirements

FR-BUILD-01 [Must] The system shall identify the configured smart-contract toolchain for a project.

FR-BUILD-02 [Must] The system shall compile supported smart-contract source code in an isolated environment.

FR-BUILD-03 [Must] The system shall record compilation success or failure and relevant diagnostics.

FR-BUILD-04 [Must] The system shall execute configured automated tests when the required toolchain is available.

FR-BUILD-05 [Must] The system shall record test outcomes and summaries.

FR-BUILD-06 [Must] The system shall produce deployment artifacts and metadata for successful builds when required by the configured toolchain.

FR-BUILD-07 [Must] The system shall associate artifacts with the relevant project, pipeline run and commit.

FR-BUILD-08 [Must] The system shall prevent artifacts from being treated as deployment-eligible when required compilation or test stages have failed.

4.6.4 AC-BUILD Acceptance scenario

Run a passing fixture and separate compile-failure and test-failure fixtures; verify artifact association and denial of deployment for failed fixtures.

4.7 Security analysis and policy enforcement

Evaluate scanner findings independently from execution success and human approval. Priority: Must for core requirements; individual exceptions are identified below.

4.7.1 Stimulus and response

Configured scanners execute; the system normalizes findings, records tool identity and evaluates versioned thresholds; permitted reviewers inspect evidence.

4.7.2 Exceptions and controls

A scanner error or missing mandatory result is not a clean scan. A blocking result cannot be bypassed by ordinary approval.

4.7.3 Functional requirements

FR-SEC-01 [Must] The system shall execute the security checks enabled in the pipeline configuration.

FR-SEC-02 [Must] The system shall store findings with severity, description, source reference where available, and originating tool.

FR-SEC-03 [Must] The system shall display findings in a security results interface.

FR-SEC-04 [Must] The system shall support configurable policy thresholds for blocking, warning and passing results.

FR-SEC-05 [Must] The system shall evaluate whether required checks have completed before determining deployment eligibility.

FR-SEC-06 [Should] The system should allow authorized security reviewers to inspect findings and record review outcomes.

FR-SEC-07 [Must] The system shall retain the security-check results associated with a deployment request.

FR-SEC-08 [Must] The system shall not treat a human approval as a bypass for mandatory security gates.

FR-SEC-09 [Should] The system should support tracking whether findings have been acknowledged, accepted under policy or remediated.

4.7.4 AC-SEC Acceptance scenario

Execute clean, blocking-finding and scanner-error fixtures; show the difference and block deployment for either unmet mandatory condition.

4.8 Deployment governance and approvals

Authorize a specific release using validated evidence and workspace policy. Priority: Must for core requirements; individual exceptions are identified below.

4.8.1 Stimulus and response

Developer creates a request from eligible evidence; the system determines required approvals; an authorized reviewer records a decision; submission eligibility is rechecked.

4.8.2 Exceptions and controls

Unauthorized approval and forbidden self-approval are rejected. Changes to reviewed inputs invalidate approval; concurrent attempts cannot create duplicate processing.

4.8.3 Functional requirements

FR-GOV-01 [Must] The system shall allow authorized users to create a deployment request from an eligible pipeline run.

FR-GOV-02 [Must] The system shall bind a deployment request to the project, commit, pipeline run, artifact and target network.

FR-GOV-03 [Must] The system shall evaluate the workspace's deployment policy before allowing submission.

FR-GOV-04 [Must] The system shall determine whether human approval is required for a deployment request.

FR-GOV-05 [Must] The system shall allow authorized reviewers to approve or reject eligible requests.

FR-GOV-06 [Must] The system shall record the approver, decision, timestamp and optional reason.

FR-GOV-07 [Must] The system shall prevent users without the required permission from approving requests.

FR-GOV-08 [Must] The system shall support a workspace setting that enables or disables self-approval.

FR-GOV-09 [Must] Only authorized owners or administrators shall be able to change the self-approval setting.

FR-GOV-10 [Must] The system shall retain an audit record of self-approval setting changes and relevant approval decisions.

FR-GOV-11 [Must] The system shall require re-evaluation if the source commit, artifact, policy or target network changes after approval.

FR-GOV-12 [Must] The system shall prevent duplicate processing of the same deployment request where the execution result is already known.

CL-GOV-01 [Must] Review evidence shall include artifact digest, constructor or deployment inputs and signer identity where material. Submission shall atomically claim a request and recheck current permissions, approval validity and mandatory gates.

4.8.4 AC-GOV Acceptance scenario

Approve an eligible request; then change a material input and confirm prior approval is unusable. Test forbidden self-approval, permission loss and simultaneous submit attempts.

4.9 Blockchain network and deployment management

Submit authorized deployment transactions and report evidence-based outcomes. Priority: Must for core requirements; individual exceptions are identified below.

4.9.1 Stimulus and response

System validates the target and inputs, rechecks gates, invokes the configured signer, records submission evidence and polls or receives receipt information.

4.9.2 Exceptions and controls

Submission error, revert, pending, timeout and unknown outcome remain distinct. Ambiguous submission is reconciled before any retry.

4.9.3 Functional requirements

FR-DEP-01 [Must] The system shall support the configured blockchain networks selected for the MVP.

FR-DEP-02 [Must] The system shall allow authorized users to configure or select an approved target network.

FR-DEP-03 [Must] The system shall validate deployment configuration before transaction submission.

FR-DEP-04 [Must] The system shall submit deployment transactions only after all mandatory gates have passed.

FR-DEP-05 [Must] The system shall record the transaction hash when a transaction has been submitted and the provider returns a hash.

FR-DEP-06 [Must] The system shall track transaction-pending, confirmed and failed states.

FR-DEP-07 [Must] The system shall record the deployed contract address when deployment succeeds and the address is available.

FR-DEP-08 [Must] The system shall associate deployment metadata with the source commit, artifact, pipeline run and target network.

FR-DEP-09 [Must] The system shall distinguish submission errors from transaction reverts and confirmation timeouts.

FR-DEP-10 [Must] The system shall display deployment history to authorized users.

FR-DEP-11 [Must] The system shall never report a deployment as successful solely because a transaction was submitted.

FR-DEP-12 [Must] The system shall support retries only when the transaction state and idempotency rules have been evaluated.

CL-DEP-01 [Must] Confirmed status shall require a successful execution receipt, expected deployment evidence and the configured confirmation policy. Receipt loss or a detected reorganization shall trigger reconciliation and a visible status correction.

4.9.4 AC-DEP Acceptance scenario

Submit to the selected test environment; observe pending before confirmed, retain a reverted transaction as failed, and inject an ambiguous response without creating a duplicate deployment.

4.10 Contract registry

Keep deployment identity and contract location traceable. Priority: Must for core requirements; individual exceptions are identified below.

4.10.1 Stimulus and response

A successful deployment produces a registry entry containing project, network, address and source deployment references; an authorized user opens its details.

4.10.2 Exceptions and controls

The same address on different networks is never conflated. External registration is optional and shall be labelled if enabled.

4.10.3 Functional requirements

FR-REG-01 [Must] The system shall maintain a registry of deployed contracts.

FR-REG-02 [Must] Each registry entry shall identify its project, workspace, network, contract address and associated deployment.

FR-REG-03 [Must] The system shall record the deployment transaction hash and timestamp where available.

FR-REG-04 [Must] The system shall allow authorized users to view contract details and deployment history.

FR-REG-05 [Should] The system should distinguish contracts deployed through ChainOps from externally registered contracts if external registration is implemented.

FR-REG-06 [Must] The system shall prevent an address on one network from being mistaken for the same address on another network.

4.10.4 AC-REG Acceptance scenario

Register successful deployments, retrieve provenance and demonstrate that identical address strings on two network profiles remain distinct identities.

4.11 Monitoring events and alerts

Observe supported contract activity, evaluate rules and retain operational history. Priority: Must for core requirements; individual exceptions are identified below.

4.11.1 Stimulus and response

Authorized user selects a contract and rule; collector retrieves supported data, records freshness and decodes compatible events; rule evaluation creates an alert that an operator can acknowledge and resolve.

4.11.2 Exceptions and controls

Unavailable or stale data is not no activity. Decode failures retain raw evidence. Duplicate collection and reorganization handling follow the rules below.

4.11.3 Functional requirements

FR-MON-01 [Must] The system shall allow authorized users to view monitored contracts and their network associations.

FR-MON-02 [Must] The system shall retrieve relevant transaction or event data from supported networks or providers.

FR-MON-03 [Must] The system shall display available transaction and event history.

FR-MON-04 [Must] The system shall allow authorized users to create, update, enable and disable alert rules.

FR-MON-05 [Must] The system shall evaluate configured rules against supported monitoring data.

FR-MON-06 [Must] The system shall create an alert record when a rule is triggered.

FR-MON-07 [Must] The system shall support alert states such as open, acknowledged and resolved.

FR-MON-08 [Must] The system shall record who acknowledged or resolved an alert and when.

FR-MON-09 [Must] The system shall display alert history and relevant event details.

FR-MON-10 [Should] The system should notify configured recipients when specified alerts or workflow events occur.

FR-MON-11 [Must] The system shall handle temporary monitoring-provider failures without falsely reporting that no events occurred.

FR-MON-12 [Must] The system shall distinguish unavailable monitoring data from a confirmed absence of activity.

CL-MON-01 [Must] The system shall retain raw event topics and data, transaction hash, log index, block reference, network and contract identity for each collected log, and shall decode it using the associated ABI when supported.

CL-MON-02 [Must] The system shall label decoded, undecodable and metadata-unavailable events separately and retain raw evidence when decoding fails.

CL-MON-03 [Must] The system shall track collection checkpoints and show last successful collection time, last processed block where applicable and monitoring status as active, paused, stale or unavailable.

CL-MON-04 [Must] The system shall resume bounded event retrieval from a persisted checkpoint and deduplicate observations using network, block hash, transaction hash and log index for EVM event monitoring.

CL-MON-05 [Must] The system shall reconcile detected chain reorganizations, mark superseded events as noncanonical and prevent them from remaining represented as confirmed evidence. Affected alerts shall retain correction history.

CL-MON-06 [Must] Alert rules shall define the monitored scope, supported condition, severity, enabled state and deduplication policy. Rule changes shall be versioned and audited.

CL-MON-07 [Must] For a given rule version and canonical event identity, reprocessing shall not create duplicate logical alerts. A new occurrence after resolution shall create a distinct attributable alert.

CL-MON-08 [Must] The UI shall describe the supported observation coverage. Contract logs alone shall not be described as complete transaction history or proof that a contract is healthy.

4.11.4 AC-MON Acceptance scenario

Ingest a known event, decode its parameters, trigger a rule, acknowledge and resolve the alert. Re-ingest the event without duplicate alerting, then simulate provider outage and verify stale status.

4.12 Notifications

Present relevant workflow messages without leaking sensitive information. Priority: Must for core requirements; individual exceptions are identified below.

4.12.1 Stimulus and response

A selected event creates an in-app notification for an authorized recipient; configured channels dispatch it and record available provider status.

4.12.2 Exceptions and controls

Provider failure does not roll back the underlying deployment or alert. Delivery acceptance is not proof of reading.

4.12.3 Functional requirements

FR-NOT-01 [Must] The system shall display relevant workflow notifications within the application.

FR-NOT-02 [Should] The system should notify authorized users about configured events such as approval requests and deployment outcomes.

FR-NOT-03 [Could] The system should support notification preferences where practical.

FR-NOT-04 [Should] The system should record notification delivery status when the provider exposes it.

FR-NOT-05 [Must] The system shall avoid including credentials, private keys or other secrets in notifications.

4.12.4 AC-NOT Acceptance scenario

Trigger an approval or alert notification; verify authorized visibility and redaction. If external delivery is enabled, simulate failure and inspect bounded retry and status.

4.13 Audit logs and reporting

Provide protected, attributable history of important actions. Priority: Must for core requirements; individual exceptions are identified below.

4.13.1 Stimulus and response

A privileged action or state transition emits an audit entry; authorized users retrieve retained records and enabled filters.

4.13.2 Exceptions and controls

Ordinary users cannot edit audit history. Sensitive values are excluded; audit-write failures follow the critical-action rule in section 5.10.

4.13.3 Functional requirements

FR-AUD-01 [Must] The system shall record security-relevant actions and important state transitions.

FR-AUD-02 [Must] Audit entries shall identify the actor or system process, action, target resource and timestamp.

FR-AUD-03 [Must] The system shall record approval decisions, permission changes and deployment events.

FR-AUD-04 [Must] The system shall restrict audit-log access to authorized users.

FR-AUD-05 [Must] The system shall prevent ordinary users from modifying audit history through normal application functions.

FR-AUD-06 [Should] The system should support filtering audit entries by time, actor, project and action.

FR-AUD-07 [Must] The system shall avoid recording plaintext secrets and unnecessary sensitive data in logs.

FR-AUD-08 [Must] The system shall retain audit records according to a defined retention policy.

4.13.4 AC-AUD Acceptance scenario

Change permissions and policy, approve and submit a request, then verify actor, target, time and outcome records; deny unauthorized read and modification.

4.14 Dashboard and reporting

Present authorized development and operational status. Priority: Must for core requirements; individual exceptions are identified below.

4.14.1 Stimulus and response

User selects a workspace; the system retrieves permitted projects, runs, pending approvals, contracts and alerts with current or explicitly dated state.

4.14.2 Exceptions and controls

Empty results, stale data and access denial are distinct. Summaries must not reveal other workspaces through counts.

4.14.3 Functional requirements

FR-DASH-01 [Must] The system shall provide a dashboard showing authorized projects and their recent pipeline activity.

FR-DASH-02 [Must] The system shall display pipeline success and failure states.

FR-DASH-03 [Must] The system shall display pending deployment approvals.

FR-DASH-04 [Must] The system shall display relevant deployments, contracts and operational alerts.

FR-DASH-05 [Must] The system shall filter displayed information according to workspace membership and permissions.

FR-DASH-06 [Should] The system should provide summary reports for pipeline, deployment and security activity.

4.14.4 AC-DASH Acceptance scenario

Populate two isolated workspaces and verify each dashboard includes only its permitted records and accurately reflects pending, failed and confirmed outcomes.

4.15 Optional AI assistant

Explain permitted evidence without controlling authorization or deployment. Priority: Could; mandatory safeguards apply if enabled.

4.15.1 Stimulus and response

When enabled, user requests an explanation; the system selects authorized redacted evidence and displays generated guidance with available references.

4.15.2 Exceptions and controls

Provider failure does not block core workflows. Generated text cannot grant permissions, approve or override a gate.

4.15.3 Functional requirements

FR-AI-01 [Could] The system could provide an assistant that explains pipeline failures and security findings.

FR-AI-02 [Must if AI is implemented] The assistant shall only use project data the requesting user is authorized to access.

FR-AI-03 [Must if AI is implemented] The assistant shall distinguish generated recommendations from verified pipeline results.

FR-AI-04 [Must if AI is implemented] The assistant shall not independently bypass deployment gates or grant permissions.

FR-AI-05 [Must if AI is implemented] The system shall avoid sending private keys, credentials and other secrets to an external AI provider.

FR-AI-06 [Should] The assistant should link explanations to available logs, findings or relevant pipeline evidence.

4.15.4 AC-AI Acceptance scenario

When included, test evidence scoping, secret exclusion, generated-content labels and provider failure; no assistant action may change release eligibility.

4.16 Summary

This chapter specified the fourteen feature groups and their requirements, controls and acceptance scenarios. The next chapter defines nonfunctional requirements, safety rules and quality verification.

Chapter 5 - Other nonfunctional requirements

5.1 Introduction

The preceding chapter specified functional behavior. This chapter establishes quality requirements, deployment safety controls, business rules and the verification profile for assessing the proposed system.

Numerical targets remain proposed until the test environment is approved. Qualitative controls are verified by the methods below. A deployment must not be advertised as meeting a service level or security certification solely because that target appears in this specification.

5.2 Performance requirements

NFR-PERF-01 [Proposed target] For ordinary dashboard and metadata API requests, the system should achieve a server response time of 2 seconds or less at the 95th percentile under the agreed test load.

NFR-PERF-02 [Must] Long-running compilation, security checks and deployments shall run asynchronously rather than block an ordinary web request until completion.

NFR-PERF-03 [Must] The UI shall show clear queued, running and completed states for asynchronous operations.

NFR-PERF-04 [Must] The team shall define a test load and document performance measurements before final evaluation.

5.3 Security requirements

NFR-SEC-01 [Must] All protected server operations shall enforce authentication and authorization on the backend.

NFR-SEC-02 [Must] The system shall apply least privilege to users, GitHub integrations, pipeline workers and external services.

NFR-SEC-03 [Must] Sensitive data shall be protected in transit and appropriately protected at rest.

NFR-SEC-04 [Must] Secrets shall not be committed to source control, embedded in application images or written to ordinary logs.

NFR-SEC-05 [Must] The system shall validate all untrusted inputs, including webhook payloads and pipeline configuration.

NFR-SEC-06 [Must] Pipeline execution shall be isolated from the main application and from other projects' execution environments.

NFR-SEC-07 [Must] The system shall apply resource limits and restrict potentially dangerous operations during pipeline execution.

NFR-SEC-08 [Must] The system shall protect deployment credentials and restrict transaction-signing capability to the configured deployment mechanism.

NFR-SEC-09 [Must] The system shall verify webhook authenticity and apply replay/duplicate handling controls.

NFR-SEC-10 [Must] The system shall not expose sensitive environment variables or private keys through build logs, error messages or AI requests.

5.4 Reliability and recoverability

NFR-REL-01 [Must] A failed pipeline stage shall produce a visible status and preserve available diagnostic evidence.

NFR-REL-02 [Must] Duplicate webhook deliveries shall not create duplicate logical runs for the same triggering event unless explicitly requested.

NFR-REL-03 [Must] The system shall handle temporary external-service failures using bounded retry or failure handling appropriate to the operation.

NFR-REL-04 [Must] The system shall persist pipeline and deployment state so that a web-server restart does not erase recorded results.

NFR-REL-05 [Must] The system shall reconcile ambiguous blockchain transaction states before any potentially duplicating retry.

NFR-REL-06 [Must] The system shall report when an external service is unavailable rather than present stale data as current.

NFR-REL-07 [Must] Database backup and recovery procedures shall be documented for the selected deployment environment.

5.5 Usability and accessibility

NFR-UX-01 [Must] The application shall provide consistent navigation for dashboard, projects, pipelines, security, deployments, monitoring and settings.

NFR-UX-02 [Must] Status labels shall distinguish success, failure, warning, pending and running states without relying on color alone.

NFR-UX-03 [Must] Forms shall validate input and provide actionable error messages.

NFR-UX-04 [Must] Destructive or high-impact actions shall clearly communicate their consequences and request confirmation where appropriate.

NFR-UX-05 [Should] The UI should support responsive layouts for common desktop and tablet screen sizes.

NFR-UX-06 [Should] The interface should provide appropriate keyboard navigation and accessible labels.

5.6 Maintainability and testability

NFR-MNT-01 [Must] The codebase shall use a consistent project structure and documented coding conventions.

NFR-MNT-02 [Must] Business rules, access-control decisions and deployment gates shall be testable independently of UI rendering.

NFR-MNT-03 [Must] The system shall provide automated tests for critical authorization, pipeline and deployment workflows.

NFR-MNT-04 [Must] Environment-specific configuration shall be separated from application source code.

NFR-MNT-05 [Must] Database changes shall be managed through versioned migrations.

NFR-MNT-06 [Must] External integrations shall be isolated behind defined interfaces where practical.

5.7 Compatibility and portability

NFR-COMP-01 [Must] The web interface shall support the agreed modern browsers.

NFR-COMP-02 [Must] The pipeline runner shall use explicitly supported toolchain versions.

NFR-COMP-03 [Must] Blockchain operations shall target only configured, supported networks.

NFR-COMP-04 [Must] The team shall document environment variables, dependencies and deployment instructions required to run the application.

5.8 Auditability and observability

NFR-OBS-01 [Must] Errors and major state transitions shall be logged with timestamps and correlation identifiers where appropriate.

NFR-OBS-02 [Must] Logs shall exclude credentials and unnecessary sensitive information.

NFR-OBS-03 [Must] Audit records shall be access-controlled and protected against ordinary application-level modification.

NFR-OBS-04 [Must] Pipeline and deployment records shall be traceable to their initiating event or user action.

NFR-OBS-05 [Must] The team shall define retention periods for logs, build artifacts and other stored records.

5.9 Safety requirements

CL-SAFE-01 [Must] The system shall not submit a deployment with failed, absent or errored mandatory validation evidence.

CL-SAFE-02 [Must] The system shall treat confirmed on-chain changes as externally persistent. Archiving or deleting application data shall not be described as reversing deployment.

CL-SAFE-03 [Must] Signing authority shall be separated from untrusted build execution and shall authorize only the intended network, artifact and material deployment inputs.

CL-SAFE-04 [Must] The system shall not automatically replace or resubmit a transaction with an unknown outcome until reconciliation establishes the permissible action.

5.10 Business rules

Table 5.1 presents business rules.

Table 5.1: Business rules

Rule	Required behavior
BR-01 Ownership	Every project belongs to exactly one workspace. Access requires both membership or permitted ownership and the requested resource capability.
BR-02 Role separation	Owner or administrator status does not by itself satisfy an approval permission check. Self-approval requires both explicit approval privilege and enabled policy.
BR-03 Evidence identity	Approval binds commit, configuration, artifact digest, target network and material deployment inputs. Relevant changes require a new evidence version and reevaluation.
BR-04 Mandatory gates	Compilation, tests and scans designated mandatory must complete successfully under the approved policy. Human approval never turns failed mandatory evidence into a pass.
BR-05 Current policy	Policy version and approval validity are rechecked immediately before submission. Policy changes cannot silently authorize stale evidence.
BR-06 Archive	Proposed archive behavior: stop new runs and deployment requests; preserve authorized read access to retained evidence. Submitted transactions must still be reconciled.
BR-07 Alerts	Acknowledgement records human attention; it does not prove recovery. Manual resolution requires actor and timestamp.
BR-08 Critical audit	Policy, permission and approval mutations shall commit with durable audit evidence. Persist submission intent before the external transaction side effect and reconcile its outcome after failure.
BR-09 Data freshness	Monitoring availability and contract activity are distinct. Empty history is valid only for the successfully observed range.
BR-10 Concurrency	Only one worker may claim the same logical execution at a time. Duplicate delivery or concurrent commands must not bypass gates or create repeated side effects.

5.11 Quality verification profile

Table 5.2 presents quality verification profile.

Table 5.2: Quality verification profile

Suite	Verification and pass evidence
VT-PERF	Proposed load: 20 concurrent authenticated users, 1,000 runs and 10,000 events, 10 minute warm-up then 15 minute measurement. Dashboard and metadata server latency p95 <= 2 seconds; long jobs excluded. Record hardware, dataset, errors and percentiles. Team approval required under TBD-08.
VT-SEC	Execute positive and negative authorization cases for each protected operation, including object IDs from another workspace. Test tampered events, untrusted inputs, isolated runner access and secret canaries. No unauthorized data or secret exposure is permitted in the tested paths.
VT-REL	Interrupt API, worker and provider separately; replay events and reconcile ambiguous submissions. Persisted evidence survives restart and no duplicate logical side effect occurs. Demonstrate backup restore against selected recovery objectives.
VT-UX	Complete sign-in, repository connection, result review, approval and alert acknowledgement using keyboard navigation; verify labels, visible focus, non-color status indicators and actionable errors on agreed browser sizes.
VT-MNT	Run automated authorization, policy and state-transition tests; apply migrations to a clean database; follow setup documentation to start the selected configuration. Record versions and results.
VT-COMP	Execute the core workflow for each approved browser, toolchain version and network profile; reject unsupported configurations explicitly.
VT-OBS	Trace one run and deployment from initiator through state transitions and audit entries. Verify correlation identifiers, access restrictions, redaction and configured retention expiry.

5.12 Summary

This chapter established performance, security, reliability, usability and other quality requirements together with business rules and verification suites. The next chapter defines information management and acceptance requirements.

Chapter 6 - Other requirements

6.1 Introduction

The preceding chapter established quality and safety expectations. This chapter defines logical information requirements, retention, privacy, recovery, release documentation and acceptance control.

6.2 Logical data requirements

This is a logical information model; database tables and implementation keys are selected during design. All records shall have stable identifiers. Resource ownership and foreign references shall preserve workspace boundaries. Retained histories shall not silently change when a current configuration is edited.

Table 6.1 presents logical information model.

Table 6.1: Logical information model

Entity group	Minimum information and integrity
Identity and access	User identity, Workspace type and owner, WorkspaceMember, Role and Permission. Membership is unique per user and workspace. Project has one workspace owner reference.
Repository and configuration	RepositoryIntegration stores provider identity and installation reference, not plaintext tokens. PipelineConfiguration stores version, stages, triggers and supported toolchain.
Execution evidence	PipelineRun stores trigger, commit, configuration version, times and status. PipelineStageRun records execution outcomes. BuildArtifact stores digest, type, storage reference and originating run.
Security evidence	SecurityFinding stores tool, severity, description and available source location. SecurityPolicy stores version, thresholds and mandatory checks. Preserve scanner execution state separately.
Governance	DeploymentRequest binds evidence, requester, network and inputs. ApprovalDecision references the exact evidence version, reviewer, decision and time.
Deployment and contract	BlockchainNetwork holds network identity and protected provider reference. Deployment stores request, attempt, signer reference, transaction evidence and state. Contract stores scoped network/address identity and deployment linkage.
Monitoring	ContractEvent stores raw and decoded evidence, canonical status and collection metadata. AlertRule is versioned. Alert references its rule version, evidence, state and acknowledgement/resolution history.
Notifications and audit	Notification stores authorized target, event, channel and available status. AuditLog stores actor/process, action, target, time, outcome and correlation reference.
Secrets and optional AI	SecretReference holds an indirect protected credential reference. Optional AIInteraction retains only permitted metadata under the approved privacy and retention policy.

6.3 Data lifecycle and retention

GitHub retains canonical source. Source checkouts and temporary execution files shall be cleaned after completion, cancellation, timeout and recoverable failure. Durable run metadata

and required evidence shall be retained under a defined policy. Large logs and artifacts may use controlled object storage.

CL-DATA-01 [Must] Retention shall be configurable by record category and shall not delete evidence referenced by an active deployment request or needed to reconcile a pending transaction.

CL-DATA-02 [Must] When retained data is archived, expired or deleted under policy, the UI shall distinguish its absence from a successful operation with no findings or events.

Log, artifact, monitoring, audit and backup retention periods remain TBD-07. Workspace deletion, account deletion, anonymization and legal retention handling require the same review; no automatic retention period is assumed.

6.4 Privacy and regulatory scope

The system shall minimize user and repository data collected for its stated purposes, restrict access and avoid secrets in ordinary logs, notifications or external AI input. Data hosting location, organizational obligations and applicable jurisdiction require project review under TBD-11. No legal compliance status is asserted.

6.5 Localization and time representation

English is the proposed initial interface and documentation language. Internal timestamps shall use UTC; displayed timestamps shall identify their timezone. Addresses, hashes and machine identifiers shall retain their original machine-safe representation. Additional language support is outside the proposed MVP unless approved.

6.6 Backup and recovery

Backup frequency, recovery point objective, recovery time objective and storage protection are unresolved operational decisions. Before acceptance, the team shall document and demonstrate restoration of application metadata and required evidence. Restoring the database shall not blindly resubmit transactions or reissue logical alerts; reconciliation against external state is required.

6.7 Configuration and release documentation

The release record shall identify supported toolchain and provider versions, deployment environment, configuration variables, secret provisioning procedure, migrations and known

limitations. Connection checks and unsupported-option validation shall precede execution. The team shall retain a reproducible demonstration using the approved test-network profile.

6.8 Acceptance and change control

Baseline approval requires closure or explicit deferral of blocking decisions in Appendix C. Product acceptance requires mapped evidence for every Must requirement and every selected conditional requirement, with no unresolved defect that permits unauthorized access, exposes deployment secrets, bypasses mandatory gates or falsely reports transaction success.

Each test record shall include requirement ID, fixture, preconditions, steps, expected result, actual result, environment, date, evidence and defect reference where applicable. Tests in this document are planned specifications, not completed test results. Mock providers may verify failure paths; the core integration demonstration must also use the selected real test environment.

Changes shall retain stable identifiers and record rationale, requester, scope impact, approvals and affected UI, schema, implementation and tests. New scope must not silently enter the MVP through an implementation choice. Superseded requirements remain traceable in version history.

6.9 Summary

This chapter defined information management, retention, privacy, recovery and change control. The references and appendices provide terminology, analysis models, traceability and the decisions that remain to be confirmed.

References

[1] Pasted text(2).txt, supplied ChainOps SRS details, version 1.0 draft dated 19 October 2026. Primary source for product scope, roles and requirement IDs. Its candidate technologies and unresolved choices retain proposed status.

[2] demo1.pdf, section 5 Software Requirement Specification Document. Used for document organization: introduction, overall description, interfaces, feature descriptions with stimulus and response sequences, quality requirements and appendices. Storage-platform features in that example do not apply to ChainOps.

Appendix A Glossary

Table A.1 presents glossary of terms.

Table A.1: Glossary of terms

Term	Meaning
Smart contract	Code deployed to a blockchain and executed according to that network's rules.
CI/CD	Continuous Integration and Continuous Delivery/Deployment.
DevSecOps	Integrating security checks and practices into the software delivery lifecycle.
GitHub App	An integration that can receive authorized access to repositories and repository events.
Webhook	An HTTP event notification sent by one system to another.
Pipeline	A sequence of automated build, test, security and deployment-related stages.
Runner / worker	An execution environment that performs pipeline tasks.
Artifact	A build output, such as compiled contract metadata or bytecode.
Security finding	A reported potential vulnerability or policy violation.
RBAC	Role-Based Access Control.
Approval gate	A rule that must be satisfied before a protected operation can proceed.
RPC	Remote Procedure Call; used here for communication with a blockchain node/provider.
Transaction hash	Identifier for a submitted blockchain transaction.
Contract address	Address identifying a deployed contract on a specific blockchain network.
Audit log	A record of important actions and state changes.
Idempotency	A property that prevents repeated processing from unintentionally duplicating an operation.
EVM	Ethereum Virtual Machine, the execution environment used by Ethereum and compatible networks.
SRS	Software Requirements Specification.
SDLC	Software Development Life Cycle.
ABI	Application Binary Interface used to interpret contract calls and event data for a compatible contract.
Canonical event	An event retained in the currently accepted chain history under the configured confirmation policy.
Reorganization	A change to the accepted chain history that can replace previously observed blocks and their events.
Artifact digest	A content-derived identifier used to bind reviewed and deployed build output.

Confirmation policy	The configured evidence and chain-depth criteria required before presenting a deployment as confirmed.

Appendix B Analysis models and traceability

B1 Logical relationships

Workspace has zero or more Projects and Memberships; each Project belongs to one Workspace. A Membership connects one User to one Workspace. A Project has configurations and runs; a Run has stage results, findings and artifacts. A DeploymentRequest refers to one evidence version and target network, with zero or more ApprovalDecisions. A Deployment is an identifiable execution attempt for a request; successful attempts may create Contract records. Contracts have observed events; versioned AlertRules produce Alerts tied to their evidence.

These relationships guide UML and ER design without prescribing class names or physical database structure. Approval workflow state is separate from transaction state so an approved request cannot be mistaken for an on-chain success.

B2 State transition model

Table B.1 presents state transition model.

Table B.1: State transition model

Object	Allowed lifecycle and guards
PipelineRun	QUEUED to RUNNING; RUNNING to SUCCEEDED, FAILED, TIMED_OUT or CANCELLED if cancellation is enabled. QUEUED may be cancelled if supported. SUCCEEDED requires all mandatory stages to pass. A retry is a new attributable run or attempt.
DeploymentRequest	DRAFT to PENDING_CHECKS; valid evidence to PENDING_APPROVAL or READY when approval is not required. Required approvals produce READY. Failed gates produce BLOCKED; rejection produces REJECTED. READY is claimed as EXECUTING and then COMPLETED or NEEDS_RECONCILIATION according to the transaction result. Material changes invalidate readiness. Cancellation is allowed only before an external submission may have occurred.
Deployment attempt	PREPARING to SUBMITTING; known hash to PENDING; successful receipt and confirmation policy to CONFIRMED; unsuccessful receipt to REVERTED. Definite pre-submission failure is SUBMISSION_FAILED. Ambiguity is UNKNOWN; waiting beyond the configured threshold is CONFIRMATION_TIMEOUT, not proof of failure. Unknown

	and timed-out attempts return to evidence-based states through reconciliation.
Monitoring	ACTIVE, PAUSED, STALE or UNAVAILABLE according to user control, checkpoint freshness and provider reachability. These states describe observation capability, not proof of contract safety.
Alert	OPEN to ACKNOWLEDGED to RESOLVED. Each manual transition requires permission, actor and timestamp. A resolved alert retains its history; a new occurrence is a separate alert.

The expanded request and transaction states above clarify the combined illustrative state chain in [1]. They are a proposed model for team approval. Blockchain reorganization can require correction of previously confirmed status; such corrections preserve history and trigger reconciliation rather than erasing evidence.

B3 Functional traceability matrix

Table B.2 presents functional requirements traceability.

Table B.2: Functional requirements traceability

Requirement range	Section	Scenario	Model focus
FR-AUTH-01 to FR-AUTH-07	4.2	AC-AUTH	Identity and session
FR-WS-01 to FR-WS-08	4.3	AC-WS	Workspace and membership
FR-GH-01 to FR-GH-10	4.4	AC-GH	Repository integration
FR-PIPE-01 to FR-PIPE-12	4.5	AC-PIPE	Run and stage
FR-BUILD-01 to FR-BUILD-08	4.6	AC-BUILD	Build artifact
FR-SEC-01 to FR-SEC-09	4.7	AC-SEC	Finding and policy
FR-GOV-01 to FR-GOV-12	4.8	AC-GOV	Request and decision
FR-DEP-01 to FR-DEP-12	4.9	AC-DEP	Deployment and network
FR-REG-01 to FR-REG-06	4.10	AC-REG	Contract
FR-MON-01 to FR-MON-12	4.11	AC-MON	Event rule and alert
FR-NOT-01 to FR-NOT-05	4.12	AC-NOT	Notification
FR-AUD-01 to FR-AUD-08	4.13	AC-AUD	Audit log

FR-DASH-01 to FR-DASH-06	4.14	AC-DASH	Authorized summaries
FR-AI-01 to FR-AI-06	4.15	AC-AI	Optional AI interaction

B4 Quality and clarification coverage

Table B.3 presents quality and clarification verification coverage.

Table B.3: Quality and clarification verification coverage

Requirements	Required verification coverage
NFR-PERF-01 to NFR-PERF-04	VT-PERF in section 5.11
NFR-SEC-01 to NFR-SEC-10	VT-SEC in section 5.11
NFR-REL-01 to NFR-REL-07	VT-REL in section 5.11
NFR-UX-01 to NFR-UX-06	VT-UX in section 5.11
NFR-MNT-01 to NFR-MNT-06	VT-MNT in section 5.11
NFR-COMP-01 to NFR-COMP-04	VT-COMP in section 5.11
NFR-OBS-01 to NFR-OBS-05	VT-OBS in section 5.11
CL-UI-01 to CL-UI-02	VT-UX and AC-GOV: state labels, errors and visible release evidence.
CL-IF-01 to CL-IF-03	VT-SEC, VT-COMP and interface contract review: scope, errors, pagination and failure policies.
CL-GOV-01	AC-GOV and AC-DEP: input mutation, permission change and concurrent submission.
CL-DEP-01	AC-DEP and VT-REL: receipt, confirmation threshold and simulated reorganization.
CL-MON-01 to CL-MON-08	AC-MON and VT-REL: decode failure, raw preservation, checkpoint recovery, duplicate ingestion, noncanonical event correction and coverage labels.
CL-SAFE-01 to CL-SAFE-04	AC-SEC, AC-GOV, AC-DEP and VT-SEC: failed gates, persistent on-chain effects, signer separation and ambiguous retries.
CL-DATA-01 to CL-DATA-02	VT-OBS and VT-REL: retention exclusions, expiry labels and reconciliation after restoration.
BR-01 to BR-10	AC-WS, AC-GOV, AC-DEP, AC-MON and AC-AUD plus concurrency and atomic audit integration tests.

B5 End to end acceptance sequence

Using the approved test profile: sign in, create a workspace and project, connect a permitted repository, process a verified event, compile and test, scan, inspect policy outcomes, request deployment, obtain required approval, submit, observe pending then evidence-based

confirmation, open the contract registry, retrieve a known emitted event, trigger and acknowledge an alert, and inspect the corresponding audit history.

Repeat relevant steps with invalid permissions, duplicate events, mandatory stage failure, a scanner error, prohibited self-approval, changed evidence, an ambiguous submission, provider outage and event reorganization. Expected results are denial, explicit failure or reconciliation as defined above, with no false success or duplicate side effect.

Appendix C Decisions to be determined

The following are explicit open decisions, not empty requirements. The document owner shall record each selected value, decision date, rationale and reviewer in the baseline review record. Suggested defaults are recommendations only. Closure is required before acceptance of the affected capability.

Table C.1 presents decisions requiring confirmation.

Table C.1: Decisions requiring confirmation

Decision	Selection or confirmation needed	Affected scope
TBD-01 Network	Select supported chain and test-network identity, confirmation depth and reorganization handling policy.	Deployment, registry, monitoring
TBD-02 Toolchain	Select Hardhat, Foundry or another supported toolchain, compiler versions, artifact format and mandatory stages.	Pipeline and build
TBD-03 Signing	Select restricted backend signer or user-managed signing; define key lifecycle, nonce/concurrency handling and recovery.	Deployment and security
TBD-04 Runner	Select isolation and queue model; approve CPU, memory, duration, network and concurrency limits.	Pipeline and security
TBD-05 Authentication	Select identity mechanism, session lifetime, revocation and applicable recovery and stronger authentication.	Identity and access
TBD-06 Governance	Select scanners, severity gates, approval quorum and personal/team self-approval defaults. Recommended team default is disabled; mandatory scanner errors block.	Security and approval
TBD-07 Retention	Approve category-specific log, artifact, event, audit and backup retention; archive and deletion policies.	Data lifecycle

TBD-08 Environment	Approve hosting, browser matrix, load profile, resource sizing and performance target measurement.	Quality acceptance
TBD-09 Monitoring	Select supported events, ABI acquisition, transaction coverage, polling/subscription method, freshness limit, supported rule predicates and deduplication details.	Monitoring and alerts
TBD-10 Delivery and AI	Select optional external notification channels and retry policy; decide whether AI is included and approve provider/data handling if enabled.	Optional capabilities
TBD-11 Privacy and recovery	Approve data location and applicable privacy obligations; set backup interval, recovery objectives and restore responsibilities.	Operational acceptance
TBD-12 Interface contracts	Finalize webhook types, API versions, timeouts, provider limits, integration failure codes and compatibility tests.	Integration acceptance

C1 Baseline consistency decisions

The supplied text contains several Must-priority requirements phrased with should. FR-PIPE-12, FR-GOV-11, FR-DEP-12 and FR-MON-12 are normalized to shall to match their stated priority. Optional requirements retain optional priority. The combined example deployment lifecycle is separated into request and transaction models in Appendix B. AI remains conditional. Team-member numbering is deliberately outside the SRS because allocation is not a product requirement and must be confirmed in the project plan.

The baseline may be approved once the team confirms MVP scope, resolves blocking selections and accepts measurable test conditions. Optional features may be explicitly deferred. Approval shall reference this document version and the recorded decisions; subsequent changes follow section 6.8.